SchoolWay: Parent Monitoring System
for Student Bus Boarding Using NFC and Flutter App

By
Chai Boon Hong

A PROPOSAL
SUBMITTED TO
Universiti Tunku Abdul Rahman
in partial fulfilment of the requirements
for the degree of
BACHELOR OF INFORMATION TECHNOLOGY (HONOURS)
COMMUNICATIONS AND NETWORKING
Faculty of Information and Communication Technology
(Kampar Campus)

FEBRUARY 2026

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my supervisor, **Miss Tan Lyk Yin**, for her guidance, patience, and continuous support throughout the completion of this project. Her advice and encouragement have helped me a lot in developing this project and deepening my understanding of system development, particularly in school transportation management systems. I am truly thankful for the opportunity and support that she has given me throughout this journey.

I would also like to give my heartfelt thanks to Liew Yi Mei for her patience, understanding, and unconditional support during both the good and difficult times. Her encouragement has meant a lot to me throughout the completion of this project.

Finally, I would like to thank my parents and family for their endless love, support, and encouragement. Their belief in me has given me the strength and motivation to complete this project successfully.

COPYRIGHT STATEMENT

© 2026 Chai Boon Hong. All rights reserved.

This Final Year Project proposal is submitted in partial fulfilment of the requirements for the degree of Bachelor of Information Technology (Honours) Communications and Networking at Universiti Tunku Abdul Rahman (UTAR). This Final Year Project proposal represents the work of the author, except where due acknowledgment has been made in the text. No part of this Final Year Project proposal may be reproduced, stored, or transmitted in any form or by any means, whether electronic, mechanical, photocopying, recording, or otherwise, without the prior written permission of the author or UTAR, in accordance with UTAR's Intellectual Property Policy.

ABSTRACT

School transportation management in Malaysia remains heavily reliant on manual processes, leaving parents without real-time visibility into their children's whereabouts and exposing students to safety risks during transit. Existing school bus monitoring solutions mainly focus on GPS-based location tracking, but often lack integrated attendance verification, role-based communication, and school-specific operational workflows. As a result, coordination among parents, drivers, and school administrators remains fragmented, leading to issues such as missed pickups, delayed updates, and uncertainty over whether students have boarded or alighted safely. The objective of this project is to develop SchoolWay, a mobile-based school bus tracking and management system that provides real-time GPS tracking, automated attendance recording via NFC scanning, and instant push notifications to bridge the communication gap between parents, drivers, and administrators. Unlike existing systems that typically provide only location tracking, SchoolWay combines real-time GPS tracking, NFC-based attendance recording, and instant push notifications in a unified platform with role-based access for parents, drivers, and administrators. SchoolWay is built using Flutter for cross-platform mobile development, Firebase as the backend for real-time data synchronisation and authentication, and Google Maps API for live bus location rendering. The system's success will be evaluated through functional testing, user acceptance testing (UAT) with target users, and performance benchmarking of real-time update latency. This project contributes a practical, scalable solution to a gap in school safety infrastructure, empowering parents with transparency, reducing administrative overhead, and ultimately enhancing the safety and accountability of school bus operations in Malaysia.

Area of study: Mobile Application Development, NFC Technology Systems, Real-Time System

Keywords: School Bus Tracking System, Real-Time GPS Tracking, Flutter, Firebase, NFC Technology, Push Notifications, Mobile Application, Role-Based Access Control

TABLE OF CONTENTS

TITLE PAGE ..... **I**
ACKNOWLEDGEMENTS ..... **II**
COPYRIGHT STATEMENT ..... **III**
ABSTRACT ..... **IV**
TABLE OF CONTENTS ..... **V**
LIST OF FIGURES ..... **VIII**
LIST OF TABLES ..... **XI**
LIST OF ABBREVIATIONS ..... **XII**
CHAPTER 1 ..... **1**
1.1 Project Background 1
1.2 Problem Statement and Motivation 2
1.3 Project Objectives 4
1.4 Project Contribution 6
1.5 Project Scope 7
1.6 Report Organization 9
CHAPTER 2 ..... **10**
2.1 Review of Technologies 10
2.1.1 Field of Study: School Transportation Management 10
2.1.2 Mobile Platforms and Development Framework 11
2.1.3 Student Attendance and Identification Technologies 15
2.1.4 Real-Time Tracking Technologies 19
2.1.5 Database and Backend Services 22
2.1.6 Map and Notification Services 25
2.2 Review of Existing Systems / Applications 29
2.2.1 Existing System A: School Bus Tracking Application 29
2.2.2 Existing System B: Student Attendance Monitoring System 30
2.2.3 Existing System C: Integrated School Transport Management System 31
2.2.4 Proposed System and Summary 32
2.3 Chapter Summary 33
CHAPTER 3 ..... **34**
3.1 System Development Models 34
3.1.1 System Development Model 1: Waterfall Model 35

3.1.2 System Development Model 2: Agile Model 36
3.1.3 System Development Model 3: Prototype Model 37
3.1.4 Selected Model: Prototype-Driven Agile Development 38
3.2 System Requirement 39
3.2.1 Introduction 39
3.2.2 Hardware Components 39
3.2.3 Software Components 42
3.3 System Design 46
3.3.1 Introduction 46
3.3.2 High-Level Architecture Components 47
3.3.3 Data Flow Summary 48
3.3.4 Conclusion of Design 48
3.4 Functional Requirements 49
3.4.1 Introduction 49
3.4.2 Functional Requirements Table 49
3.4.3 Requirement Explanation 50
3.5 Project Milestone 51
3.5.1 Project I Timeline 51
3.5.2 Project II Timeline 51
3.6 Concluding Remark 52
CHAPTER 4 53
4.1 Software Setup and Configuration 53
4.1.1 Environment Preparation 53
4.1.2 Integration of External Services 58
4.1.3 Database Configuration 64
4.1.4 Cloud Services 66
4.1.5 Deployment and Hosting 68
4.1.6 Monitoring and Analytics 69
4.2 Implementation Details 70
4.2.1 Authentication and Role Routing 70
4.2.2 Startup Gate and Permission Initialization 71
4.2.3 GPS Tracking Implementation 72
4.2.4 NFC Boarding Event Implementation 73

4.2.5 Live Map Rendering Implementation 74
4.2.6 Notification Pipeline Implementation 76
4.2.7 CSV Import and Bulk Administration Implementation 77
4.3 System Operation 78
4.3.1 Design Layout 78
4.3.2 Functional Operation 80
4.4 Implementation Issues and Challenges 90
4.5 Concluding Remark 91
CHAPTER 5 92
5.1 Project Limitations 93
5.2 Future Work for Project II 94
REFERENCES 96
APPENDIX 101
A.1 Poster 101

LIST OF FIGURES

Figure Number	Title	Page
Figure 3.1.1	Waterfall Model Process Flow	35
Figure 3.1.2	Agile Iterative Sprint Cycle	36
Figure 3.1.3	Prototype Development Cycle	37
Figure 3.1.4	Prototype-Driven Agile Development	38
Figure 3.2.2.1	Developer device specifications	40
Figure 3.2.2.2	Android device settings showing GPS and NFC enabled	40
Figure 3.2.3.1	Flutter SDK and IDE setup	43
Figure 3.2.3.2	Firebase project console	43
Figure 3.2.3.3	App permissions page on Android	44
Figure 3.3.1	System Architecture Diagram	46
Figure 3.3.3	Data Flow Diagram for location and notifications	48
Figure 3.5.1	Project I Timeline in Gantt Chart	51
Figure 3.5.2	Project II Timeline in Gantt Chart	51
Figure 4.1.1.1	Flutter and Dart SDK Setup Verification	54
Figure 4.1.1.2	Flutter and Dart extensions from VS Code Marketplace	55
Figure 4.1.1.3	Android SDK Setup Verification	55
Figure 4.1.1.4	Firebase CLI Setup verification	56
Figure 4.1.1.5	FlutterFire CLI Setup verification	56
Figure 4.1.1.6	Dependency installation completion in project terminal	57
Figure 4.1.2	External service architecture for authentication, messaging, map, location, and NFC	58
Figure 4.1.2.1	Firebase Authentication provider setup screen	59
Figure 4.1.2.2	Cloud Messaging configuration and token registration flow	60
Figure 4.1.2.3	OpenStreetMap through Flutter Map Layer	61
Figure 4.1.2.4	NFC access is usable on supported platforms	63

Figure Number	Title	Page
Figure 4.1.3.1	Firestore collection structure used by SchoolWay	64
Figure 4.1.3.2	Composite index configuration for operational queries	65
Figure 4.1.3.3	Firestore security rule setup in project console	65
Figure 4.1.4.1	Firebase project overview and platform app registration	66
Figure 4.1.4.2	Application startup sequence with cloud initialization gate	67
Figure 4.1.5	Android release build pipeline and signing checkpoint	68
Figure 4.1.6	Admin reporting interface used for operational monitoring	69
Figure 4.2.1	Authentication sequence from credential submission to role-based navigation	70
Figure 4.2.2	Startup gate flow for permission and messaging initialization	71
Figure 4.2.3	GPS upload cycle for active driver session	72
Figure 4.2.4	NFC scan-to-event recording pipeline	73
Figure 4.2.5	Real-time map rendering with route and position overlays	74
Figure 4.2.6	Notification flow from token sync to inbox presentation	76
Figure 4.2.7	CSV import workflow and batch processing feedback	77
Figure 4.3.1.1	Planned Advanced Alert Escalation Dashboard Layout (Admin)	78
Figure 4.3.1.2	Planned Parent Notification Preferences Layout	78
Figure 4.3.1.3	Planned Driver Offline Sync Recovery Layout	79
Figure 4.3.1.4	Planned Analytics Trend Dashboard Layout	79
Figure 4.3.2.1.1	Onboarding entry screen of SchoolWay	80
Figure 4.3.2.1.2	User sign-in interface with authentication options	80
Figure 4.3.2.1.3	Successful authentication and role-based landing view	81
Figure 4.3.2.2.1	Admin Dashboard Main Navigation	82
Figure 4.3.2.2.2	Live Fleet Monitoring Map in Admin Module	82
Figure 4.3.2.2.3	Route Planner Screen with Waypoint Planning Controls	83
Figure 4.3.2.2.4	CSV Importer Screen for Bulk Data Setup	83

Figure Number	Title	Page
Figure 4.3.2.2.5	Emergency Alerts Management Screen	84
Figure 4.3.2.2.6	Administrative Reporting and Analytics Summary screen	84
Figure 4.3.2.3.1	Driver trip management screen before departure	85
Figure 4.3.2.3.2	Active route session state in driver module	85
Figure 4.3.2.3.3	NFC scanning interface for boarding and alighting events	86
Figure 4.3.2.3.4	Driver live map with route and current position	86
Figure 4.3.2.3.5	Assigned student roster with boarding status indicators	87
Figure 4.3.2.4.1	Parent dashboard main interface	87
Figure 4.3.2.4.2	Children overview screen in parent module	88
Figure 4.3.2.4.3	Parent live map showing real-time bus location	88
Figure 4.3.2.4.4	Trip history view for student boarding events	89
Figure 4.3.2.4.5	Parent notification inbox with event updates	89

LIST OF TABLES

Table Number	Title	Page
Table 2.1	Comparison of Android, iOS, and Flutter	14
Table 2.2	Comparison of Manual, QR, RFID, and NFC attendance methods	18
Table 2.3	Comparison of GPS, GSM/SMS, and Wi-Fi/Bluetooth tracking methods	21
Table 2.4	Comparison of SQLite, MySQL, and Firebase	24
Table 2.5	Comparison of map and notification service options	28
Table 2.6	Comparison of existing systems and the proposed SchoolWay system	32
Table 3.1.1	Strengths and Weaknesses of Waterfall Model	35
Table 3.1.2	Strengths and Weaknesses of Agile Model	36
Table 3.1.3	Strengths and Weaknesses of Prototype Model	37
Table 3.2.2	Summary of Hardware Components and Minimum System Requirements	39
Table 3.2.3	Software Requirements for the SchoolWay Mobile Application	42
Table 3.4.2	Functional Requirements Table	49
Table 4.4	Summary of key implementation challenges and mitigation actions	89

LIST OF ABBREVIATIONS

ETA	Estimated Time of Arrival
GPS	Global Positioning System
GPRS	General Packet Radio Service
GSM	Global System for Mobiles Communications
NFC	Near-field Communication
OS	Operating System
RFID	Radio-Frequency Identification
SMS	Short Message Service
UI	User Interface

CHAPTER 1

Introduction

In this chapter, the background and motivation of the research are presented, together with the problem statement, project objectives, scope, and expected contributions of the study. This chapter also introduces the significance of the proposed project in addressing current limitations in school transportation management. Finally, the outline of the thesis is provided to give an overview of the structure and organisation of the subsequent chapters.

1.1 Project Background

School transportation is an essential support service in the education sector because it ensures that students travel safely and punctually between home and school. In many schools, transportation management is still conducted using conventional methods such as manual attendance recording, phone calls, and informal communication between parents, drivers, and school administrators [1] . Although these practices are commonly used, they often lead to several operational limitations, including delayed information sharing, inaccurate attendance records, and the absence of real-time monitoring of school bus movements. Therefore, parents may experience anxiety regarding their children's safety, while schools and drivers may face difficulties in maintaining efficient and accountable transportation operations [2].

Recent developments in mobile computing, cloud-based systems, Global Positioning System (GPS) technology, and Near Field Communication (NFC) have created opportunities to improve school transportation management. A mobile-based platform integrated with GPS can provide real-time bus location tracking, enabling parents and school administrators to monitor journeys more effectively [3] . At the same time, NFC-based attendance recording can automate student check-in and check-out processes, thereby reduce human error and improve data accuracy [4] . Cloud services further support instant data synchronization and notification delivery, allowing important updates to be shared immediately with relevant users [2] . These technologies offer clear benefits to users and administrators by improving transparency, reducing

manual workload, strengthening communication, and supporting safer transportation management [1].

This project proposes SchoolWay as a mobile-based school transportation monitoring system, implemented through a Flutter mobile application for school bus tracking and student attendance management. The proposed system integrates real-time GPS tracking, NFC-based attendance recording, and cloud-based data synchronization to provide a unified platform for parents, drivers, and school administrators. The significance of the system benefits in its potential to replace inefficient manual practices with a more reliable and systematic digital solution. The expected outcome is a school transportation management system that reduces manual attendance errors through NFC-based check-in/check-out, enables real-time bus location visibility via GPS tracking, and provides immediate status updates through cloud-synchronized notifications. These functions improve parents' ability to verify student boarding status, support administrators in monitoring route and attendance records, and help drivers coordinate pickups and drop-offs more accurately.

1.2 Problem Statement and Motivation

School transportation is an essential support service within the education sector, as it ensures that students are conveyed safely, punctually, and consistently between home and school. In parallel with the advancement of digital technologies, parents and school authorities now place greater emphasis on service transparency, timely information, and operational reliability. Nevertheless, many school transportation services continue to depend on conventional and largely manual practices, resulting in a mismatch between parents' expectations and the actual level of service provided. This situation underscores the need to examine the limitations of existing school bus management practices and to justify the development of a more systematic and technology-driven solution [5] .

A major issue in current school transportation management is the absence of real-time visibility of school bus movements. In many cases, parents and school administrators rely on phone calls or text-based communication to obtain updates about bus location, arrival times, or unexpected delays. Such methods are inefficient, inconsistent, and highly dependent on the immediate availability of the driver. This lack of accurate and timely tracking information can create anxiety among parents, reduce

confidence in the transportation service, and limit the ability of school administrators to monitor daily operations effectively [6], [7]. If this issue remains unresolved, the safety and accountability of student transportation may be adversely affected. Therefore, a real-time bus tracking mechanism is necessary to improve transparency, responsiveness, and user trust.

Another significant problem is the continued use of manual methods for recording student attendance during boarding and drop-off. Practices such as handwritten attendance lists or verbal confirmation are susceptible to human error, incomplete documentation, and delayed updates [8]. Therefore, uncertainty may arise regarding whether a student has successfully boarded the bus, arrived at school, or returned home. Such inaccuracies may compromise student safety and lead to misunderstandings between parents, drivers, and school management. If left unaddressed, these weaknesses may undermine confidence in the transportation process and increase administrative difficulties in verifying student movement. Hence, an automated attendance mechanism is important to ensure greater accuracy, reliability, and efficiency in student monitoring.

A further concern is the lack of an integrated communication platform connecting parents, drivers, and school administrators. Existing communication practices are often fragmented, informal, and dependent on individual initiative rather than a centralised system. Consequently, important updates such as route changes, delays, emergencies, or student boarding status may not be communicated promptly or uniformly to all relevant parties [4]. This communication gap can result in confusion, delayed decision-making, and reduced service effectiveness. If this condition persists, coordination among parents and school administrators will remain inefficient, and parental confidence in school transportation services may continue to decline. For this reason, a mobile-based platform with centralised information sharing and instant notifications is essential to support timely communication and more effective coordination.

In summary, the lack of real-time bus tracking, the reliance on manual attendance recording, and the absence of efficient communication mechanisms represent critical weaknesses in existing school transportation management [5]. If these issues are not addressed, student safety may be placed at risk, parents may continue to experience uncertainty and anxiety, and school authorities may face ongoing

operational inefficiencies. These concerns collectively justify the need for a digital and integrated solution. Accordingly, the proposed SchoolWay mobile application is intended to address these limitations by providing real-time tracking, automated attendance management, and improved communication among the users, thereby enhancing the safety, efficiency, and reliability of school transportation services [8].

1.3 Project Objectives

Based on the background and problem statement discussed in the preceding sections, this mobile application establishes a set of objectives to guide the development of a systematic solution for the identified shortcomings in school transportation management. The issues previously highlighted, namely inefficient information sharing among parents and school administrators, dependence on manual student attendance recording, and the lack of real-time visibility of school bus movements, indicate the need for a mobile-based system that can improve operational efficiency, monitoring capability, and student safety [5]. In line with the overall aim of the system, the following objectives have been formulated to provide clear direction for the design and implementation of SchoolWay.

The first objective of this project is to design and develop a mobile-based school transportation monitoring system, through a Flutter mobile application with parent, driver, and administrator modules to support student bus activity monitoring and efficient information sharing among users [9]. This objective is significant because it establishes the core platform through which the needs of the three main user groups can be addressed within a unified mobile application. By providing dedicated modules for parents, drivers, and administrators, the application is expected to improve information accessibility and support real-time communication through centralized in-app updates and instant notifications on delays, route changes, and student boarding status, thereby strengthening stakeholder coordination. The successful achievement of this objective can be demonstrated through the completion of a functional mobile-based school transportation monitoring system, through a Flutter mobile application containing the specified user modules and essential features within the defined project scope.

The second objective of this project is to implement and validate NFC-based automated student identification for real-time boarding and exiting event logging [10]. This objective directly addresses the limitation of manual attendance recording, which

is often associated with inaccurate, incomplete, or delayed records. Through the application of NFC technology, student identification can be performed in a more efficient and reliable manner, thereby enabling the automatic recording of boarding and exiting events during daily transportation operations. The accomplishment of this objective may be measured through the system's ability to capture and store attendance events accurately, as well as through validation testing to verify the effectiveness and consistency of the NFC-based identification mechanism.

The third objective of this project is to develop and integrate a GPS-based real-time bus tracking module into the mobile application, providing continuous location updates to parents and administrators [5]. This objective is essential in addressing the lack of real-time visibility of school bus movements, which often contributes to parental uncertainty and difficulties in administrative supervision. By integrating GPS tracking functionality into the application, the mobile application is expected to provide timely and continuous bus location information, thereby enhancing transparency and improving the overall safety and accountability of school transportation services. The achievement of this objective can be evaluated through the successful implementation of live location updates and the accurate display of bus movement information within the application interface.

In conclusion, the objectives of this project provide a clear and structured framework for the development of SchoolWay as a mobile-based school transportation management solution. Collectively, these objectives address the principal operational and safety issues identified in the earlier sections and reflect the intended contribution of the mobile application to supporting real-time bus location tracking, automated NFC-based attendance logging for student boarding and exiting, and timely communication among stakeholders through instant notifications. Furthermore, the objectives are designed to be specific, measurable, achievable, relevant, and realistic within the allocated mobile application duration, thereby ensuring their suitability for the final year project report.

1.4 Project Contribution

The proposed SchoolWay mobile application contributes to the improvement of school transportation management by introducing a mobile-based digital solution that addresses the key limitations of existing manual practices. In particular, the mobile application is intended to enhance communication among users, improve the accuracy of student attendance recording, and provide real-time visibility of school bus movements [6]. By aligning its contributions with the identified problem statements and project objectives, SchoolWay is expected to offer practical value to parents, drivers, and school administrators while supporting safer and more efficient transportation operations.

The first contribution of this mobile application is the provision of an integrated mobile application with dedicated parent, driver, and administrator modules. This contribution directly addresses the problem of inefficient information sharing and fragmented communication among users [5]. Through a unified platform, relevant users can access student bus activity information in a more organised and timely manner. As a result, communication delays can be reduced, coordination can be improved, and parent and school administrator can rely on a more systematic method of receiving and sharing transportation-related updates. This contributes to greater operational efficiency and strengthens confidence in the management of school bus services.

The second contribution of the mobile application is the implementation of NFC-based automated student identification for real-time boarding and exiting event logging [11]. This contribution is significant in addressing the reliance on manual attendance recording, which is often associated with inaccuracies, incomplete records, and difficulties in verification. By automating the identification process, the mobile application can generate more reliable attendance data and reduce the likelihood of human error during daily transit operations. The expected benefit of this contribution is improved accuracy in student movement records, which in turn supports better safety monitoring, easier record validation, and reduced administrative burden for drivers and school staff [6].

The third contribution of the mobile application is the integration of a GPS-based real-time bus tracking module into the mobile application. This contribution responds directly to the problem of limited real-time visibility of school bus locations [12]. Through continuous location updates, parents and school administrators can

monitor bus movements more effectively and obtain timely information regarding travel progress, delays, or route status. This improves transparency in school transportation services and helps reduce parental anxiety concerning student travel. In addition, real-time tracking supports stronger accountability and enables more effective oversight of transportation operations by school authorities [5].

In summary, the contributions of SchoolWay include improvements in the communication between parents and school administrators through an integrated mobile platform, increasing the reliability of attendance management through NFC-based automation, and enhancing transportation visibility through GPS-based real-time tracking. Collectively, these contributions address the main weaknesses identified in the problem statement and support the overall aim of the mobile application, which is to provide a safer, more efficient, and more transparent school transportation management system [11]. Through these contributions, the mobile application is expected to deliver both practical and academic value within the context of mobile-based school transport solutions.

1.5 Project Scope

The project scope defines the boundaries of the proposed mobile application by identifying what is included and excluded from development. A clear scope ensures the mobile application remains focused, achievable within the allocated timeline, and aligned with the project objectives. For SchoolWay, the scope is centred on a mobile-based school transportation monitoring system developed using Flutter. The mobile application addresses key issues such as inefficient communication between parents and school administrators, manual student attendance recording, and limited visibility of school bus movement.

Inclusions

1. This project develops a Flutter-based mobile application as the primary platform for the SchoolWay system.
2. The mobile application provides three user roles, namely parent, driver, and administrator, with role-based access and module-specific functions.
3. The application supports student transportation activity monitoring, including attendance and trip-related status information.

4. The system includes an NFC-based student identification workflow for recording boarding and alighting events within the application data flow.
5. The mobile application supports GPS-based bus location tracking with real-time location display for relevant users through Firebase-backed updates.
6. The system enables real-time information sharing among users through a centralized cloud-connected mobile platform.
7. The system stores and manages data digitally, including student records, boarding event logs, route sessions, emergency alerts, and GPS tracking data.
8. The system includes functional testing and validation through defined test guides and checklists to verify core module behaviour.

Exclusions

1. The mobile application does not include online payment gateway integration, as financial transaction processing is outside the core transportation monitoring objective.
2. The mobile application does not include AI-based prediction or analytics features such as route optimization, forecasting, or delay prediction.
3. The mobile application does not include web deployment as a target deliverable; the system scope is limited to mobile use despite framework-generated web scaffolding.
4. The mobile application does not include multi-school or large-scale fleet management; it is limited to a prototype context suitable for the system setting.
5. The mobile application does not include advanced communication features such as in-app chat, chatbot support, or voice-based communication.
6. The mobile application does not include custom hardware development and relies on existing mobile device capabilities such as NFC and GPS.
7. The mobile application does not cover full live production rollout; the emphasis is on design, development, integration, and prototype-level validation within the project duration.

In summary, SchoolWay is scoped as a mobile-based school transportation monitoring system, implemented through a Flutter mobile application with parent, driver, and administrator modules, NFC attendance workflow, and GPS tracking

support. The scope is intentionally constrained to core monitoring and attendance objectives while excluding advanced, non-essential, or large-scale production features to keep development manageable and aligned with the project timeline.

1.6 Report Organization

This report is systematically organized into five interconnected chapters to present the study. Chapter 1 establishes the foundation by introducing the project background, problem statement, and objectives, ultimately defining the study's scope and significance. Building upon this context, Chapter 2 reviews relevant literature and existing technologies, including GPS tracking and NFC identification, to pinpoint the specific gaps that justify the proposed SchoolWay application. To address these identified gaps, Chapter 3 details the system methodology, outlining the chosen development model, system requirements, and the targeted milestones for both Project I and Project II. Following the methodology phase, Chapter 4 presents the preliminary development work by highlighting the software setup, current prototype functionality, and any technical challenges encountered. Finally, Chapter 5 concludes the report by summarizing the overall project outcomes, evaluating the achievement of the initial objectives, and offering recommendations for future improvements.

CHAPTER 2

Literature Reviews

2.1 Review of Technologies

This section reviews the technologies and technical approaches relevant to the development of **SchoolWay**, a mobile-based school transportation monitoring system implemented through a Flutter mobile application. The review covers the application domain, mobile platforms and development framework, attendance and identification technologies, real-time tracking methods, database and backend services, and map and notification services. Each technology is introduced and evaluated in terms of its strengths and weaknesses to justify the selections made for the proposed mobile application. Through this review, the chapter establishes the technical foundation of SchoolWay and explains why specific technologies are more suitable than their alternatives for the intended project scope.

2.1.1 Field of Study: School Transportation Management

School transportation management refers to the planning, monitoring, and administration of student transport services between home and school. It is a critical operational domain because it directly affects student safety, punctuality, and the overall confidence of parents and school administrators in school transport services [13]. In many educational environments, especially where students depend on buses or vans for daily travel, transportation is not merely a logistical service but an extension of the school's duty of care. Effective transportation management contributes to timely arrivals, reduced parental uncertainty, improved route coordination, and better accountability in student movement [14].

Traditionally, school transportation management has relied heavily on manual practices. Parents often depend on phone calls or informal messaging to confirm whether a bus has arrived, while drivers or attendants may record attendance manually using paper-based lists or verbal checks. These approaches are simple to implement, but they are also limited in accuracy, speed, and transparency [15]. Manual records are vulnerable to miss and human error, and conventional communication methods do not

provide real-time visibility into bus location or student boarding status [16]. As a result, delays, uncertainty, and poor coordination can arise, particularly when the transport schedule changes unexpectedly.

The growing availability of smartphones, mobile internet access, and cloud-based systems has created a strong case for digital transformation in this field. A mobile-based school transportation system can centralize communication, automate attendance capture, and provide live location updates to relevant users [16]. Such a solution is particularly valuable because it serves the needs of multiple user groups simultaneously, including parents, drivers, and school administrators. Therefore, the field of school transportation management provides an important and practical context for the development of SchoolWay as a digital mobile application that addresses operational inefficiencies and enhances student safety.

2.1.2 Mobile Platforms and Development Framework

Mobile applications are typically delivered through operating systems such as Android and iOS or built using cross-platform frameworks that allow a single codebase to target multiple operating systems. The choice of platform and framework significantly affects project cost, development speed, hardware access, user reach, maintainability, and long-term scalability. Since SchoolWay is intended to provide mobility, real-time updates, and interaction with device features such as GPS and NFC, the selection of an appropriate mobile platform and development framework is a critical architectural decision.

Native development on Android or iOS offers deep access to device features and high platform-specific performance. However, building separate native applications for each platform often increases the development workload and introduces duplication in implementation and maintenance. Cross-platform frameworks, by contrast, aim to reduce development overhead while keeping acceptable performance and access to native capabilities [17]. In the context of a university project with a defined scope and duration, a cross-platform framework is often more practical if it can support the required device integrations.

2.1.2.1 Android

Android is the most widely adopted mobile operating system globally and is known for its open ecosystem, broad device support, and strong integration with mobile hardware features. From a development perspective, Android provides direct access to sensors and communication capabilities such as GPS, Bluetooth, NFC, and notifications, making it well suited for location-aware and identification-based applications.

A major strength of Android is its flexibility. Developers can deploy applications across a wide range of device types and price points, which improves accessibility for both institutional users and individual users. Android also offers comparatively fewer restrictions on hardware access than some competing platforms, which is beneficial for applications that rely on real-time tracking and NFC interactions. In addition, many lower-cost smartphones used in operational settings, such as devices assigned to drivers, are Android-based.

Despite these strengths, Android also presents weaknesses. Device fragmentation is a persistent challenge because different manufacturers and operating system versions may behave differently, increasing testing and maintenance effort [18]. User interface consistency can vary across devices, and application behaviour may be affected by battery optimization settings or manufacturer-specific restrictions. As a result, Android is highly capable but can require careful compatibility management [19].

2.1.2.2 iOS

iOS is Apple's mobile operating system and is recognized for its high performance, polished user experience, strong security model, and consistent device ecosystem. For application developers, iOS offers stable platform behaviour, predictable hardware-software integration, and high standards of usability and interface quality [20].

One of the main strengths of iOS is platform consistency. Since Apple controls both the hardware and operating system, applications can be optimized for a relatively limited range of devices. This reduces fragmentation and can simplify quality assurance. iOS also has a strong reputation for security and privacy, which is valuable in applications handling student-related data and location information [21].

However, iOS is less flexible than Android in several respects. The ecosystem is closed, development and testing require Apple-specific tools and hardware, and access to certain device functions may be more tightly controlled. In addition, iOS

devices are often more expensive, which can limit accessibility for deployment scenarios that depend on dedicated operational devices. For features such as NFC, support is available but can be more constrained depending on the use case and the device environment. Consequently, iOS is robust and secure, but it may not be the most practical primary target in a cost-sensitive project requiring broad hardware flexibility.

2.1.2.3 Flutter

Flutter is a cross-platform application development framework developed by Google. It enables developers to build mobile applications using a single codebase, primarily in the Dart programming language, and deploy them across platforms such as Android and iOS. Flutter has gained popularity because it combines cross-platform productivity with strong interface rendering performance and an expressive widget-based design model [22].

The primary strength of Flutter is development efficiency. A shared codebase reduces duplication across platforms, shortens implementation time, and simplifies maintenance. This is particularly helpful in a final year project, where development time is limited and the emphasis is on delivering a functional system prototype. Flutter also offers consistent UI rendering across devices and a rich library of widgets that support the development of modern, responsive user interfaces. In addition, Flutter can access device capabilities such as GPS, networking, notifications, and NFC through plugins and platform channels [23].

Its weaknesses are mainly related to ecosystem dependence and feature-specific integration. Although Flutter supports many mobile capabilities, some advanced device functions may depend on third-party plugins or platform-specific adjustments. Application size can also be larger compared to some native implementations, and troubleshooting platform-specific issues may still be necessary when using hardware-dependent features. Nevertheless, these limitations are generally manageable within the scope of SchoolWay.

2.1.2.4 Summary

The comparison of Android, iOS, and Flutter shows that each option offers distinct advantages. Android is highly flexible and provides broad access to hardware features, but device fragmentation increases maintenance complexity. iOS offers strong

consistency, security, and performance, but it is more restrictive and can raise development and deployment costs. Flutter, while not an operating system, provides a practical cross-platform framework that allows a single codebase to serve multiple mobile environments while still supporting the required features through available plugins and integrations.

Table 2.1 Comparison of Android, iOS, and Flutter

Technology	Type	Strengths	Weaknesses
Android	Mobile operating system	Broad market reach, flexible hardware access, strong support for GPS and NFC, cost-effective device ecosystem	Device fragmentation, inconsistent behaviour across manufacturers, higher compatibility testing effort
iOS	Mobile operating system	Stable ecosystem, strong security, consistent performance, polished user experience	Closed ecosystem, higher device cost, more restricted hardware flexibility in some cases
Flutter	Cross-platform framework	Single codebase, faster development, consistent UI, good performance, plugin support for mobile features	Reliance on plugins, platform-specific tuning may still be needed, larger app size in some cases

In summary, **Flutter** is selected for SchoolWay because it offers the best balance between development efficiency, cross-platform capability, maintainability, and functional suitability [24] . Since the system requires a unified application with parent, driver, and administrator modules, Flutter supports rapid development of a single mobile solution without the need to maintain separate native applications. It also provides sufficient integration capability for GPS, Firebase, notifications, and NFC-based attendance features, making it the most appropriate framework for the system.

2.1.3 Student Attendance and Identification Technologies

Student attendance and identification are central to the SchoolWay mobile application because the project aims to automate the recording of boarding and exiting events [4]. In transportation settings, an effective attendance technology must be accurate, fast, easy to use, and practical in real-world boarding conditions. Several approaches can be considered, ranging from purely manual methods to contactless digital identification mechanisms such as QR codes, RFID, and NFC [7].

The selection of an identification technology should consider more than technical functionality alone. It should also reflect operational constraints such as scanning speed, physical proximity, reliability in mobile environments, accidental misreads, ease of deployment, and suitability for school-aged users. For this reason, a comparison of alternative attendance approaches is necessary before selecting the most appropriate method for SchoolWay.

2.1.3.1 Manual Attendance Recording

Manual attendance recording typically involves checking student names from a printed list, marking attendance by hand, or using verbal confirmation during boarding and drop-off. This method has been widely used because it is simple, requires no specialized equipment, and can be implemented immediately without software infrastructure [4].

Its primary strength lies in low cost and ease of use. Since it does not depend on network access, mobile devices, or scanning technology, manual attendance can be performed in nearly any environment. It also allows human judgment in situations where student identity needs to be confirmed directly.

However, the weaknesses of manual attendance are large. This approach is prone to human error, illegible records, delayed updates, and inconsistent information [4] . It is also difficult to share in real time with parents or administrators and creates unnecessary administrative workload for drivers or transport staff. In the context of school transportation, where timely and accurate confirmation is important, manual attendance is inadequate for scalable and accountable operations [4], [7].

2.1.3.2 QR Code-Based Attendance

QR code-based attendance uses printed or digital QR codes that are scanned by a mobile device camera to identify a student and record an event [15]. This method is widely

used in educational and event management systems because QR codes are inexpensive to generate and easy to distribute [15].

The strengths of QR code systems include low implementation cost, minimal hardware requirements beyond a smartphone camera, and ease of integration into software applications. QR codes can also encode unique identifiers and work well for check-in workflows where controlled scanning conditions are available [15].

Nonetheless, QR-based attendance has practical weaknesses in transportation environments. Camera-based scanning may be slower than tap-based approaches, especially when lighting conditions are poor, camera focus is inconsistent, or printed codes are damaged [15]. Students may also need to position their code correctly, which can create queueing during boarding. In addition, QR codes can be shared or duplicated more easily than short-range tap interactions, which may reduce security in some situations. Therefore, while QR codes are practical in many settings, they are not always ideal for fast and consistent bus boarding workflows.

2.1.3.3 RFID-Based Attendance

RFID-based attendance relies on radio frequency identification tags and readers to detect a student's presence or identity [7]. RFID systems are widely used in access control, inventory management, and automated check-in scenarios. In school environments, RFID cards or tags can be assigned to students and detected by a reader mounted near an entrance [7].

A key strength of RFID is its speed and contactless operation. It can support automated identification with minimal user interaction and is effective in environments where repeated, routine scanning is required. Depending on the type of RFID system used, tags can be read quickly and from varying distances [7].

However, RFID also has limitations. Dedicated RFID readers are typically required, increasing hardware cost and integration complexity [7]. Longer reading ranges, while useful in some contexts, may lead to unintended reads if multiple tags are detected at once. This can reduce precision in transportation use cases where intentional confirmation of boarding or exiting is preferred [7]. RFID systems may therefore be effective but less practical for a mobile-app-centred system that aims to leverage standard smartphone capabilities rather than additional hardware infrastructure.

2.1.3.4 NFC-Based Attendance

NFC-based attendance uses Near Field Communication to identify a student when an NFC-enabled tag or card is tapped against a compatible mobile device [25], [26]. NFC is a short-range communication technology designed for close-proximity interaction, which makes it suitable for intentional, secure, and quick event logging [26].

Its strengths are closely aligned with the needs of SchoolWay. NFC interactions are fast, simple, and user-friendly, requiring only a short intentional tap [25] . The short communication range reduces the likelihood of accidental reads and supports clearer event confirmation than longer-range methods [26]. NFC can also be implemented using widely available Android devices equipped with NFC capability, which reduces the need for separate dedicated readers in a prototype environment [25]. In a boarding process, this makes NFC efficient and operationally suitable [25].

The weaknesses of NFC include dependency on compatible devices and the need to manage tags or cards assigned to students. Some mobile platforms or device models may impose restrictions on NFC usage, and deployment must account for hardware compatibility. Even so, in a system focused on controlled attendance capture during transport, NFC provides a strong balance between usability, accuracy, and implementation feasibility [26].

2.1.3.5 Summary

The comparison of attendance technologies shows that manual attendance is the least efficient and least reliable option, despite its simplicity. QR code-based attendance is inexpensive and easy to deploy, but camera-based interaction can slow down boarding and is affected by environmental conditions. RFID offers fast contactless identification but often requires dedicated reader hardware and may be less precise if tags are read unintentionally. NFC provides a more deliberate short-range interaction, making it especially suitable for recording bus boarding and exiting events in a controlled, intentional manner.

Table 2.2 Comparison of Manual, QR, RFID, and NFC attendance methods

Method	Strengths	Weaknesses	Cost or Complexity
Manual Attendance	Very low cost, simple to implement, no hardware dependency	Error-prone, slow, not real-time, poor transparency, difficult to share instantly	Low
QR Code	Low cost, easy to generate, smartphone camera compatible, simple software integration	Slower scanning, affected by lighting and print quality, easier to duplicate/share	Low to Moderate
RFID	Fast, contactless, efficient for repetitive identification	Requires dedicated readers, higher hardware cost, possible unintended reads	Moderate to High
NFC	Fast tap-based interaction, short-range precision, low accidental read risk, suitable for mobile integration	Requires NFC-capable device and student tags/cards, platform compatibility considerations	Moderate

In summary, **NFC** is selected for SchoolWay because it offers the most appropriate balance of speed, intentionality, accuracy, and integration practicality. Unlike manual recording, it automates event capture. Unlike QR codes, it does not depend on camera conditions. Unlike broader-range RFID approaches, it supports controlled close-range interactions that are well suited to boarding and exiting workflows. For these reasons, NFC is the most suitable attendance and identification technology for the proposed system.

2.1.4 Real-Time Tracking Technologies

Real-time tracking technologies are essential to SchoolWay because one of the main objectives of the mobile application is to provide continuous bus location updates to parents and administrators [14]. An effective tracking technology must be capable of supporting route-scale movement visibility, timely updates, and integration with mobile applications. Since school transportation occurs primarily in outdoor road environments, tracking must also remain meaningful over relatively large geographical areas rather than short indoor distances.

Several technologies can be used to monitor movement or location, including GPS-based tracking, GSM or SMS-assisted tracking, and proximity-based methods such as Wi-Fi or Bluetooth. These technologies differ in accuracy, communication model, infrastructure requirements, and suitability for real-time monitoring in transportation scenarios [8].

2.1.4.1 GPS-Based Tracking

GPS-based tracking uses satellite positioning to determine the geographic location of a device. It is widely used in navigation, fleet management, transportation monitoring, and ride-sharing services [14]. In mobile applications, GPS can be combined with network connectivity to transmit device coordinates to a backend service, where the location can then be visualized on a map for end users [10].

The major strength of GPS is its suitability for outdoor location tracking across large distances [8]. It can provide continuous position updates as a bus move along its route and supports map visualization, route awareness, and estimated movement monitoring. Since smartphones commonly include GPS hardware, implementation is practical in mobile-based systems without specialized external tracking devices [4].

Its weaknesses include battery consumption, signal limitations in tunnels or dense urban environments, and dependence on internet connectivity for real-time transmission [27]. GPS may also require location permission handling and optimization to balance accuracy against battery usage. Despite these considerations, GPS remains the standard and most suitable approach for route-scale outdoor transport tracking [10].

2.1.4.2 GSM / SMS-Based Tracking

GSM or SMS-based tracking generally uses cellular communication to send location-related information or status updates from a device to another system or user [7]. In some implementations, GSM is combined with GPS, where GPS obtains coordinates and GSM or SMS is used as the communication channel to transmit them [5].

The strength of GSM or SMS-based tracking is its ability to operate in environments where internet-based data services may be limited or unreliable. SMS notifications can be useful for simple event-based status alerts, such as notifying a parent that a bus has departed or reached a checkpoint [7]. This method can also be easier to understand in low-bandwidth settings.

However, GSM or SMS alone is not ideal for rich, continuous real-time tracking. It does not inherently provide map-based live visualization and is less efficient for frequent updates. Costs may accumulate if message-based communication is used repeatedly, and the user experience is less interactive than that of an application displaying live movement on a map. Therefore, while GSM or SMS is useful as a communication support mechanism, it is less suitable as the primary tracking model for SchoolWay [5].

2.1.4.3 Wi-Fi / Bluetooth-Based Proximity Tracking

Wi-Fi and Bluetooth-based proximity tracking are commonly used to determine whether a device is near a known access point, beacon, or another device [28]. These approaches are well suited to indoor localization, room-level presence detection, or checkpoint-based monitoring in controlled environments.

Their strengths include low-power operation in some configurations and usefulness for short-range proximity detection. Bluetooth beacons, for example, can help determine whether a device has entered or left a specific zone [28]. In limited spaces, these technologies can support localized presence awareness.

Their weaknesses make them unsuitable as primary bus route tracking technologies. Proximity-based systems do not provide continuous geographic coordinates over long outdoor routes and usually depend on additional infrastructure such as beacons or known networks. They are better suited to location approximation near fixed points than to full route-scale transport visibility [28]. Consequently, their role in SchoolWay, if any, would be secondary rather than central.

2.1.4.4 Summary

The comparison of tracking technologies shows that GPS-based tracking is the most appropriate method for outdoor, route-based transportation monitoring. GSM or SMS can support simple event transmission but does not provide an adequate experience for live tracking. Wi-Fi and Bluetooth are useful for proximity detection in controlled spaces but are not designed for continuous road-based monitoring. Since SchoolWay requires parents and administrators to view ongoing bus movement in real time, GPS clearly provides the strongest functional fit.

Table 2.3 Comparison of GPS, GSM/SMS, and Wi-Fi/Bluetooth tracking methods

Technology	Strengths	Weaknesses	Best Use Case
GPS-Based Tracking	Accurate outdoor positioning, supports map visualization, widely available on smartphones, suitable for continuous movement tracking	Battery usage, signal limitations in some environments, requires network transmission for real-time sharing	Route-scale outdoor transport monitoring
GSM / SMS-Based Tracking	Can operate in basic communication environments, useful for event-based alerts, simple status delivery	Not ideal for live map-based tracking, less interactive, repeated message costs may apply	Simple alerts and low-bandwidth updates
Wi-Fi / Bluetooth-Based Proximity Tracking	Useful for short-range detection, suitable for checkpoints or indoor zones, can be low power	Not suitable for long-distance tracking, depends on local infrastructure, does not provide full route coordinates	Indoor or fixed-point proximity monitoring

In summary, **GPS** is selected for SchoolWay because it is the most suitable technology for real-time school bus monitoring over road networks. It provides precise outdoor location data, supports continuous tracking, and integrates naturally with mobile maps and cloud backends. These strengths directly address the system requirement for providing continuous location updates to parents and administrators.

2.1.5 Database and Backend Services

Database and backend services are essential to SchoolWay because the mobile application must store user data, attendance logs, bus location records, and notification-related information [4], [10]. The backend also plays a critical role in synchronization among multiple users, especially when parents, drivers, and administrators need to access updated information at different times and from different devices [5], [12]. The choice of database architecture therefore affects not only storage but also responsiveness, scalability, and ease of integration [14].

For a mobile-based system such as SchoolWay, the backend should support structured data storage, secure access, and timely synchronization [10]. The technologies considered in this review are SQLite, MySQL, and Firebase, each of which reflects a different architectural approach to application data management.

2.1.5.1 SQLite

SQLite is a lightweight relational database engine commonly embedded within mobile or desktop applications [29]. It is designed for local data storage and is often used when an application needs offline persistence without a separate database server [29], [30].

Its main strength is simplicity. SQLite is easy to deploy, fast for local operations, and suitable for storing app data directly on a user's device [29]. It is also efficient for caching, offline data retention, and local transactional storage [30]. Because it does not require a network connection or external server, it is useful for device-level persistence [29].

However, SQLite is not designed to function as a multi-user real-time backend [30]. It does not provide native cloud synchronization, centralized access for multiple users, or built-in authentication [29]. In the context of SchoolWay, where parents, drivers, and administrators require shared access to updated records, SQLite alone is insufficient as the primary data platform [10], [29].

2.1.5.2 MySQL

MySQL is a widely used relational database management system that supports structured data storage, server-side access, and multi-user applications [30]. It is commonly used in web and enterprise systems where a centralized database server is required [30].

Its strengths include maturity, reliability, strong relational modelling, and support for structured queries [30]. MySQL is suitable for systems with clearly defined schemas and can serve as the backend foundation for complex applications when combined with a server-side application layer [30]. It also supports data consistency and administrative control effectively.

Its weaknesses for SchoolWay relate mainly to integration complexity. Using MySQL typically requires a separate backend application or API layer for mobile clients to access and update data securely [30]. This increases development effort and infrastructure setup compared to serverless or integrated backend solutions [30]. Real-time synchronization must also be built explicitly rather than being available by default [30], [31]. Therefore, MySQL is powerful but may be less convenient for a system emphasizing rapid development and real-time mobile interaction [30].

2.1.5.3 Firebase

Firebase is a cloud-based backend platform that provides services such as real-time databases, authentication, cloud messaging, analytics, and file storage [31]. It is widely used in mobile application development because it reduces the need to build and maintain a traditional custom backend from scratch [31].

Its strengths are highly relevant to SchoolWay. Firebase supports real-time data synchronization, which is valuable for updating attendance and bus location data across multiple users [10], [31]. It also provides integrated authentication services, cloud messaging for notifications, and a managed infrastructure that simplifies deployment [31]. Because it is designed to work closely with mobile applications, it reduces backend complexity and accelerates development [31].

Its weaknesses include dependence on a third-party ecosystem, possible cost growth as usage increases, and a less traditional relational data model compared with SQL databases [31]. Developers must also design data access carefully to avoid inefficient reads or writes [31]. Despite these limitations, Firebase provides a strong practical advantage for a prototype mobile application requiring real-time interaction among mobile users [10], [31].

2.1.5.4 Summary

SQLite is effective for local storage but does not meet the centralized multi-user requirements of SchoolWay. MySQL is robust and structured, but it generally requires additional backend services and more implementation effort to support real-time mobile synchronization. Firebase offers direct advantages in real-time updates, authentication, push integration, and rapid development, making it especially appropriate for a mobile transportation management system.

Table 2.4 Comparison of SQLite, MySQL, and Firebase

Technology	Strengths	Weaknesses	Best Use Case
SQLite	Lightweight, fast local storage, simple integration, supports offline persistence	Not centralized, no native real-time sync, unsuitable as shared multi-user backend	Device-level local storage
MySQL	Mature relational model, strong query support, centralized storage, reliable for structured systems	Requires separate backend/API layer, more setup effort, real-time features need additional implementation	Traditional client-server applications
Firebase	Real-time synchronization, integrated authentication, cloud messaging support, managed infrastructure, rapid mobile integration	Vendor dependence, cost scaling considerations, less suited to complex relational models	Real-time mobile and cloud-connected applications

In summary, **Firebase** is selected for SchoolWay because it best satisfies the project's need for centralized storage, real-time synchronization, user authentication, and notification support within a mobile-first architecture. Its managed backend services reduce implementation complexity while enabling the project to focus on core application features such as attendance logging, communication among the user, and bus tracking.

2.1.6 Map and Notification Services

Map and notification services are important supporting components in the development of **SchoolWay** because the mobile application requires both visual representation of bus location and timely communication with users [4]. Since one of the core functions of the proposed application is to provide real-time visibility of school bus movement, an appropriate mapping service is necessary to display location data clearly and accurately [5]. At the same time, a notification service is needed to inform parents and administrators of important transportation events such as student boarding, exiting, and possible delays [10]. The selection of these services must therefore consider integration capability, cost, usability, and suitability for a mobile-based school transportation system [8].

In this project, map service selection is particularly significant because it directly affects the practicality and sustainability of the tracking feature. While commercial mapping solutions offer rich functionality, open-source and flexible alternatives may provide greater affordability and customization for academic and prototype-based systems [32]. Similarly, push notification services must be reliable enough to support real-time communication between the system and its users. For these reasons, this section reviews **Google Maps API**, **OpenStreetMap**, and **push notification services** before justifying the final choice for SchoolWay.

2.1.6.1 Google Maps API

Google Maps API is one of the most widely used mapping platforms for mobile and web applications. It provides interactive maps, geolocation support, markers, route visualization, and location-based services that can be integrated into digital systems. Due to its popularity, Google Maps API is often considered a standard solution for applications requiring map display and navigation-related features [5].

One of its major strengths is its accuracy and maturity. Google Maps API offers reliable map data, familiar visual presentation, and extensive documentation for developers. It also supports real-time map interaction, making it suitable for applications that require users to monitor moving vehicles such as school buses [12]. In addition, the widespread familiarity of Google Maps may improve usability, as many users are already accustomed to its interface and map conventions.

However, Google Maps API also has several limitations. The service is commercial in nature and may involve usage limits, billing requirements, and API quota restrictions, particularly when location requests become frequent. This may create challenges for student projects or small-scale systems that aim to remain cost-effective. Furthermore, its level of customization may be more restricted when compared to some open-source alternatives. Although Google Maps API is technically strong, these limitations make it less attractive for a project that prioritizes affordability and flexibility.

2.1.6.2 OpenStreetMap

OpenStreetMap (OSM) is a collaborative, open-source mapping platform that provides freely accessible geographic data for use in web and mobile applications [32]. Unlike proprietary map providers, OpenStreetMap allows developers to use and customize map data with greater flexibility, making it an attractive option for educational, research, and prototype-based projects. It has become a widely adopted alternative in systems that require map-based visualization without the cost implications of commercial platforms.

The primary strengths of OpenStreetMap are its openness, cost-effectiveness, and adaptability [33]. Since the platform is community-driven and openly licensed, it is well suited for projects that require a sustainable and low-cost mapping solution. OpenStreetMap also allows greater customization in map styling and data usage, which can benefit systems such as SchoolWay where only specific transport-related visualization is needed. In addition, OpenStreetMap can be integrated into Flutter applications through compatible mapping libraries, making it practical for mobile-based development [24].

Nevertheless, OpenStreetMap also has certain weaknesses. The quality and completeness of mapping data may vary depending on the region, as the data is maintained collaboratively by contributors rather than a single centralized provider [33]. In some locations, it may offer less polished visualization or fewer advanced built-in services compared to commercial platforms. Furthermore, implementation may require more configuration effort depending on the libraries and tools selected. Despite these limitations, OpenStreetMap remains highly suitable for projects that value flexibility, affordability, and open access.

2.1.6.3 Push Notification Services

Push notification services enable applications to deliver real-time alerts and updates directly to users' devices. In the context of SchoolWay, such services are essential because parents and administrators need to be informed promptly about important events, including student boarding, bus departure, route updates, and student drop-off [2]. Without a notification mechanism, users would need to repeatedly open the application to check for updates, which would reduce convenience and the effectiveness of the mobile application.

A commonly used push notification solution is **Firebase Cloud Messaging (FCM)**. This service supports the delivery of notifications to mobile devices and is widely integrated into modern mobile applications [31]. Its strengths include reliable message delivery, ease of integration with Firebase-based backends, and support for event-triggered communication. This makes it suitable for SchoolWay, especially since the project already adopts Firebase for data storage and synchronization [31]. Through FCM, the mobile application can send immediate alerts when attendance events are recorded or when transport status changes occur.

The main limitations of push notification services are not in their functionality, but in their implementation and management. Notifications must be designed carefully to avoid excessive or unnecessary alerts, which may lead users to ignore them [34]. In addition, permission handling must be managed properly so that users allow the application to receive notifications. Despite these considerations, push notification services remain an essential component for improving user awareness and communication efficiency in a school transportation mobile application.

2.1.6.4 Summary

The comparison of mapping and notification services indicates that both Google Maps API and OpenStreetMap can support location visualization in mobile applications, while Firebase Cloud Messaging is suitable for real-time alert delivery. Google Maps API offers strong map accuracy, familiarity, and extensive built-in support, but it introduces cost and quota considerations that may not be ideal for an academic project. OpenStreetMap, on the other hand, provides a more flexible and cost-effective alternative, making it particularly suitable for prototype development and projects that require customization without dependency on a commercial provider.

Table 2.5 Comparison of map and notification service options

Service	Category	Strengths	Weaknesses
Google Maps API	Mapping	Accurate maps, familiar interface, strong mobile integration, rich documentation	Usage limits and billing considerations, third-party dependence
OpenStreetMap	Mapping	Flexible, customizable, potentially lower dependence on a single provider	May require more configuration, varying integration complexity
Firebase Cloud Messaging	Notifications	Real-time push delivery, integrates well with Firebase, broad mobile support	Requires permission management and notification strategy design

In summary, **OpenStreetMap** is selected as the mapping service for SchoolWay because it provides a flexible, open-source, and cost-effective solution that is well aligned with the scope of the project. Its ability to support map visualization without the financial and quota-related constraints of proprietary alternatives makes it a practical choice for a university-level mobile application. At the same time, **Firebase Cloud Messaging** is retained as the push notification service because it complements the selected Firebase backend and enables timely communication of important transportation events to parents and administrators.

2.2 Review of Existing Systems / Applications

This section reviews existing systems and applications relevant to school transportation tracking, student attendance monitoring, and integrated transport management. Examining real-world applications is important because it reveals current industry practices, highlights strengths that can inform the proposed design, and identifies limitations that SchoolWay can address. The review focuses on three systems: a school bus tracking application, a student attendance monitoring system, and an integrated school transport management platform. The purpose is not only to describe these systems but also to identify the functional gaps that justify the development of SchoolWay.

2.2.1 Existing System A: School Bus Tracking Application

A suitable example of a school bus tracking application is **SafeStop**, a mobile-focused platform designed to provide parents and schools with visibility into school bus movement and arrival status [35]. SafeStop is known for enabling parents to track bus routes and receive updates associated with daily transportation [35].

The main features of SafeStop include live bus location monitoring, route awareness, estimated arrival support, and status updates through a user-accessible mobile interface [35]. From a user perspective, the system is valuable because it reduces uncertainty around bus arrival and allows parents to make better decisions regarding pickup or readiness [5]. The platform reflects a strong emphasis on real-time transport visibility [35].

Its strengths lie in usability and direct problem relevance [10]. SafeStop addresses one of the most common pain points in school transportation, namely the lack of location transparency [5]. By focusing on route tracking and updates, it improves the parent experience and reduces the need for repeated phone-based inquiries [35]. Its mobile-centred approach also aligns with the practical expectation that transport information should be accessible on demand [10].

However, SafeStop also presents limitations when considered against the broader objectives of SchoolWay. Its functional emphasis is primarily on tracking rather than integrated student attendance management [35]. It does not inherently resolve the challenge of verifying whether a specific student has boarded or exited the bus through an identification mechanism such as NFC [25]. In addition, the system may

emphasize parent-facing visibility more than full tri-role coordination among parents, drivers, and administrators within a unified application workflow [10].

The gap revealed by this system is therefore clear: while real-time tracking is valuable, tracking alone does not fully solve the accountability problem in student transportation. SchoolWay extends beyond this model by combining bus location visibility with student event logging and role-based coordination [26].

2.2.2 Existing System B: Student Attendance Monitoring System

A relevant example of a student attendance monitoring system is **SchoolPass**, a platform widely associated with attendance, dismissal, and student movement management in school environments [36]. Although its primary context is broader campus workflow rather than school bus-only operations, it provides a useful reference for digital student status management [36].

The main features commonly associated with SchoolPass include digital attendance processes, dismissal coordination, guardian-related student release workflows, and administrative oversight of student status changes [36]. Its value lies in digitizing workflows that are often handled manually in many institutions [15]. By replacing paper-based records or fragmented procedures with a more structured digital process, the system improves traceability and administrative efficiency [15].

The strengths of SchoolPass include centralization of student status information, improved record visibility, and support for school operational workflows beyond simple manual logging [36]. It demonstrates how digital attendance-related systems can strengthen institutional control, reduce errors, and improve communication around student movement events [15]. The administrative perspective is particularly strong, as the system helps schools standardize how attendance and dismissal information is handled [36].

Its weaknesses, when viewed from the perspective of SchoolWay, are related to transport-specific functionality [10]. A campus attendance and dismissal platform does not necessarily provide real-time bus tracking or route visualization [5]. It may also lack a dedicated driver-oriented operational module or a direct mechanism for recording transportation-specific boarding and exiting events through NFC-based tap interactions [25]. In other words, it improves institutional attendance processes but does not fully address school transport monitoring as an integrated mobile activity [15].

The gap identified here is that attendance monitoring systems may digitize student status effectively but remain disconnected from transport-specific real-time tracking [26]. SchoolWay seeks to address this gap by linking identification-based event capture directly with bus movement visibility and user modules [10], [25].

2.2.3 Existing System C: Integrated School Transport Management System

A strong example of an integrated school transport management system is **Zonar Systems**, which is widely recognized in student transportation and fleet monitoring contexts [37]. Zonar offers solutions related to fleet visibility, route operations, driver support, and transport oversight for educational institutions and other fleet-based environments [37].

The main features associated with Zonar include fleet tracking, route monitoring, operational dashboards, driver-related tools, and transportation management capabilities that support institutional oversight [37]. This makes it more comprehensive than a simple consumer-facing tracking application [14]. It demonstrates how transport technology can support large-scale coordination, visibility, and accountability at the operational level [37].

The strengths of Zonar include broad transport management functionality, strong institutional orientation, and the ability to support administrative oversight across vehicle operations [14]. It is more comprehensive than systems focused solely on parent-facing visibility and shows the value of combining tracking data with operational decision support [37]. Its design perspective is especially relevant to school transportation providers and administrators.

Its weaknesses, relative to SchoolWay, stem from system complexity and focus. Enterprise-grade transport platforms can be more operationally heavy and may not prioritize a lightweight unified mobile experience for parents, drivers, and administrators within a single project-scale application [14]. They may also not focus on simple NFC-based boarding and exiting confirmation at the student level as a core mobile workflow. For a university project targeting direct user communication and student-level event logging, such enterprise systems can be functionally broad but not necessarily optimized for the same interaction model [10].

The gap shown by Zonar is that strong fleet management capability does not automatically imply a simple, student-centric, mobile-integrated system with

attendance and parent communication features combined in one coherent application [14]. SchoolWay addresses this by focusing on a narrower but more integrated set of features tailored to daily student transport monitoring [26].

2.2.4 Proposed System and Summary

The review of existing systems demonstrates that each solution addresses an important part of the school transportation problem space, but none fully aligns with the integrated feature set proposed for SchoolWay [10]. SafeStop emphasizes real-time bus tracking and improves parent visibility, but it does not adequately address automated student attendance capture [35]. SchoolPass demonstrates the value of digitized student status and attendance workflows, but it is not primarily designed as a transport tracking system with route-level real-time monitoring [36]. Zonar Systems provides broad transport management capabilities, yet its enterprise orientation does not directly translate into a lightweight mobile application centred on the combined needs of parents, drivers, and administrators at the student-event level [37].

SchoolWay is proposed as a mobile-based system that integrates the strengths of these approaches while addressing their limitations [10], [25]. It includes real-time GPS bus tracking, NFC-based student boarding and exiting logs, role-based modules for parents, drivers, and administrators, push notifications, and centralized data storage [10], [25], [26]. This integration is important because it connects transport visibility, student accountability, and communication among the user within a single mobile application architecture [5]. Rather than solving only one part of the problem, SchoolWay is designed to support a more complete daily transport workflow.

Table 2.6 Comparison of existing systems and the proposed SchoolWay system

System	Mobile-based	Parent module	Driver module	Admin module	NFC attendance	Real-time GPS tracking	Push notification	Centralized data storage
SafeStop	✅	✅	⚠	⚠	❌	✅	✅	✅
SchoolPass	✅	⚠	❌	✅	❌	❌	✅	✅
Zonar Systems	✅	⚠	✅	✅	❌	✅	✅	✅
SchoolWay (Proposed)	✅	✅	✅	✅	✅	✅	✅	✅

✅ Yes
❌ No
⚠ Limited

In summary, SchoolWay builds upon the practical strengths observed in existing systems while addressing the gaps that remain across them [10]. The proposed system combines mobile accessibility, student-level event logging, user-specific modules, location visibility, and centralized information handling in a single platform [26]. This integrated approach constitutes the main contribution of the proposed system at the design level and justifies its development within the context of school transportation management [14].

2.3 Chapter Summary

This chapter has reviewed the main technologies and existing systems relevant to the development of SchoolWay. The review began with the field of school transportation management, highlighting the limitations of traditional manual practices and the need for a digital mobile solution. It then examined mobile platforms and development frameworks, identifying Flutter as the most suitable choice due to its cross-platform efficiency, consistent interface design, and practical support for the required application features.

The chapter also reviewed attendance and identification technologies, where NFC was selected because it offers fast, intentional, and reliable student event logging for boarding and exiting activities. In the area of real-time tracking, GPS was identified as the most appropriate technology for monitoring bus movement over outdoor routes. For backend services, Firebase was chosen because it provides centralized cloud storage, authentication, and real-time synchronization suitable for a mobile multi-user application. Finally, Google Maps API and Firebase Cloud Messaging were identified as suitable support services for map visualization and push notifications respectively.

The review of existing systems showed that current applications often address only part of the problem space. Some systems focus on bus tracking, others on attendance or campus workflows, and others on broader transport operations. However, the combination of role-based mobile access, NFC attendance logging, real-time GPS tracking, push notifications, and centralized data management remains insufficiently integrated in a single lightweight application. These gaps justify the development of SchoolWay as a focused and practical digital solution for school transportation management.

CHAPTER 3

Proposed Method/Approach

3.1 System Development Models

System development models are important because they provide a structured approach for planning, implementing, and validating software projects. A suitable model helps reduce technical and managerial risks, improves communication among stakeholders, and ensures that deliverables are produced in a controlled and measurable manner throughout the project lifecycle [38], [39].Since different projects have different requirement stability, risk levels, and user involvement needs, this study compares three development models before selecting the most appropriate one for the SchoolWay mobile application.

3.1.1 System Development Model 1: Waterfall Model

The Waterfall model is a linear and phase-based methodology where each phase, such as requirements, design, implementation, testing, and deployment, is completed before moving to the next [38]. It emphasizes strong documentation and clear phase boundaries. Feedback loops are limited, so changes are costly once development has progressed.

Table 3.1.1: Strengths and Weaknesses of Waterfall Model

Strengths	Weaknesses
Clear structure and documentation at every phase.	Low flexibility when requirements change mid-project.
Easier to manage for projects with stable and well-defined requirements.	User feedback usually arrives late, often during testing or deployment.
Milestones and deliverables are straightforward to monitor.	Risk of mismatch between user expectations and final product.

Suitability for this project

Waterfall is only partially suitable because SchoolWay includes evolving requirements such as, role workflows, live tracking behaviour, and UI refinements, which require iterative feedback.

Figure of Model

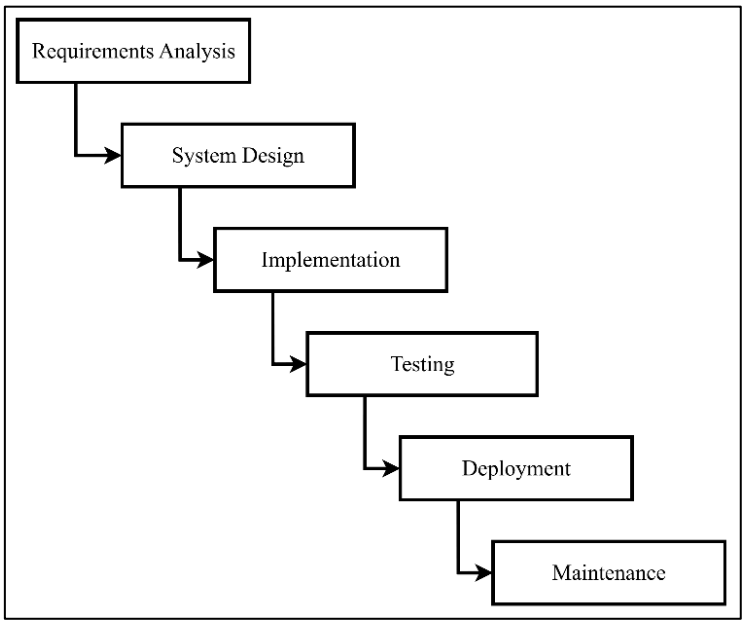


Figure 3.1.1: Waterfall Model Process Flow

3.1.2 System Development Model 2: Agile Model

Agile is an iterative and incremental model where development is delivered in short cycles (sprints), with frequent stakeholder feedback and continuous adaptation [40], [41]. Requirements can be refined during development. Working software is prioritized early and often.

Table 3.1.2: Strengths and Weaknesses of Agile Model

Strengths	Weaknesses
High adaptability to changing requirements.	Requires consistent stakeholder participation and fast decision-making.
Frequent stakeholder involvement improves requirement accuracy.	Scope creep may occur if backlog control is weak.
Early and continuous delivery reduces project risk.	Documentation can become lightweight unless explicitly managed.

Suitability for this project

Agile is suitable because SchoolWay contains multiple user roles such as admin, driver, parent and features that benefit from iterative validation and continuous refinement.

Figure of Model

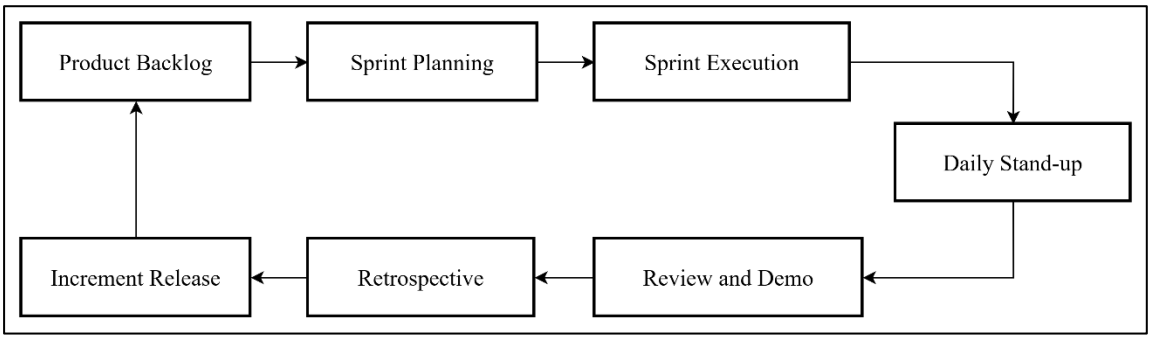


Figure 3.1.2: Agile Iterative Sprint Cycle

3.1.3 System Development Model 3: Prototype Model

The Prototype model develops an early working version of the system to gather user feedback before full-scale implementation [38], [39].The prototype may be revised multiple times until core requirements become clearer. This is useful when user expectations are not fully known at the beginning.

Table 3.1.3: Strengths and Weaknesses of Prototype Model

Strengths	Weaknesses
Improves requirement clarity through early user interaction.	Can increase development time if prototype revisions are excessive.
Reduces risk of building unusable interfaces or workflows.	Users may confuse prototype quality with final system quality.
Particularly effective for UI-intensive or workflow-heavy systems.	May lead to weak architecture if quick fixes are overused.

Suitability for this project

Prototype is highly suitable for SchoolWay because dashboard layout, map-based interactions, and role-specific user experience require repeated usability feedback.

Figure of Model

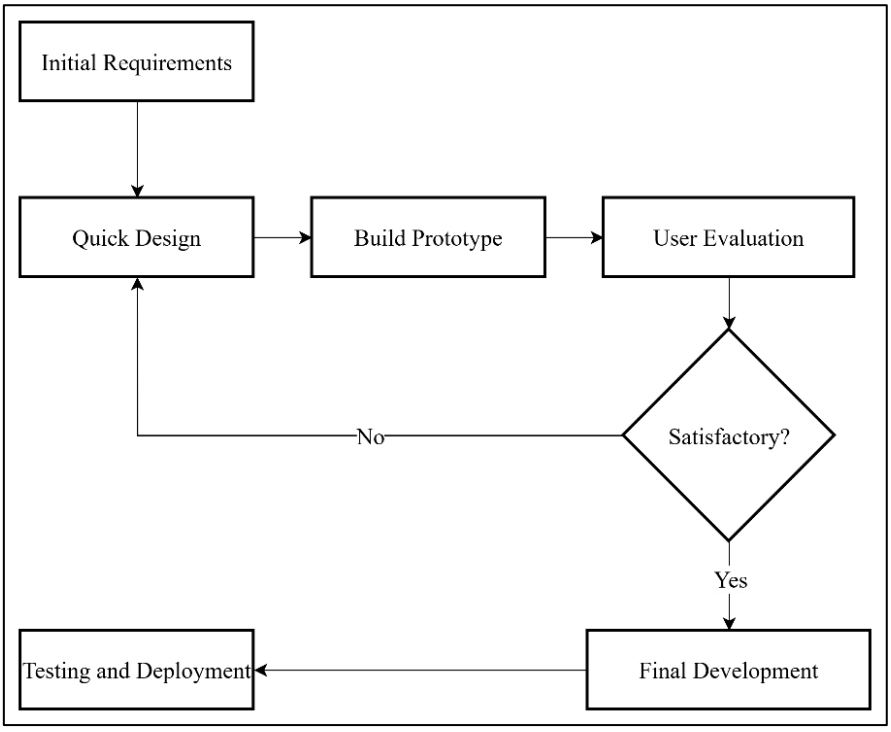


Figure 3.1.3: Prototype Development Cycle

3.1.4 Selected Model: Prototype-Driven Agile Development

The selected approach for this project is a Prototype-driven Agile model. This is because Agile iterations supported by early and repeated prototyping.

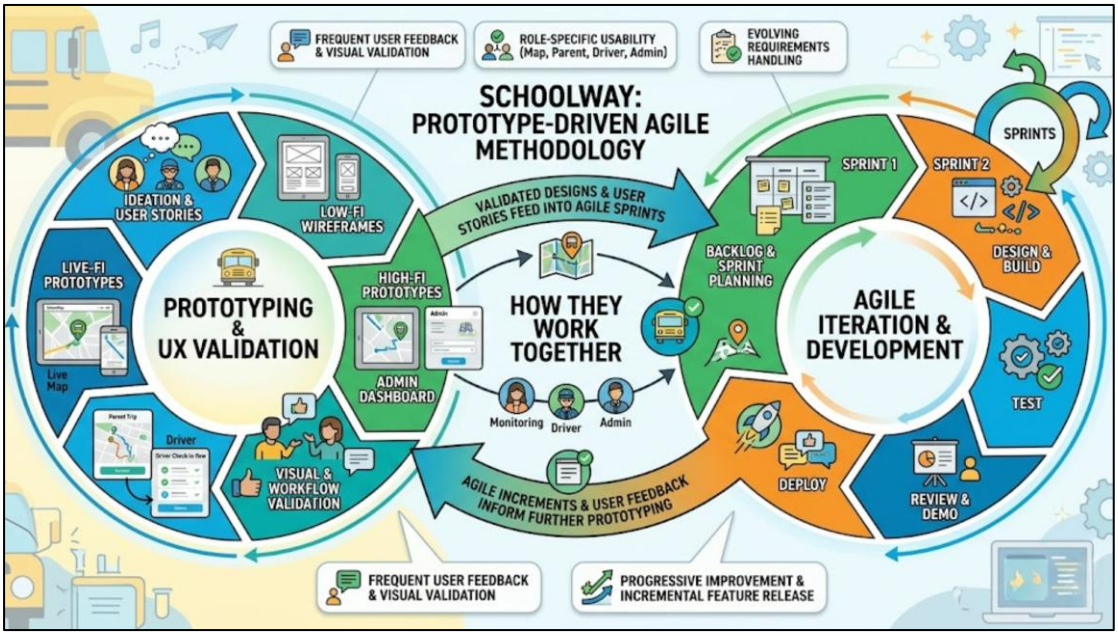


Figure 3.1.4: Prototype-Driven Agile Development

Justification

SchoolWay is a user-facing mobile system with multiple actors and interaction-heavy modules such as live map views, parent trip monitoring, driver check-in flows, and administrative control screens. These characteristics require frequent user feedback, visual validation, and progressive improvement, which are strongly supported by both prototyping and Agile iteration [40], [41].

Compared with this approach, Waterfall is less suitable because it assumes stable requirements and delayed feedback, which does not match a real-time transport monitoring application. Pure Agile is strong for iteration, but adding explicit prototyping further improves UI/UX clarity and workflow validation early in development. Therefore, Prototype-driven Agile best matches project needs which include evolving requirements, role-specific usability concerns, and incremental feature release.

3.2 System Requirement

3.2.1 Introduction

The successful development and deployment of SchoolWay require well-defined system requirements. These requirements are categorized into hardware and software components to ensure the project can be developed, tested, and operated effectively. Hardware requirements ensure sufficient processing, memory, connectivity, and device capability, while software requirements provide the tools, frameworks, and cloud services needed to implement secure authentication, real-time tracking, notifications, and data management.

3.2.2 Hardware Components

The hardware requirements define the essential devices and equipment needed for SchoolWay, including a development computer, Android phone, GPS-enabled device, NFC-capable phone, and stable internet connection. These components provide computing power, connectivity, and user interaction to ensure smooth development, live tracking, and reliable system communication.

Table 3.2.2: Summary of Hardware Components and Minimum System Requirements

Component	Minimum Requirement	Recommended Requirement	Purpose
Developer Laptop/PC	Intel i5 (or equivalent), 8 GB RAM, 20 GB free storage	Intel i7/Ryzen 7, 16 GB RAM, SSD 256 GB+	Used for Flutter development, build, emulator usage, and testing.
Android Test Device	Android 8.0+, 3 GB RAM, GPS support	Android 11+, 4 GB RAM+, NFC + GPS	Used to test parent/driver/admin mobile flows and location features.
Internet Connectivity	Stable 10 Mbps	30 Mbps+	Required for Firebase authentication, Firestore/Realtime Database sync, and push notifications.
GPS Sensor (mobile)	Built-in GPS	High-accuracy GPS + mobile data	Required for real-time bus location update and map rendering.
NFC-capable Phone (driver use case)	Android with NFC hardware	Android with NFC + secure element support	Needed for NFC-based attendance/check-in features.

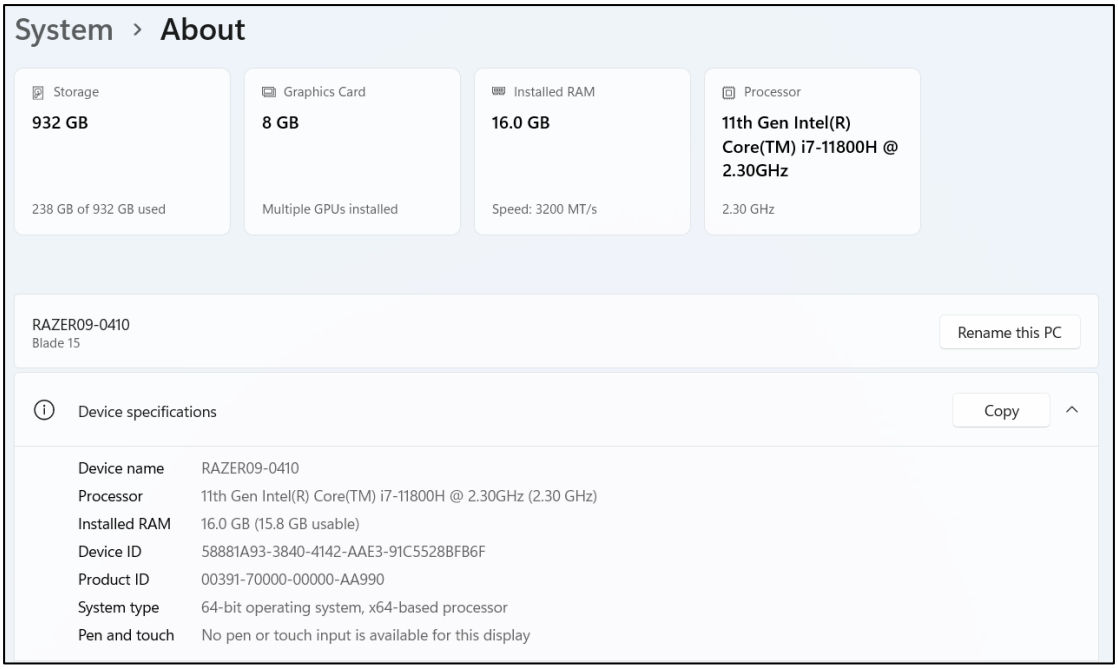


Figure 3.2.2.1: Developer device specifications

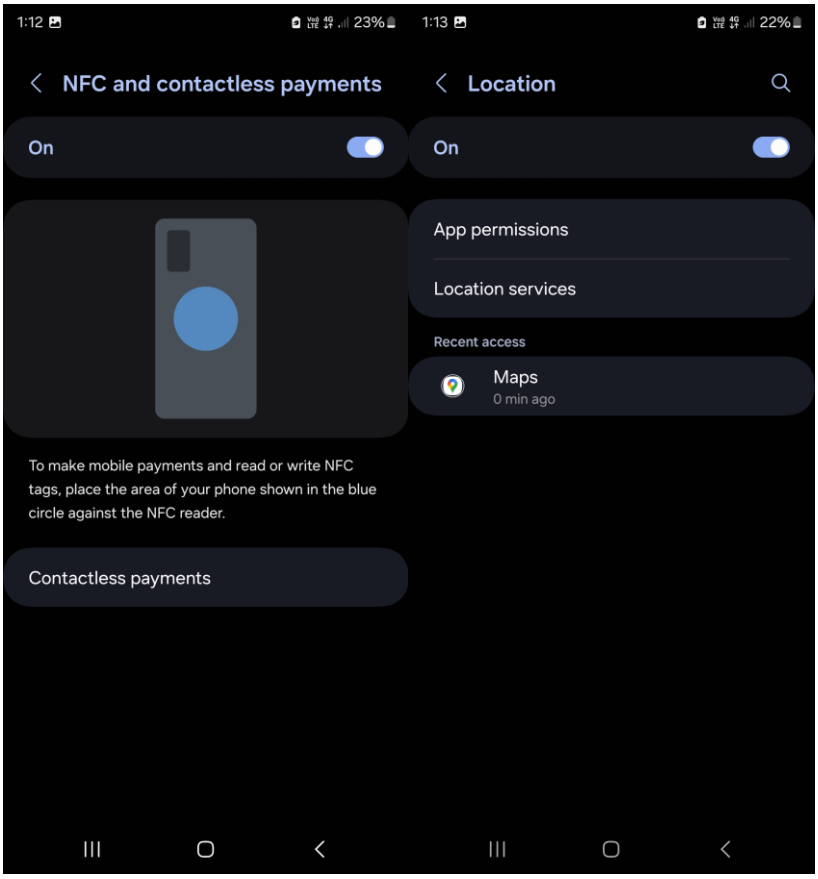


Figure 3.2.2.2: Android device settings showing GPS and NFC enabled

Table 3.2.2 outlines the hardware requirements needed to support the SchoolWay mobile application. The development laptop or PC functions as the primary platform for writing code, performing debugging activities, and executing the Flutter project effectively. The Android test device is required to evaluate the application under real operating conditions, particularly for the parent- and driver-related functions. In addition, stable internet connectivity is essential because the system depends on Firebase services for authentication, cloud data storage, real-time synchronisation, and notification delivery. A GPS-enabled device is also necessary to facilitate location-based tracking, while an NFC-capable phone is required to support driver attendance and check-in operations. Collectively, these hardware components ensure that the application can be developed, tested, and operated in a dependable and efficient manner.

Figure 3.2.2.1 presents the specifications of the developer's device, including the processor, memory, storage capacity, and system type. These specifications indicate that the laptop possesses adequate computational resources to support Flutter development, emulator execution, and debugging processes.

Figure 3.2.2.2 shows that the Android device has both GPS and NFC functions enabled. This configuration is significant because GPS is required for live bus tracking, whereas NFC is used in the driver module for tap-based check-in and attendance recording.

Overall, the table and figures confirm that the project is supported by suitable hardware for both development and operational testing.

3.2.3 Software Components

The software requirements define the applications, platforms, and tools needed for SchoolWay, including Flutter, Dart, Firebase, Android Studio, and related packages. These components support app development, testing, authentication, database management, real-time updates, and notification services, ensuring the system runs efficiently and reliably.

Table 3.2.3: Software Requirements for the SchoolWay Mobile Application

Software	Version / Platform	Purpose
Flutter SDK	Stable channel (3.x)	Cross-platform mobile UI development framework.
Dart SDK	Matching Flutter SDK	Main programming language for business logic and UI.
Android Studio / VS Code	Current stable	IDE and emulator/device debugging support.
Firebase Core Services	Firebase project setup	Backend integration entry point for authentication and data services.
Firebase Authentication	Cloud service	Secure login and account management (email/password, role-gated flows)
Cloud Firestore	Cloud NoSQL database	Stores users, routes, notifications, and operational records
Firebase Realtime Database	Cloud database	Supports low-latency updates for live location/state synchronization.
Firebase Cloud Messaging	Cloud push service	Sends app notifications and alerts.
flutter_map + map tiles	Package-based map stack	Displays route and live bus movement in map interface.
Geolocator	Flutter package	Device location collection and permission handling.
Hive	Local storage package	Local/offline data persistence and queue buffering.

Minimum installation requirements for Android emulator:

1) Android version: 8.0 (Oreo) or later
2) Free storage: at least 200 MB
3) RAM: at least 3 GB
4) Active internet connection
5) Location permission enabled
6) Notification permission enabled
7) NFC enabled (driver use case only)

Table 3.2.3 presents the software requirements and development tools used in the SchoolWay mobile application. It defines the core development stack, including Flutter SDK, Dart SDK, and Android Studio or Visual Studio Code, which together provide the environment for interface development, coding, debugging, and testing. The table also identifies backend and supporting packages required for authentication, database operations, real-time updates, map rendering, location access, and offline storage, thereby establishing the technical foundation needed for reliable system implementation.

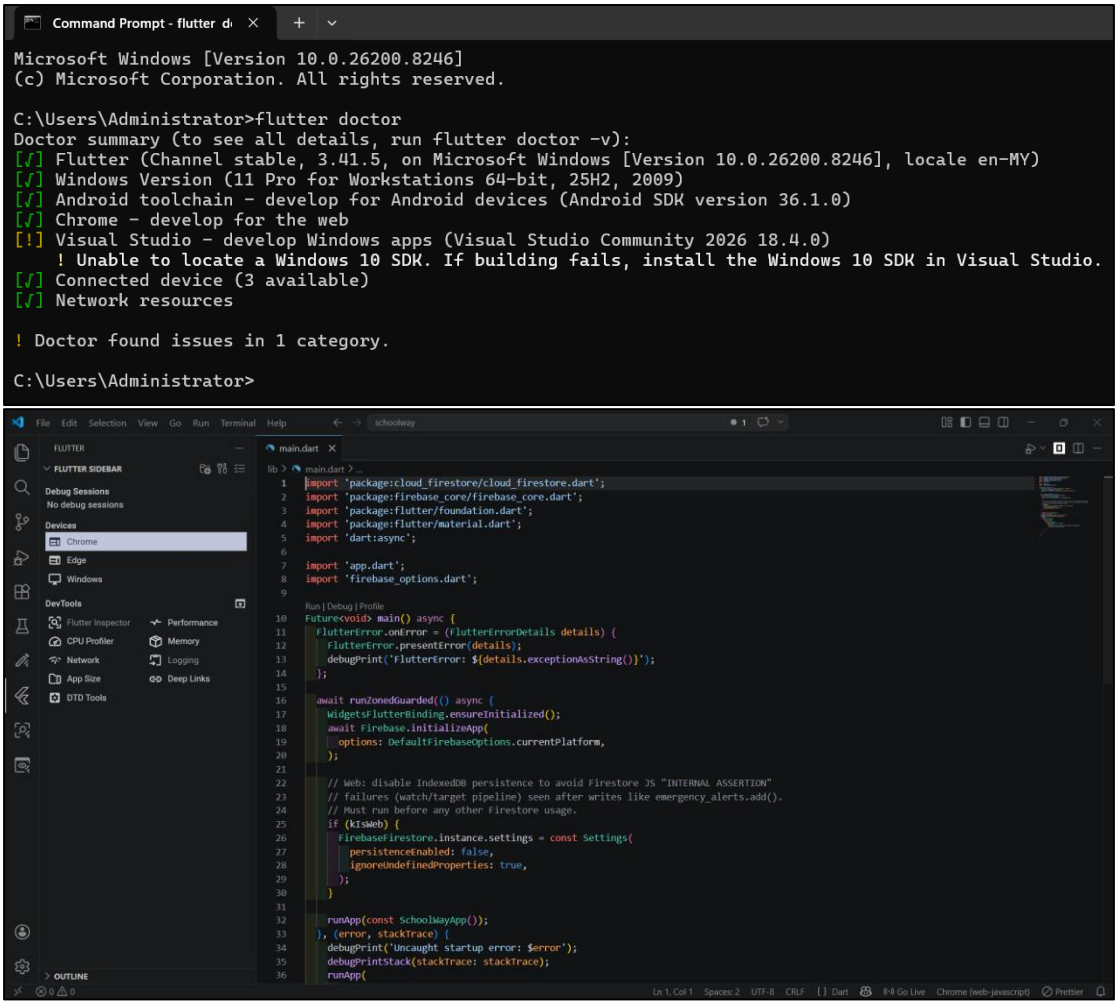


Figure 3.2.3.1: Flutter SDK and IDE setup

Figure 3.2.3.1 shows the local development environment used for implementation. The Flutter diagnostic output indicates that the essential toolchain is installed and functional, while the IDE interface confirms that project source code is being developed and managed in a proper coding workspace. This figure supports the

practicality of the software setup listed in Table 3.2.3 and demonstrates that the project can be built, executed, and debugged in a controlled development environment.

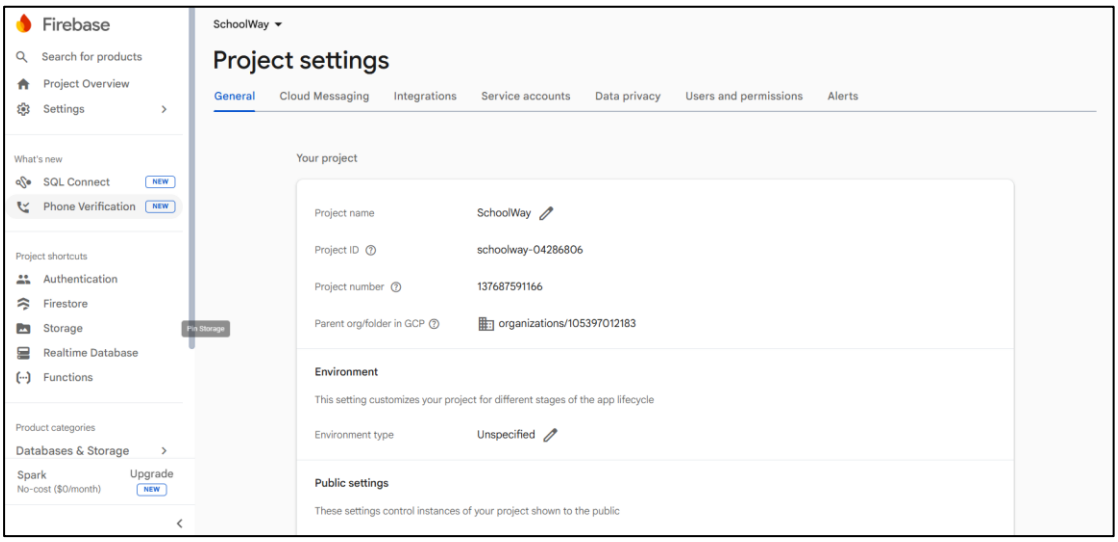


Figure 3.2.3.2: Firebase project console

Figure 3.2.3.2 shows the Firebase project console with configured services. This figure provides evidence that the cloud backend environment is established and ready to support system operations. It verifies that the project is connected to a valid Firebase workspace and that the required cloud modules can be integrated for authentication, data storage, synchronization, and notification delivery.

Firebase services form the backend core of SchoolWay. Firebase Authentication is used for secure user login and account management, Cloud Firestore stores structured operational data such as users, buses, routes, and notifications, Firebase Realtime Database supports low-latency synchronization for live tracking updates, and Firebase Cloud Messaging enables real-time alert and notification delivery. These services are essential because the system relies on continuous cloud communication and role-based data access across parent, driver, and admin modules [42].

Other services and packages support application-specific features. The flutter_map package is used for route visualization and live map display [43]. Geolocator provides device positioning and permission handling for tracking features, and Hive supports local persistence and queue buffering when connectivity is unstable [44]. Together with the core development tools, these services improve performance, usability, and resilience of the application during both development and deployment.

In summary, Table 3.2.3 and Figure 3.2.3.1 to Figure 3.2.3.2 collectively confirm that SchoolWay is supported by a complete and operational software ecosystem. The integration of development tools, Firebase backend services, and supporting packages provides a stable platform for secure authentication, real-time monitoring, data management, and dependable mobile application performance.

3.3 System Design

3.3.1 Introduction

The system architecture of SchoolWay is designed to provide a scalable, secure, and responsive platform for school transportation monitoring and coordination. The architecture organizes front-end mobile modules, cloud backend services, and communication channels into a modular structure so each subsystem can be developed and maintained independently while still working cohesively. The overall architecture is illustrated in Figure 3.3.1.



Figure 3.3.1: System Architecture Diagram

3.3.2 High-Level Architecture Components

1. **Presentation Layer (Mobile Application)**

   The mobile frontend is built using Flutter, enabling a unified codebase with role-based interfaces for Admin, Driver, and Parent users. Each role has dedicated screens and permissions, improving usability and access control.

2. **Authentication and Authorization Layer**

   Firebase Authentication manages user sign-up, sign-in, and session handling. Role-based access is enforced by storing role metadata and routing users to appropriate modules after authentication.

3. **Data Management Layer**

   Cloud Firestore stores structured entities such as user profiles, student records, route planning data, notifications, and attendance-related events. Firebase Realtime Database is used for low-latency, frequently changing data such as live bus location and movement updates.

4. **Location and Tracking Layer**

   GPS data from driver devices is periodically captured and synchronized to backend services. Parent and admin views retrieve this data to visualize live fleet movement and trip progress.

5. **Notification and Alert Layer**

   Firebase Cloud Messaging delivers push notifications for important operational events, such as emergencies, trip updates, and transport alerts.

6. **Integration Layer**

   External package integrations include map rendering, geolocation, NFC reading (driver workflow), and CSV-based import/export for administrative operations.

7. **Security and Reliability Layer**

   Security rules in Firebase services, role checks, and controlled permissions protect data access. Error handling and local persistence mechanisms improve resilience during unstable connectivity.

3.3.3 Data Flow Summary

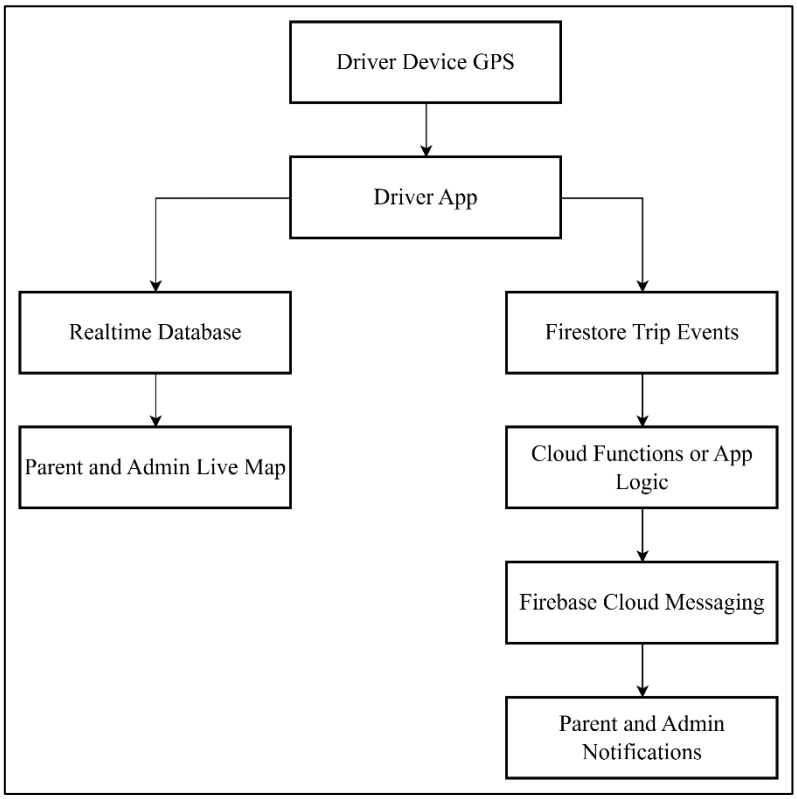


Figure 3.3.3: Data Flow Diagram for location and notifications

As shown in Figure 3.3.3, the SchoolWay data flow illustrates how authentication, real-time tracking, and notification services are integrated to support continuous communication between drivers, parents, and administrators.

1. User authenticates through Firebase Authentication.
2. System identifies user role and loads corresponding interface.
3. Driver location and event data are uploaded to cloud services.
4. Parent/admin modules subscribe to updates and display live status.
5. Notifications are dispatched via cloud messaging when trigger conditions are meet.

3.3.4 Conclusion of Design

In conclusion, the SchoolWay architecture establishes a robust foundation for secure user access, real-time tracking, role-based operations, and scalable cloud-backed data services. Its modular design supports maintainability and future enhancements such as analytics, route optimization, and expanded platform support.

3.4 Functional Requirements

3.4.1 Introduction

This section defines the key functional requirements that describe what SchoolWay must do to satisfy project objectives. These requirements cover user identity management, operational workflows, real-time tracking, communication, and data administration.

3.4.2 Functional Requirements Table

A functional requirement is a specific description of a system behaviour or action that the system must perform. In the context of the SchoolWay Mobile Application, functional requirements define the exact operations that each module must execute to achieve the project objectives of secure user access, real-time school bus tracking, route and user management, attendance/event recording, notification delivery, and operational reporting. These requirements ensure that the system can process data accurately, coordinate communication among admin, driver, and parent roles, and provide reliable transport monitoring services.

Table 3.4.2:

ID	Functional Requirement	Description
FR-01	User Registration and Login	The system shall allow users to register and authenticate securely using supported methods.
FR-02	Role-Based Access Control	The system shall grant access to modules based on assigned role (Admin, Driver, Parent).
FR-03	User Profile Management	The system shall allow users to view and update personal account information where authorized.
FR-04	Student and Parent Management	Admin shall be able to create, edit, import, and manage student-parent records.
FR-05	Bus and Driver Management	Admin shall be able to manage bus records, driver assignments, and related operational data.
FR-06	Route Planning and Assignment	Admin shall be able to define routes and assign buses / drivers / students to routes.
FR-07	Driver Trip Operation	Driver shall be able to start, update, and complete trip sessions through the mobile app.

FR-08	Real-Time Bus Tracking	The system shall capture and display live bus location to authorized users.
FR-09	Boarding/Event Recording	The system shall record student boarding/alighting or attendance events (including NFC-based flow where applicable).
FR-10	Notification Delivery	The system shall send push notifications for alerts, operational updates, and emergency events.
FR-11	Emergency Alert Management	Admin/authorized users shall be able to issue and monitor emergency alerts.
FR-12	Reports and History	The system shall provide trip history and operational records for review and reporting.
FR-13	Data Import/Export	Admin shall be able to import and manage certain records using CSV templates.

Table 3.4.2 presents the key functional requirements of the SchoolWay system by specifying what the application must do for each user role. The requirements begin with identity and access functions, namely user registration/login, role-based access control, and profile management, which ensure secure and authorized use of the system. The table then defines core administrative operations, including student-parent management, bus-driver management, route planning, and data import/export, to support daily transport coordination.

The table also highlights operational functions for real-time service delivery. Driver trip operation, real-time bus tracking, and boarding/event recording ensure that trip activities are captured and monitored continuously. Notification delivery and emergency alert management provide timely communication and safety response mechanisms for parents, drivers, and administrators. Finally, reports and history functions ensure that past trip and operational data can be reviewed for monitoring, decision-making, and accountability.

3.4.3 Requirement Explanation

The above functional requirements ensure that SchoolWay supports end-to-end school transport operations. Authentication and role control protect access boundaries, while management modules maintain operational data integrity. Driver and tracking functions enable real-time visibility. Notification and alert functions support timely communication and safety response. Reporting and import capabilities improve administrative efficiency and long-term maintainability.

3.5 Project Milestone

3.5.1 Project I Timeline

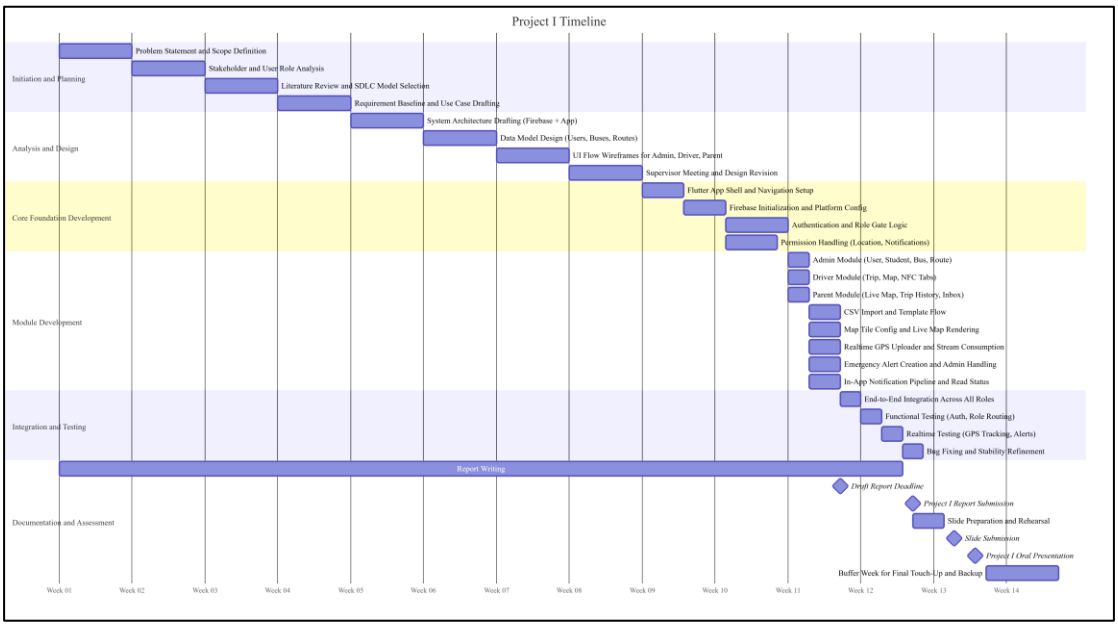


Figure 3.5.1: Project I Timeline in Gantt Chart

3.5.2 Project II Timeline

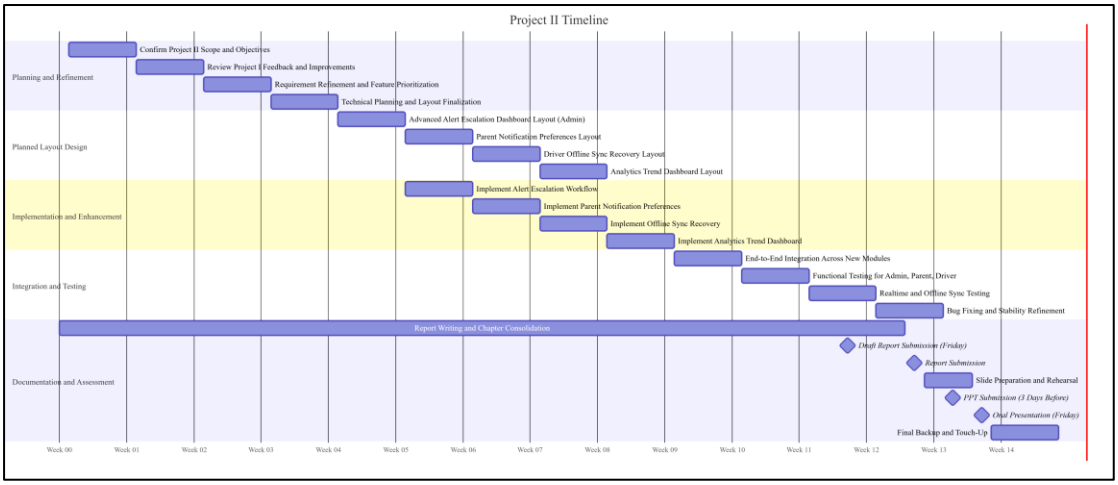


Figure 3.5.2: Project II Timeline in Gantt Chart

3.6 Concluding Remark

This chapter presented the development foundation of SchoolWay, including model selection, technical requirements, architecture design, functional requirements, and milestone planning. After evaluating Waterfall, Agile, and Prototype approaches, a Prototype-driven Agile model was selected to support evolving requirements and interface-intensive workflows. The defined system requirements, architecture components, and functional scope establish a clear roadmap for implementation, testing, and final deployment of the school transportation monitoring application.

CHAPTER 4

Preliminary Work

4.1 Software Setup and Configuration

This section explains the software environment setup, third-party integration, cloud and database configuration, deployment preparation, and monitoring setup for the SchoolWay mobile application. The focus is on installation and configuration procedures.

4.1.1 Environment Preparation

The SchoolWay development environment was prepared on Windows using Flutter, Dart, Visual Studio Code, Android SDK tools, and Firebase tooling. The software was downloaded from official sources to ensure version reliability and compatibility.

4.1.1.1 Flutter and Dart SDK Setup

Flutter SDK was downloaded from the official Flutter website: https://docs.flutter.dev/get-started/install/windows.

Installation was performed using the default package, then the Flutter bin directory was added to the system Path environment variable. No custom installation parameters were required.

Verification was performed using:

- flutter --version
- dart --version
- flutter doctor

```
C:\Users\Administrator>flutter --version
Flutter 3.41.5 • channel stable • https://github.com/flutter/flutter.git
Framework • revision 2c9eb20739 (5 weeks ago) • 2026-03-17 16:14:01 -0700
Engine • hash c1db59d880ca73dd86cec08a6663f287522d9f39 (revision 052f31d115)
(1 months ago) • 2026-03-17 20:29:11.000Z
Tools • Dart 3.11.3 • DevTools 2.54.2

C:\Users\Administrator>dart --version
Dart SDK version: 3.11.3 (stable) (Tue Mar 17 01:06:16 2026 -0700) on "windo
ws_x64"

C:\Users\Administrator>flutter doctor
Doctor summary (to see all details, run flutter doctor -v):
[√] Flutter (Channel stable, 3.41.5, on Microsoft Windows [Version
    10.0.26200.8246], locale en-MY)
[√] Windows Version (11 Pro for Workstations 64-bit, 25H2, 2009)
[√] Android toolchain - develop for Android devices (Android SDK version
    36.1.0)
[√] Chrome - develop for the web
[!] Visual Studio - develop Windows apps (Visual Studio Community 2026
    18.4.0)
    ! Unable to locate a Windows 10 SDK. If building fails, install the
      Windows 10 SDK in Visual Studio.
[√] Connected device (3 available)
[√] Network resources

! Doctor found issues in 1 category.
```

Figure 4.1.1.1: Futter and Dart SDK Setup Verification

4.1.1.2 Visual Studio Code setup

Visual Studio Code was downloaded from the official website: https://code.visualstudio.com/.

Installation used default settings. After installation, the Flutter and Dart extensions were installed from the VS Code Extensions Marketplace.

Verification was performed by checking:

- code –version
- Flutter and Dart extensions shown as Installed in VS Code

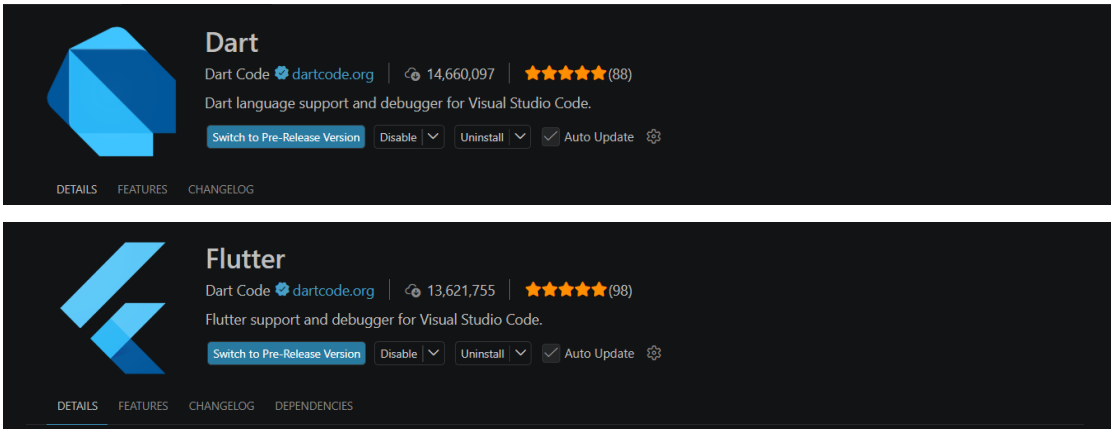


Figure 4.1.1.2: Flutter and Dart extensions from VS Code Marketplace

4.1.1.3 Android SDK and Device Tooling Setup

Android Studio was downloaded from: https://developer.android.com/studio.

Installation used default settings, with required SDK components installed through SDK Manager (Android SDK Platform, Build-Tools, Platform-Tools, and Emulator). For debugging, either an Android Emulator or USB-debugging-enabled physical device was configured.

Verification was performed using:

- flutter doctor (Android toolchain section)
- adb devices

Installation was considered successful when flutter doctor marked Android toolchain as available and adb devices detected an emulator or device.

```
C:\Users\Administrator>adb devices
List of devices attached
emulator-5554   device
```

Figure 4.1.1.3: Android SDK Setup Verification

4.1.1.4 Firebase CLI Setup

Firebase CLI setup followed the official documentation: https://firebase.google.com/docs/cli.

Node.js LTS was installed first from https://nodejs.org/, then Firebase CLI was installed globally. Default installation settings were used.

Verification was performed using:

- firebase --version
- firebase login

Installation was successful when the CLI returned a valid version and authentication completed without errors.



Figure 4.1.1.4: Firebase CLI Setup verification

4.1.1.5 FlutterFire CLI and Firebase Project Linking

FlutterFire CLI setup followed: https://firebase.google.com/docs/flutter/setup. The tool was installed with default settings and then used to connect the project to Firebase. The SchoolWay Firebase project was configured using the project identifier schoolway-04286806.

Verification was performed using:

- flutterfire --version
- flutterfire configure

Installation and configuration were successful when Firebase options were generated, and platform configurations were linked correctly.

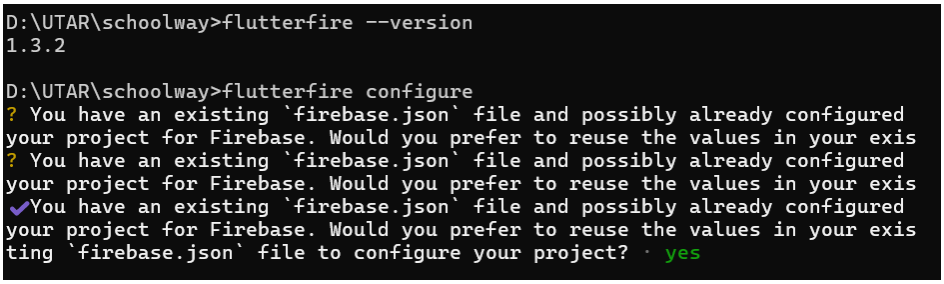


Figure 4.1.1.5: FlutterFire CLI Setup verification

4.1.1.6 Project Dependency Installation and Build Validation

After environment preparation, project dependencies were retrieved and the build process was validated.

Verification commands:

- flutter pub get
- flutter analyze
- flutter run

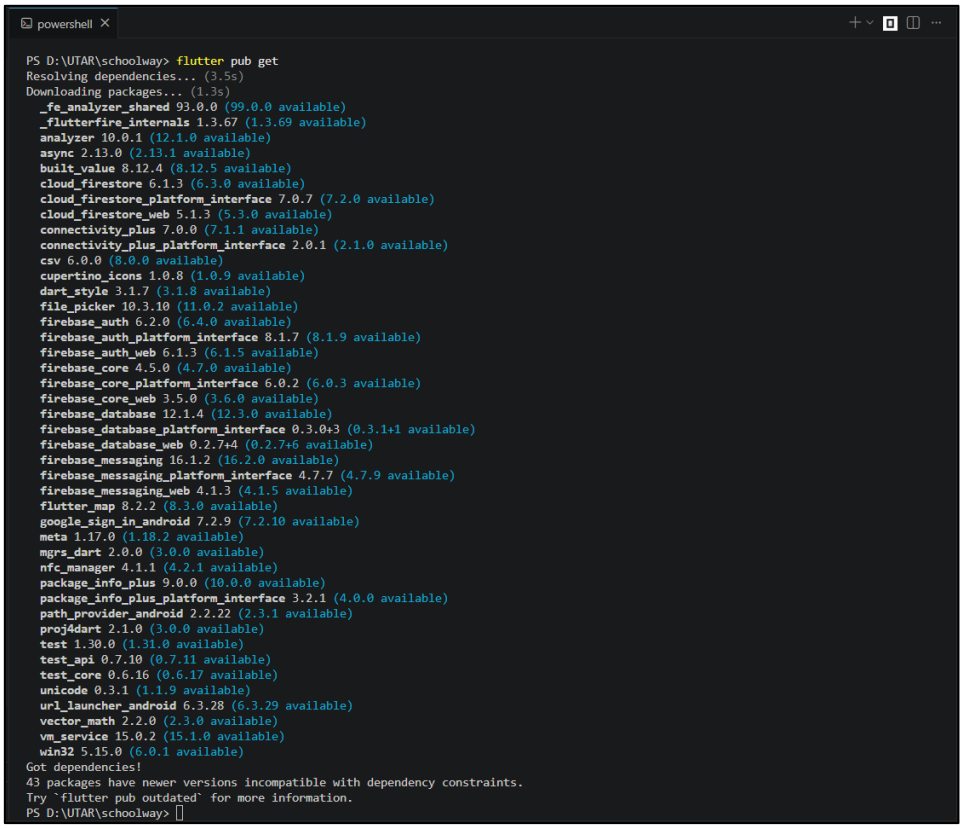


Figure 4.1.1.6: Dependency installation completion in project terminal.

Setup was confirmed as successful when dependencies were resolved, static analysis completed without blocking issues, and the application launched on the selected target platform.

Overall, the environment preparation process ensured a consistent and reproducible baseline for implementation, testing, and integration of SchoolWay modules.

4.1.2 Integration of External Services

SchoolWay integrates several external services to support authentication, communication, real-time tracking, and operational attendance workflows. These services include Firebase Authentication, Firebase Cloud Messaging, map and location services, and NFC services. Their integration ensures that system access is secure, transport updates are timely, and operational records are captured accurately.



Figure 4.1.2: External service architecture for authentication, messaging, map, location, and NFC

4.1.2.1 Authentication Service Integration

Firebase Authentication is configured for account-based login and Google sign-in. Platform credentials are configured for Android. After successful sign-in, user profile verification and role assignment are performed before navigation.

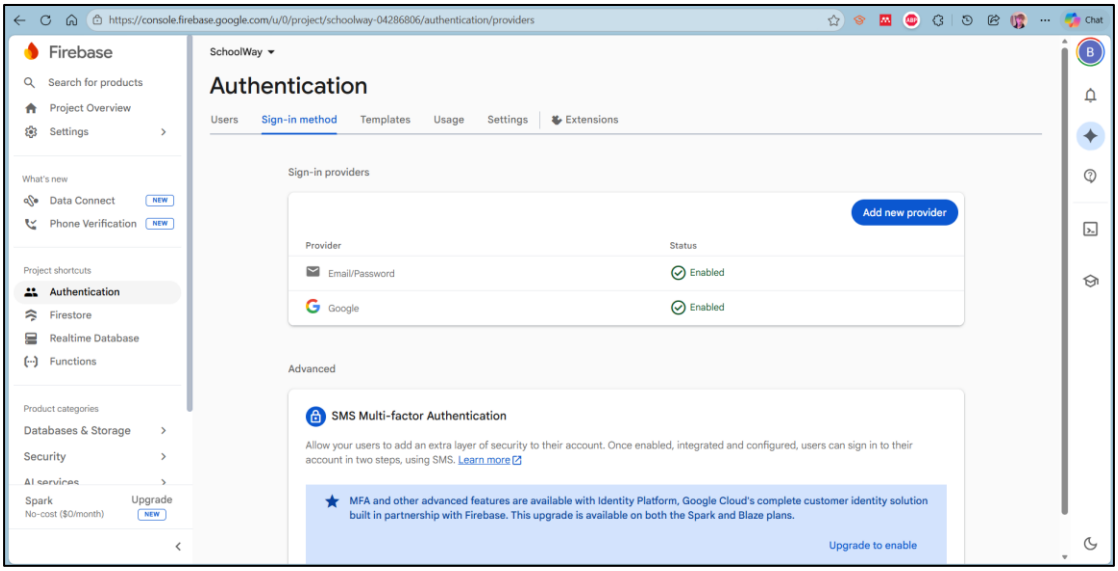


Figure 4.1.2.1: Firebase Authentication provider setup screen.

Installation and setup:

Firebase Authentication was prepared by creating and configuring a Firebase project in the Firebase Console, then enabling the required sign-in providers under the Authentication module. Email and password authentication was enabled for account-based access, and Google sign-in was enabled for federated login. The Android platform registration was completed in project settings so authentication traffic can be associated with the correct application identity.

Integration in application:

The authentication SDK was integrated into SchoolWay so users can sign in through either email credentials or Google sign-in. After successful authentication, the application retrieves the user identity and checks the corresponding profile record before routing the user to the correct module. This integration ensures that only authenticated users proceed to operational screens.

Configuration details:

Authentication configuration includes provider enablement, platform registration, and OAuth-related values for Google sign-in. These values are stored in the project configuration and should not be disclosed in full in documentation.

Verification:

Successful integration was verified by executing sign-in tests using valid credentials and confirming that authentication sessions were created correctly. Additional checks confirmed that signed-in users were routed according to their role information and that invalid or unregistered accounts were handled safely. This verification confirms that the authentication service is active and correctly integrated with user access flow.

4.1.2.2 Push Notification Service Integration

Firebase Cloud Messaging is initialized during startup. On successful authentication, the device token is obtained and synchronized with the user profile for push delivery readiness.

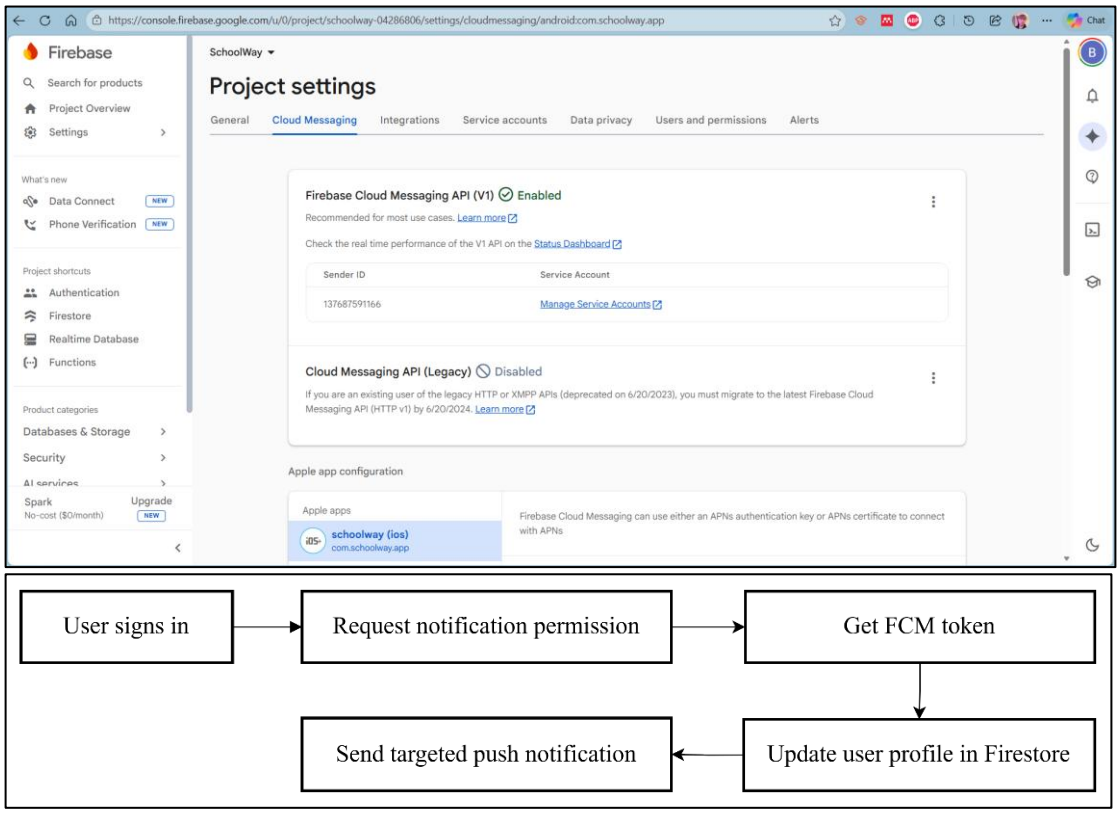


Figure 4.1.2.2: Cloud Messaging configuration and token registration flow.

Installation and setup:

Firebase Cloud Messaging was enabled in the same Firebase project through the Cloud Messaging module in Firebase Console. Notification support was prepared for Android

so the application can request permission and receive push messages at runtime. The messaging service setup was linked with the application as part of Firebase integration.

Integration in application:

The messaging SDK was integrated into SchoolWay during startup initialization, so notification services are prepared before normal operation. After user authentication, the system retrieves a device token and synchronizes it with the corresponding user profile record. This enables targeted message delivery based on authenticated user identity.

Configuration details:

Configuration includes notification permission behaviour, token registration flow, and platform messaging parameters. Any token or key referenced in documentation should be masked to protect security-sensitive values while still explaining how token-based delivery works.

Verification:

Integration was verified by confirming successful permission prompts, token retrieval, and token synchronization into the user profile. Test push notifications were then sent to validate end-to-end delivery behaviour. Results confirmed that messaging services are correctly configured and operational.

4.1.2.3 Map and Location Service Integration

Map display uses OpenStreetMap tiles through a Flutter map layer. Location services are initialized with runtime permission checks and high-accuracy position retrieval for driver tracking workflows.

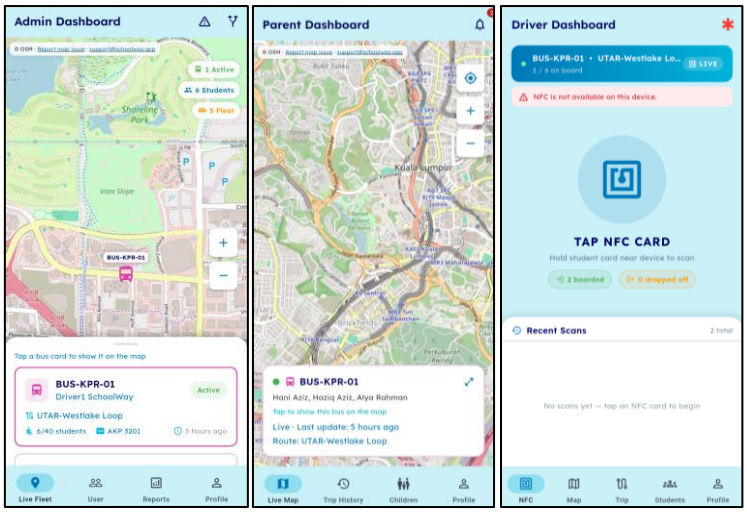


Figure 4.1.2.3: OpenStreetMap through Flutter Map Layer

Installation and setup:

Map and location functions were prepared by adding map rendering and geolocation packages to the project dependency configuration. OpenStreetMap was selected as the tile provider for map visualization, and location access requirements were prepared to allow runtime permission handling. Android location readiness was configured to support GPS-based tracking workflows.

Integration in application:

Map rendering was integrated into SchoolWay to display route context and vehicle positions in role-specific views. Location retrieval was integrated for high-accuracy tracking during driver operation, and position updates were reflected in live monitoring interfaces. This integration supports real-time visibility for both administrators and parents.

Configuration details:

Configuration includes map tile endpoint settings, location accuracy parameters, and runtime permission checks. The current map implementation using OpenStreetMap does not require a private API token. If a token-based map provider is adopted in future, key storage and masking requirements should be documented as part of security controls.

Verification:

Integration was verified by checking that map tiles loaded correctly, location permission prompts appeared as expected, and GPS coordinates were retrieved after user consent. Live map views were then observed to confirm that position updates were reflected during active operation. These checks verified that map and location services were functioning correctly.

4.1.2.4 NFC Service Integration

NFC access is configured for supported platforms to read student card identifiers. Reader integration is used in the driver workflow for boarding and alighting records.

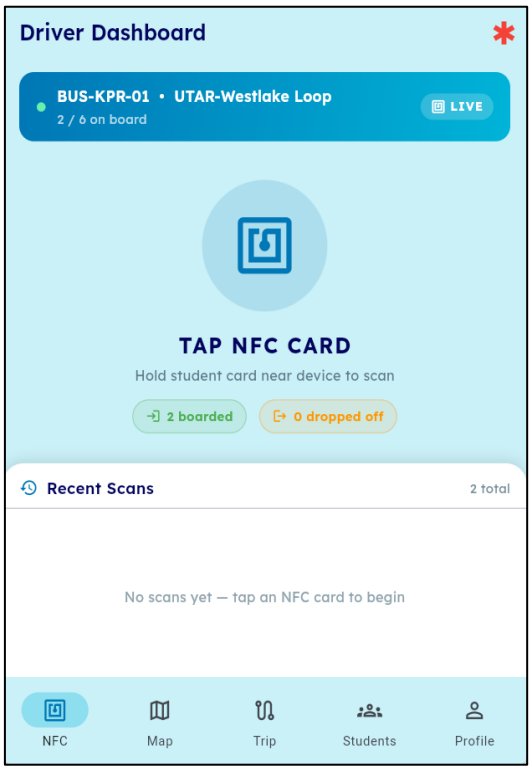


Figure 4.1.2.4: NFC access is usable on supported platforms

Installation and setup:

NFC support was prepared by adding NFC-related libraries and enabling NFC capability on supported mobile test devices. Setup focused on ensuring that NFC reading could be initiated in compatible runtime environments. This provided the required hardware-software base for card scanning workflows.

Integration in application:

NFC scanning was integrated into the driver workflow so card scans can be captured during boarding and alighting. The scanned value is decoded and matched against student data before event records are created. This integration enables attendance events to be captured as part of real-time bus operations.

Configuration details:

NFC configuration focuses on platform support checks, scan handling behaviour, and identifier matching logic. No external cloud API token is required for NFC scanning in

the current implementation. Because NFC capability depends on device hardware, unsupported environments must use fallback behaviour without scan execution.

Verification:

Integration was verified by performing scans with both valid and invalid NFC cards. Valid scans were confirmed through successful student matching and event recording, while invalid scans correctly triggered warning behaviour without incorrect data writes. Immediate UI updates after scan events further confirmed successful NFC workflow integration.

4.1.3 Database Configuration

SchoolWay uses Cloud Firestore as the primary operational database. The data model is organized into collections for users, buses, students, route sessions, boarding events, GPS tracking, emergency alerts, notifications, and planned routes. Composite indexes were configured to support filtered and ordered queries for dashboards, trip history, and activity reports. Firestore security rules were configured to require authenticated access across core collections. During development, permissive authenticated write rules were used to accelerate module integration and testing. Seed CSV datasets were prepared for buses, drivers, parents, students, and route assignments to speed up functional validation.

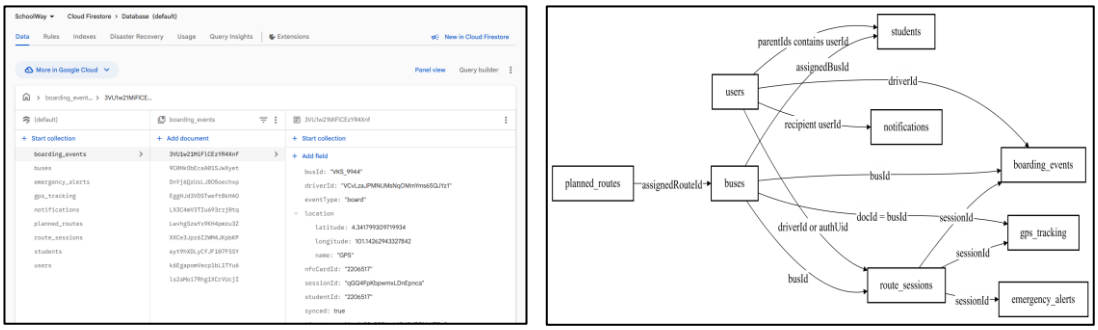


Figure 4.1.3.1: Firestore collection structure used by SchoolWay.

Figure 4.1.3.1 illustrates the Cloud Firestore database structure used in SchoolWay. The upper panel presents the actual Firestore console view, while the lower panel summarizes the logical relationships among core collections, including users, students, buses, route sessions, boarding events, GPS tracking, emergency alerts,

notifications, and planned routes. This structure supports role-based operation, trip monitoring, and event recording by linking operational entities through identifiers such as busId, sessionId, and user-related references. The dual presentation provides both implementation evidence and conceptual clarity of the data model used in the system.

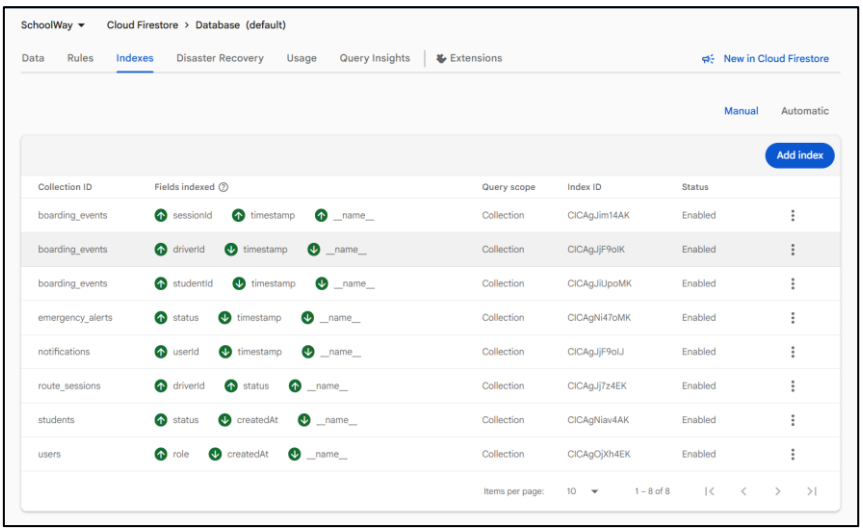


Figure 4.1.3.2: Composite index configuration for operational queries.

Figure 4.1.3.2 shows the composite index configuration defined for operational query performance in SchoolWay. These indexes are required to support filtered and ordered queries across high-usage datasets, especially for boarding events, route sessions, notifications, user lists, and emergency records. By predefining composite indexes, the system can execute dashboard and report queries efficiently without runtime index creation errors. This configuration is important for maintaining responsive data retrieval in live monitoring and historical analysis screens.

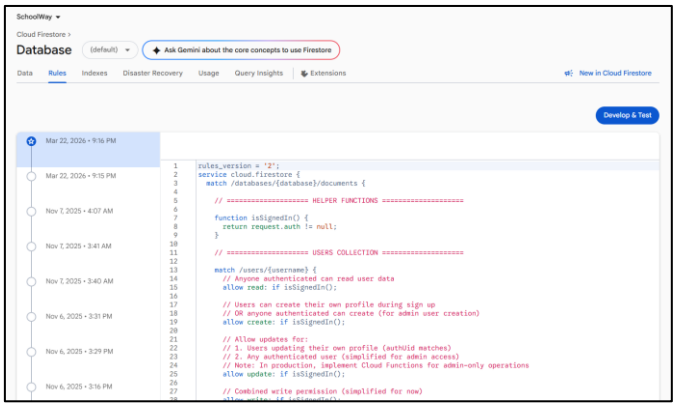


Figure 4.1.3.3: Firestore security rule setup in project console.

Figure 4.1.3.3 presents the Firestore security rule configuration used during the development stage. The rules enforce authenticated access for core collections so only signed-in users can read or write operational data. For faster feature integration and testing, write permissions were temporarily configured in a permissive authenticated mode, while destructive actions such as selected delete operations remain restricted. This setup provides a practical balance between development agility and baseline access control, and it forms the foundation for future production hardening with stricter role-based authorization.

4.1.4 Cloud Services

Cloud service setup for SchoolWay was implemented through a single Firebase project that serves as the central backend for authentication, configuration, and runtime service connectivity. Platform-specific registration was completed for Android, and the generated Firebase app identifiers and configuration values were integrated into the Flutter application so cloud services can initialize correctly at startup. In addition to project linking, startup flow design includes cloud bootstrapping, essential permission handling, and push messaging initialization before users enter operational modules.

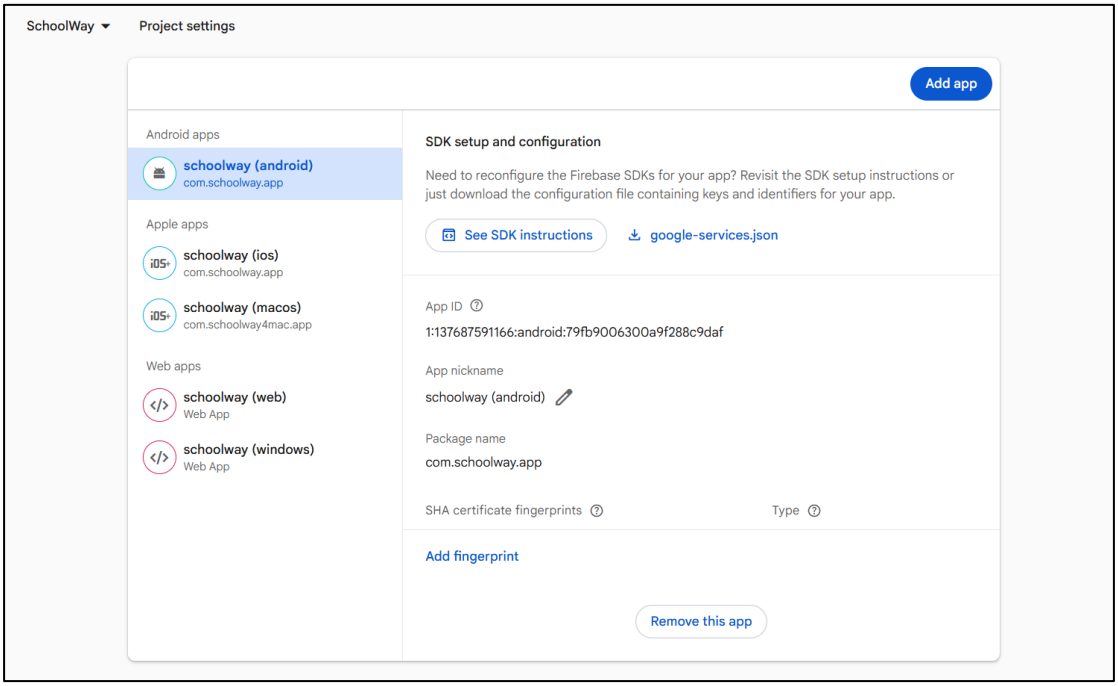


Figure 4.1.4.1: Firebase project overview and platform app registration.

Figure 4.1.4.1 shows the Firebase project overview and platform application registration used in SchoolWay. The figure confirms that the Android application is registered in the Firebase project with its platform identity and package mapping. This registration is required so Firebase services can correctly associate authentication, messaging, and other cloud interactions with the deployed mobile app. The figure therefore provides configuration evidence that cloud-platform linkage was completed before runtime integration.

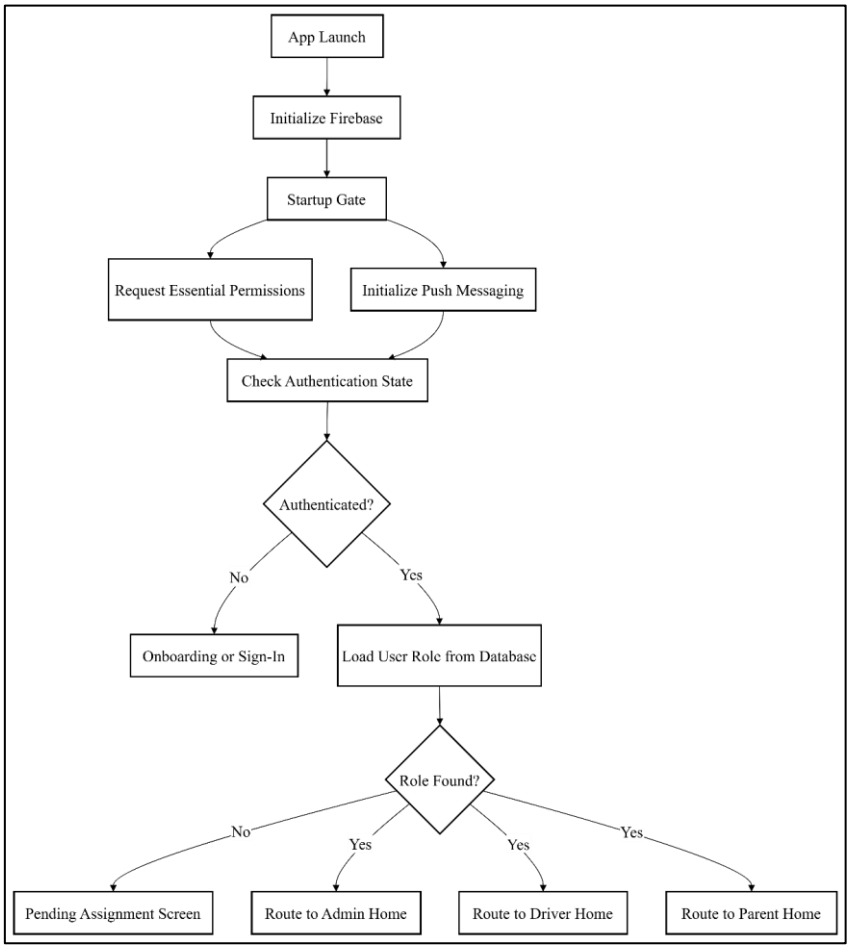


Figure 4.1.4.2: Application startup sequence with cloud initialization gate.

Figure 4.1.4.2 illustrates the application startup sequence with the cloud initialization gate. As shown in the flow, the app launches, initializes Firebase services, executes startup checks, requests essential permissions, and initializes push messaging before proceeding to authentication and role routing. This gated startup approach ensures that cloud-dependent features are prepared before user-facing operations begin, reducing runtime errors and improving initialization reliability across different user roles.

4.1.5 Deployment and Hosting

For Android deployment, the SchoolWay application was prepared for release distribution after the signing configuration and build environment were finalized. The deployment process began with build verification on the target platform, followed by dependency cleanup and package retrieval to ensure the project was in a stable state before release generation. As shown in Figure 4.1.5, the application was successfully built in release mode using the Android build pipeline, and the generated APK confirmed that the release artifact was produced correctly. This release build checkpoint verifies that the application can be packaged properly for Android deployment after configuration and dependency validation.

The deployment process also includes validation of authentication settings and cloud endpoint connectivity to ensure the application can connect correctly to Firebase services during runtime. After the release build is generated, the APK can be uploaded to a hosting or distribution channel for installation testing or production deployment. Post-deployment smoke testing is then carried out to confirm that sign-in, map loading, and core module navigation operate correctly on the target device. These checks ensure that the release package is functionally ready for distribution and that the application behaves as expected after deployment.

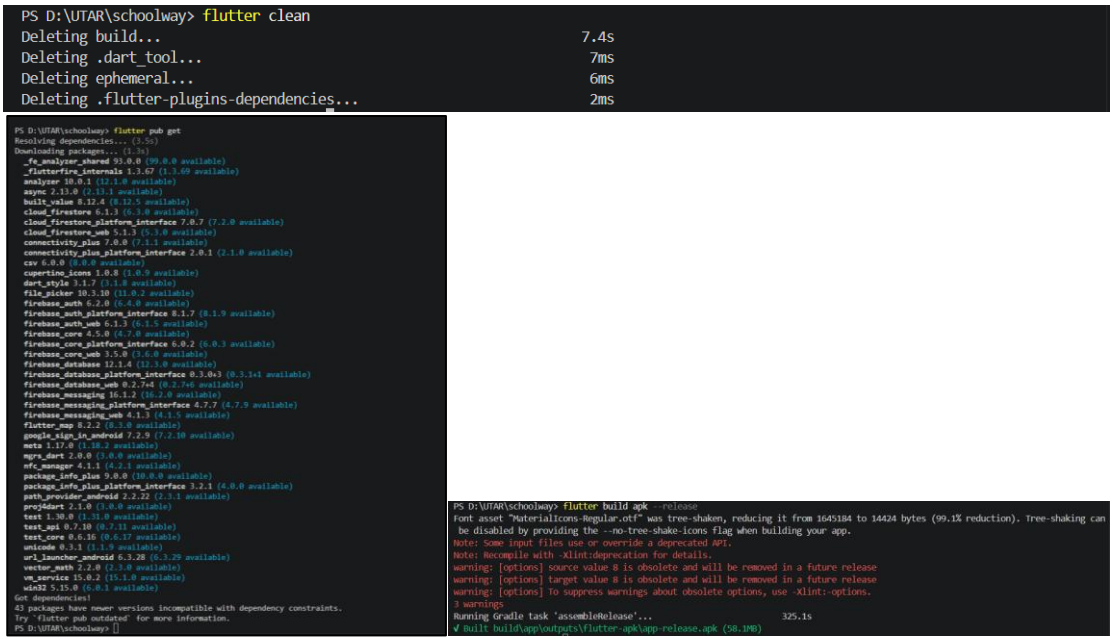


Figure 4.1.5: Android release build pipeline and signing checkpoint.

4.1.6 Monitoring and Analytics

System monitoring in SchoolWay currently relies on runtime error logging, operational dashboards, and Firestore-based activity reporting. The application captures key operational events and makes them accessible through role-specific monitoring interfaces. Administrative report screens summarize attendance events, bus activity, driver activity, and emergency records, providing practical operational oversight during development and pilot deployment phases.

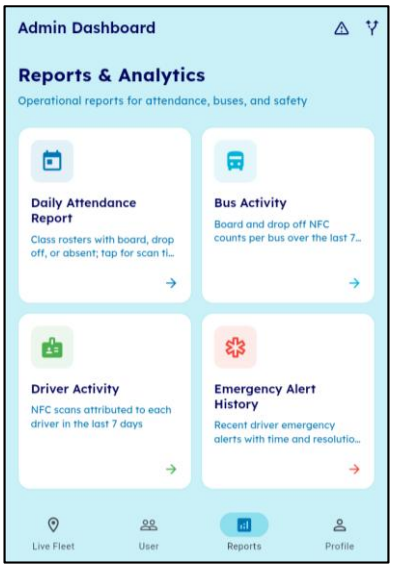


Figure 4.1.6: Admin reporting interface used for operational monitoring.

Figure 4.1.6 shows the Admin reporting interface used for operational monitoring, presenting four report cards that focus on specific monitoring areas: the Daily Attendance Report displays class rosters with student boarding, drop-off, or absence status, allowing administrators to tap the card to scan time-based data and verify attendance patterns across classes; Bus Activity summarizes boarding and drop-off NFC scan counts for each bus over the last seven days, enabling administrators to review transport volume and identify anomalies in bus usage; Driver Activity shows NFC scans attributed to each driver over the last seven days, helping administrators assess driver engagement and verify that attendance events are being recorded during operation; and Emergency Alert History displays recent driver-initiated emergency alerts with timestamps and resolution status, allowing administrators to monitor safety incidents and track response actions.

At the current development stage, a dedicated analytics software development kit (SDK) is not integrated into the system. This is identified as future enhancement work for supporting deeper user behaviour analysis, event funnel tracking, and trend-based insights. The current implementation focuses on real-time operational reporting rather than longitudinal analytics, which is appropriate for the pilot stage of development.

4.2 Implementation Details

This section focuses on technical realization of features, including module-level behaviour, data flow, and runtime processing.

4.2.1 Authentication and Role Routing

The authentication implementation supports both account-based login which is email and password and Google-based federated authentication.

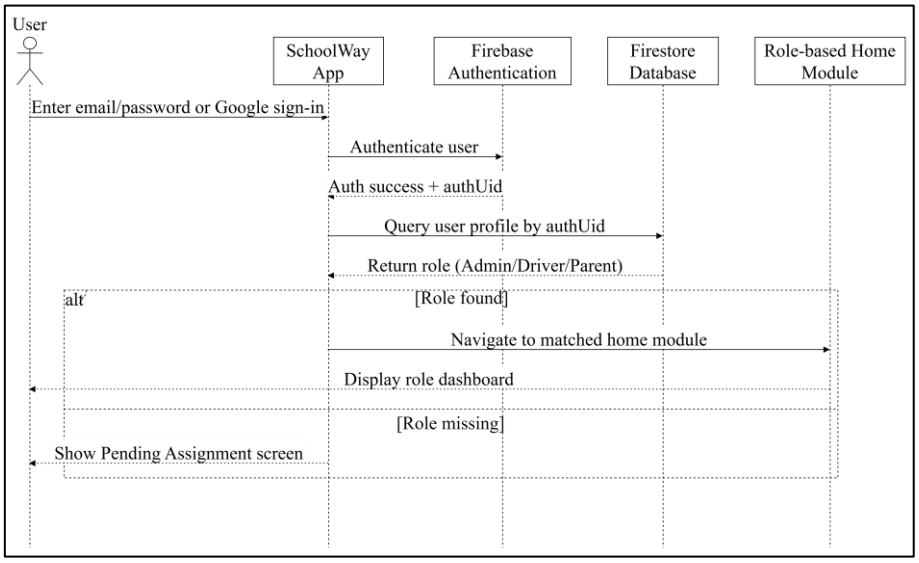


Figure 4.2.1: Authentication sequence from credential submission to role-based navigation.

Figure 4.2.1 illustrates the authentication sequence from credential submission through role-based navigation. When a user enters their credentials or initiates Google sign-in, the SchoolWay app submits these credentials to Firebase Authentication for validation. Upon successful authentication, Firebase returns an authentication token and a unique authUid. The application then queries the Firestore database to retrieve the user profile associated with that authUid. From this profile record, the system extracts the user's role metadata such as Admin, Driver, or Parent. With the role determined, the application navigates to the corresponding home module and displays the role-specific dashboard. If the role metadata is missing or has not yet been assigned, the system displays a Pending Assignment screen, preventing access to role-restricted features until administrative provisioning is complete.

This architecture ensures that users are routed to their correct operational context immediately upon login, while maintaining a controlled handoff mechanism for accounts awaiting role assignment.

4.2.2 Startup Gate and Permission Initialization

The startup gate is a critical initialization phase that executes before users can access any operational feature.

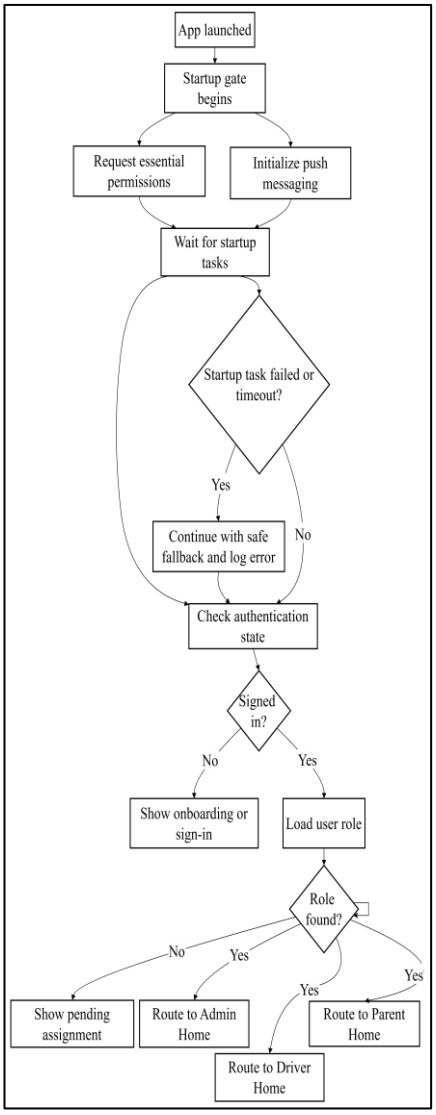


Figure 4.2.2: Startup gate flow for permission and messaging initialization

Figure 4.2.2 shows the startup gate flow, which begins immediately when the application is launched. The startup sequence initiates two concurrent initialization tasks: requesting essential device permissions (such as location and notification access) and initializing cloud push messaging infrastructure. These tasks run in parallel to minimize startup latency. The application then enters a waiting state until both initialization tasks complete. If either task fails or exceeds the timeout threshold, the system logs the error and continues with safe fallback mechanisms, ensuring that a single initialization failure does not prevent user access. After the startup gate tasks complete, the application checks the authentication state. If no user is signed in, the onboarding or sign-in screen is displayed. If a user is authenticated, the system loads their role metadata from Firestore. The application then validates whether a role has been assigned. Users with an assigned role (Admin, Driver, or Parent) are routed to their respective home modules; users without an assigned role are shown the pending assignment screen.

This design reduces the likelihood of late permission failures during operation, improves runtime reliability, and ensures that critical services are initialized before users begin interacting with transport monitoring features.

4.2.3 GPS Tracking Implementation

The GPS tracking module is activated when the driver starts an active route session, at which point the uploader is initialized with session context such as, busId and sessionId to ensure each location update is linked to the correct trip record.

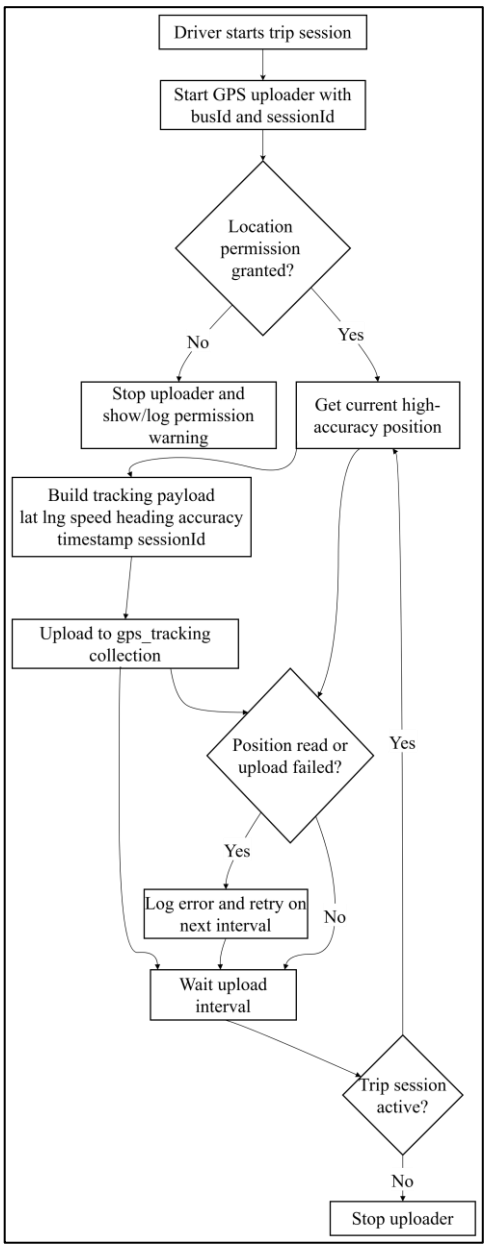


Figure 4.2.3: GPS upload cycle for active driver session

As illustrated in Figure 4.2.3, the module first checks whether location permission is granted. If permission is denied, the uploader does not start, and the condition is handled gracefully by recording a warning or error log without crashing the application. When permission is available, the system acquires high-accuracy position data at a configured periodic interval and constructs a tracking payload containing latitude, longitude, heading, speed, timestamp, and session linkage fields. This payload is then written to cloud tracking storage which is gps_tracking collection to support live fleet visibility and downstream reporting. If a position read or upload operation fails intermittently due to network or sensor instability, the failure is logged and the periodic scheduler continues at the next interval, rather than terminating the tracking loop. The cycle repeats while the trip session remains active and stops automatically when the route session ends.

This implementation provides resilient, session-bound GPS telemetry suitable for real-time monitoring during school transport operations.

4.2.4 NFC Boarding Event Implementation

The NFC boarding module is designed to capture student boarding activity quickly and reliably during live route operations.

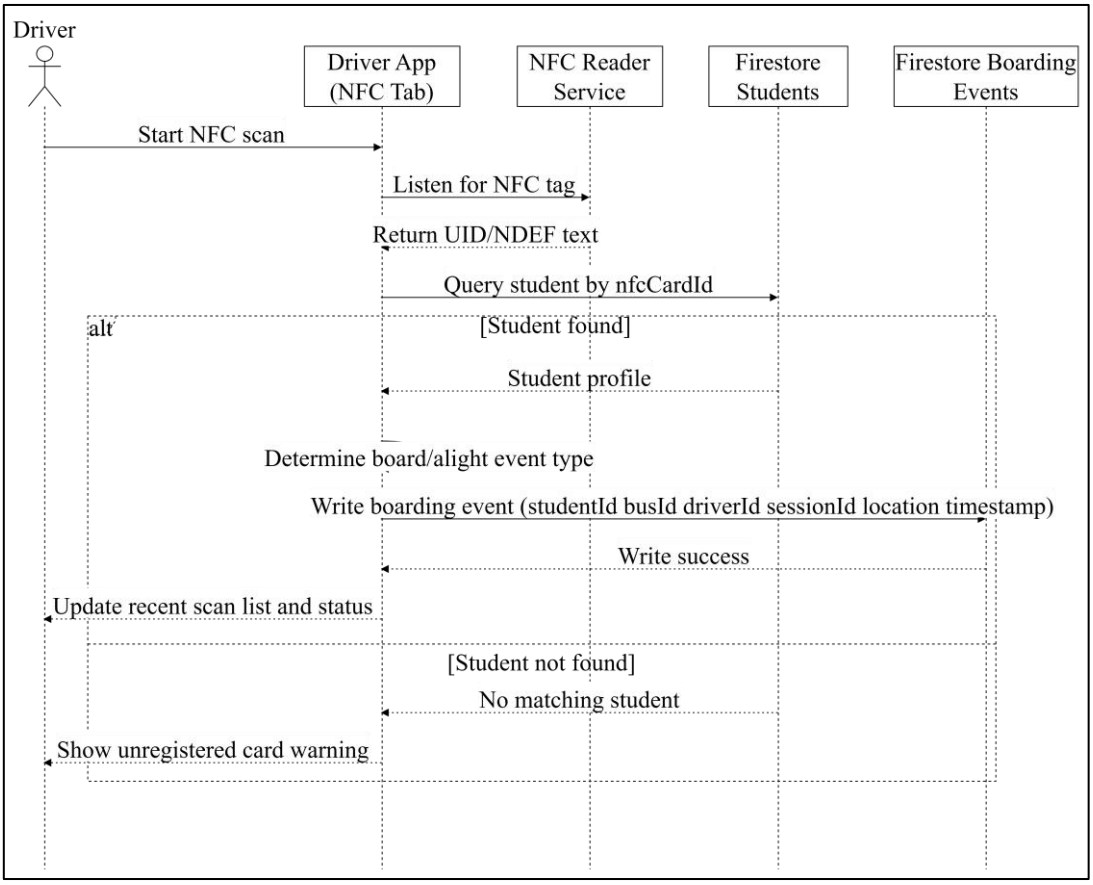


Figure 4.2.4: NFC scan-to-event recording pipeline.

As shown in Figure 4.2.4, the process begins when the driver starts NFC scanning from the Driver NFC interface. The NFC reader service listens for nearby tags and decodes either a supported NDEF text payload or a fallback identifier value (UID) when NDEF text is unavailable. The resulting card value is then used to query student records in Firestore based on the registered NFC card identifier. If a matching student profile is found, the system determines the event type (boarding or alighting) according to the operational context and writes a boarding event record containing studentId, driverId, busId, sessionId, timestamp, and location details. This record has persisted in the boarding events collection for attendance tracking and reporting. If no matching student is found, the system handles the scan gracefully by showing an unregistered-card warning and avoiding invalid event insertion.

To support fast operational verification by drivers during pickup and drop-off, the UI provides immediate scan feedback after each tap and updates a recent event list in near real time. This allows the driver to confirm that each scan has been accepted, identify unmatched cards immediately, and continue passenger flow without manual reconciliation during the trip. Figure 4.2.4 therefore represents the full NFC scan-to-event recording pipeline, from tag read and student lookup to contextual event writing and on-screen confirmation.

4.2.5 Live Map Rendering Implementation

The live map implementation provides real-time visual tracking by consuming GPS updates from cloud tracking streams and rendering moving bus markers on OpenStreetMap tiles.

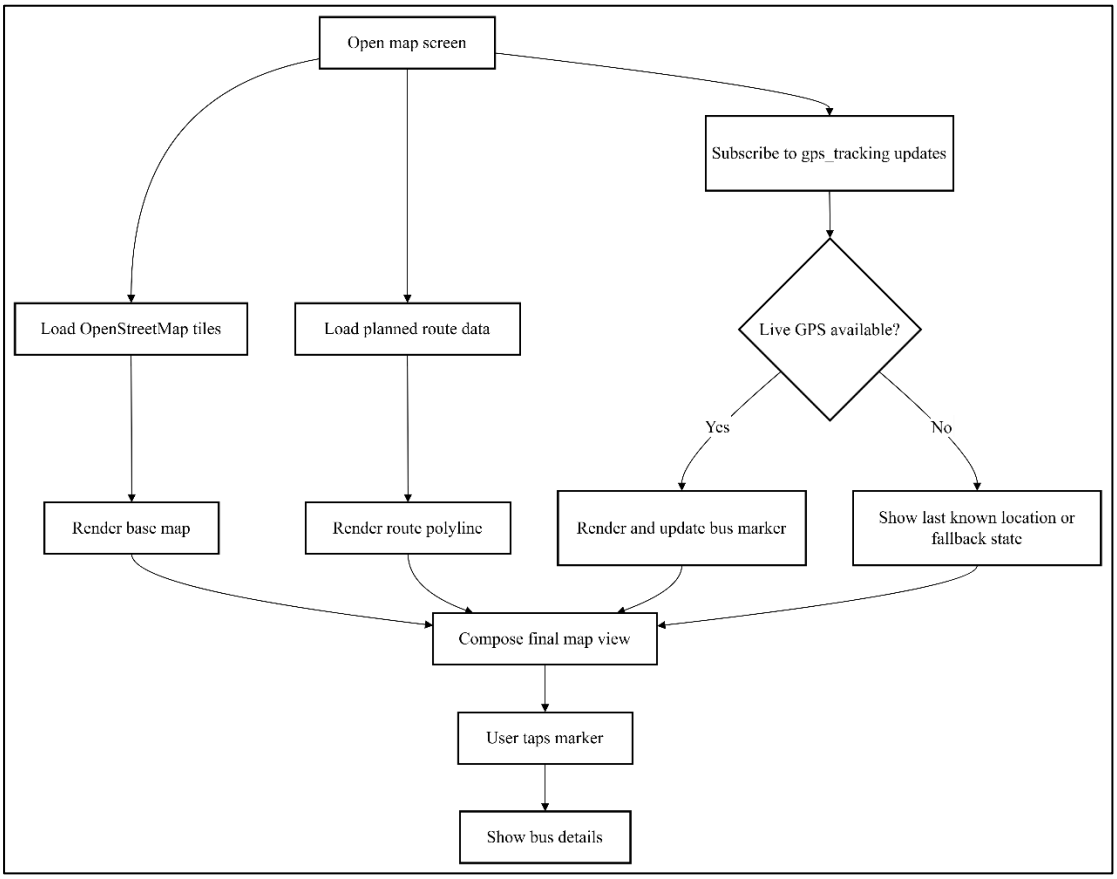


Figure 4.2.5: Real-time map rendering with route and position overlays.

As illustrated in Figure 4.2.5, once the map screen is opened, the module initializes three core data layers: base map tiles, planned route geometry, and live vehicle position

updates. The base layer renders OpenStreetMap tiles for geographic context, while the route layer draws assigned route polylines to show expected travel paths. In parallel, the live layer subscribes to gps_tracking updates and continuously refreshes bus markers when new coordinates are received. If live updates are temporarily unavailable, the interface falls back to the last known location or a safe placeholder state, preserving continuity for users.

Planned route overlays and fit-to-route camera logic improve situational visibility by automatically framing relevant route extents and active vehicle positions within the viewport. This reduces manual panning and helps users quickly assess whether a bus is on or off the expected path. For admin and parent modules, the composed map view emphasizes fleet or child-related transport visibility through route-plus-marker overlays and marker detail interactions. For driver modules, the map combines own-position context with assigned route visualization, supporting operational awareness during trip execution. Figure 4.2.5 therefore represents the full real-time rendering flow, from map initialization and data subscription to layered route and position

4.2.6 Notification Pipeline Implementation

The notification pipeline is implemented as a full flow from permission and token provisioning to in-app presentation and inbox state management.

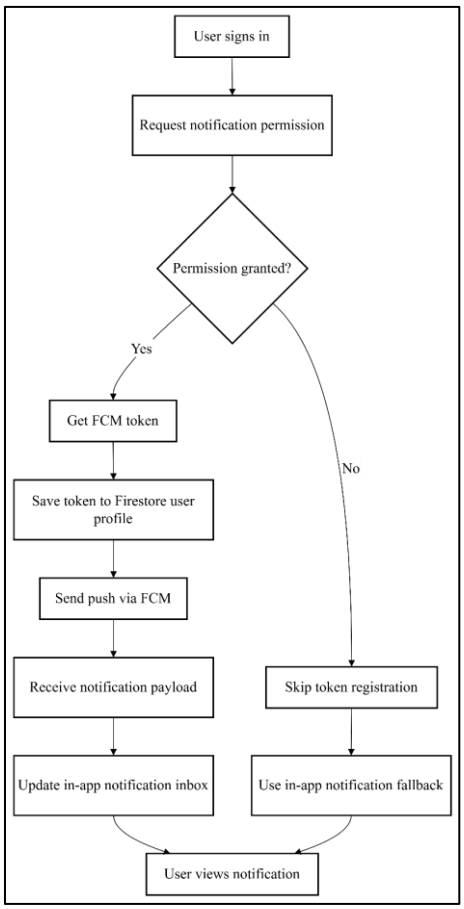


Figure 4.2.6: Notification flow from token sync to inbox presentation.

As shown in Figure 4.2.6, notification handling begins when a user signs in and the application requests notification permission from the device. If permission is granted, the system retrieves the Firebase Cloud Messaging (FCM) token and synchronizes it to the user profile in Firestore so cloud notifications can be targeted correctly. If permission is not granted, token registration is skipped and the application continues with an in-app fallback path to avoid blocking core usage. For permitted users, push payloads sent through FCM are received by the client and mapped into application notification records. Foreground notification behaviour is explicitly configured to ensure visibility while the app is active, so important transport and safety updates are not silently missed.

Parent-facing notification inbox screens are synchronized with cloud notification records, allowing users to view recent alerts in chronological order and inspect operational messages such as attendance or emergency-related updates. The inbox also supports read-state updates, where opened items are marked as read and reflected in the interface state, improving message tracking and reducing repeated alert noise. This architecture ensures reliable notification delivery across permission conditions while preserving a consistent in-app communication channel. Figure 4.2.6 therefore represents the end-to-end notification flow from token synchronization to inbox presentation and read-status handling.

4.2.7 CSV Import and Bulk Administration Implementation

The CSV import module provides a structured onboarding pipeline for high-volume administrative data setup.

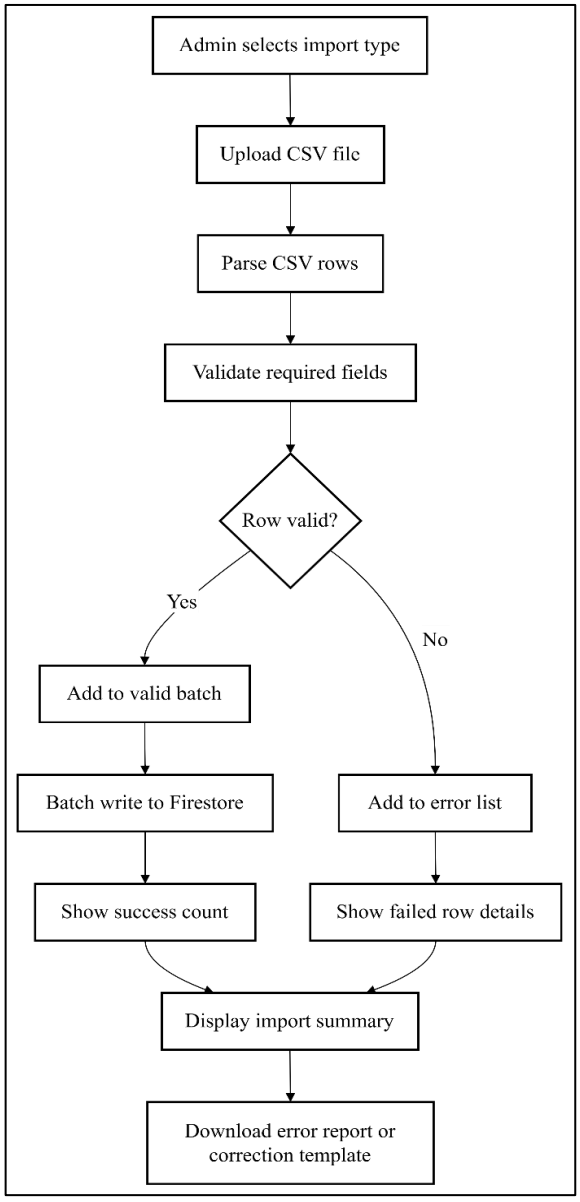


Figure 4.2.7: CSV import workflow and batch processing feedback.

As illustrated in Figure 4.2.7, the process begins when an administrator selects an import type and uploads a CSV file. The system then parses rows, validates required fields, and evaluates each record for schema and business-rule compliance before database insertion. Valid rows are grouped into a batch and written to Firestore using bulk operations for efficiency, while invalid rows are separated into an error list with row-level failure details. During execution, the interface provides progress visibility through success counts and failed-row reporting, followed by an import summary that allows administrators to assess completion quality. The workflow also supports downloading an error report or correction template so failed records can be fixed and re-imported quickly.

This implementation significantly accelerates initial system setup for students, users, buses, and planned route assignments, while preserving robustness through validation controls and transparent batch feedback.

4.3 System Operation

This section presents visual operation evidence using screenshots for implemented modules and wireframe layouts for planned modules not yet completed.

4.3.1 Design Layout

For modules not yet fully implemented, planned layouts should be presented as wireframes with interaction intent.

Planned Layout Figures

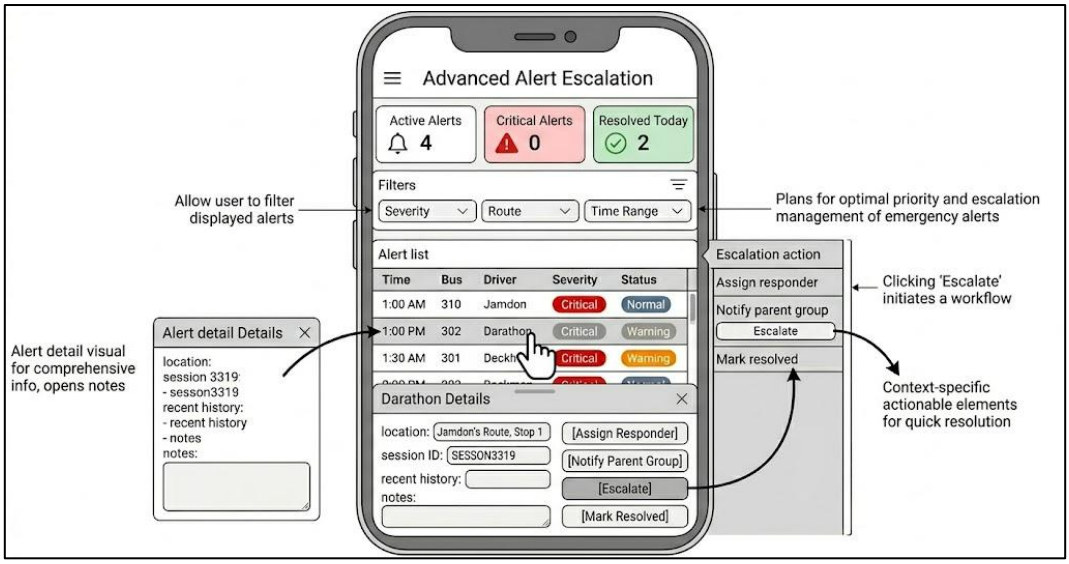


Figure 4.3.1.1: Planned Advanced Alert Escalation Dashboard Layout (Admin).

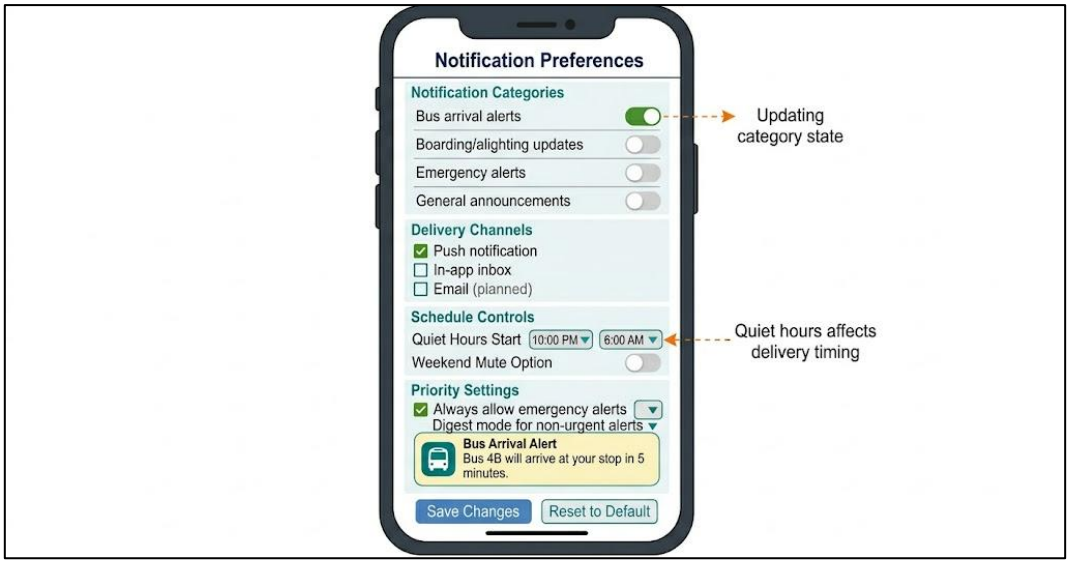


Figure 4.3.1.2: Planned Parent Notification Preferences Layout.

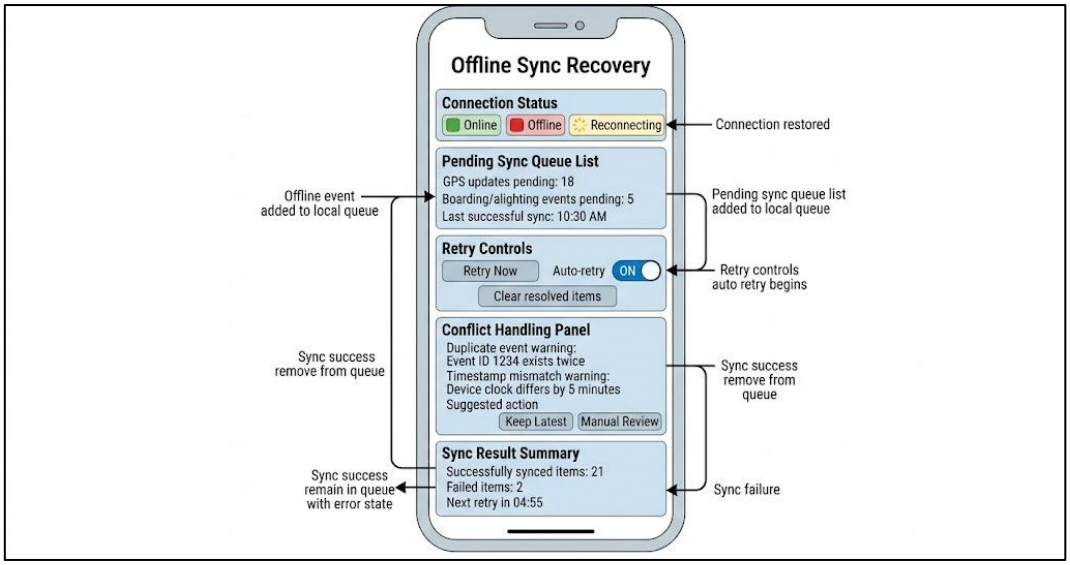


Figure 4.3.1.3: Planned Driver Offline Sync Recovery Layout.

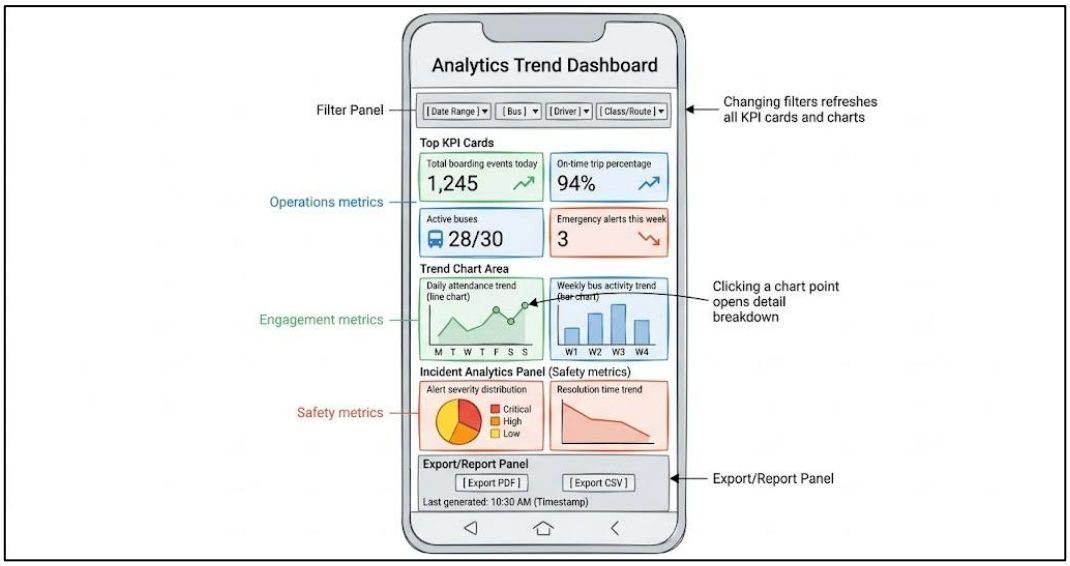


Figure 4.3.1.4: Planned Analytics Trend Dashboard Layout.

4.3.2 Functional Operation

4.3.2.1 Onboarding and Authentication Module

The user starts from onboarding pages and enters sign-in. After authentication, role-based routing sends the user to the corresponding dashboard. Role selection is system-assigned via database, not user-selected on UI.

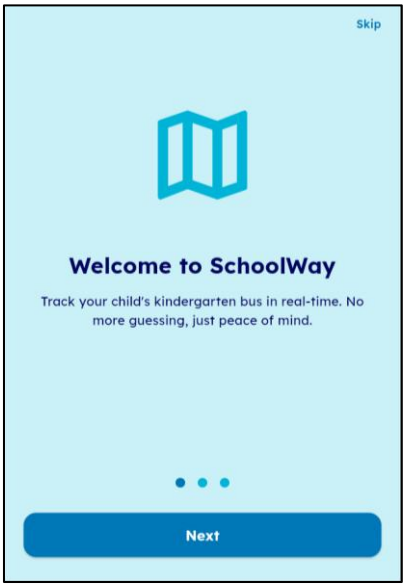


Figure 4.3.2.1.1: Onboarding entry screen of SchoolWay.

Figure 4.3.2.1.1 shows the initial SchoolWay onboarding screen, which appears before authentication to introduce the system's core purpose. Users are provided with two simple navigation controls. The Next button allows them to proceed through the remaining introduction slides step by step. Alternatively, the Skip option lets users completely bypass the onboarding process and proceed immediately to the main login page.

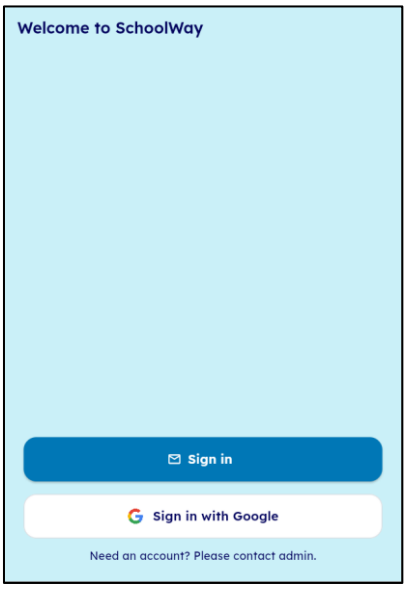


Figure 4.3.2.1.2: User sign-in interface with authentication options.

Figure 4.3.2.1.2 displays the SchoolWay sign-in interface, offering two authentication methods: standard email/password login via the "Sign in" button, and single sign-on via the "Sign in with Google" button. A message directs new users to contact the administrator for an account. This restricted, admin-managed registration process ensures that only authorized individuals can access the SchoolWay system.

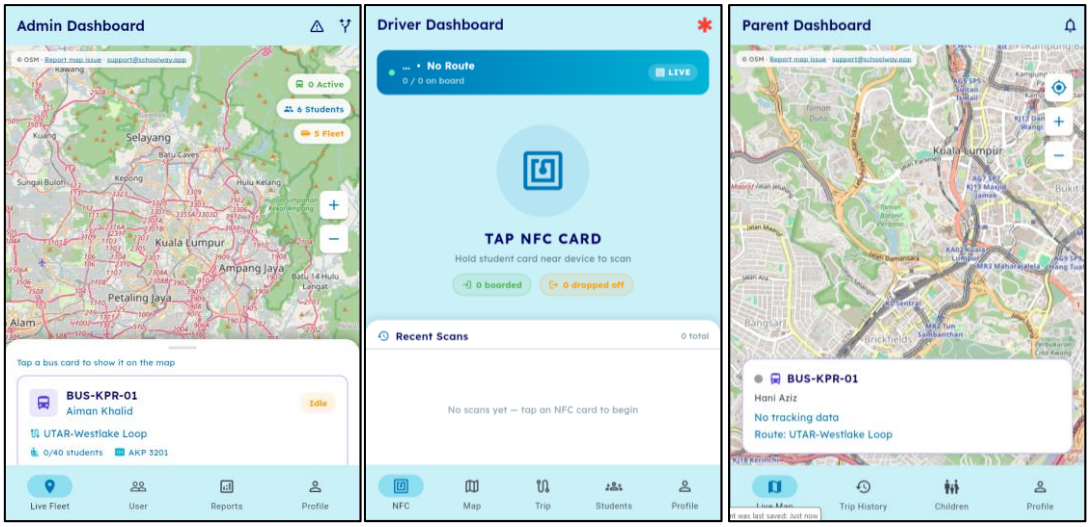


Figure 4.3.2.1.3: Successful authentication and role-based landing view.

Figure 4.3.2.1.3 shows the successful authentication result and the role-based landing view after login. After a user signs in, the application checks the account role stored in the database and automatically routes the user to the correct dashboard. The user does not manually select a role from the interface; instead, the system assigns the appropriate landing page based on the role linked to the account. In this way, Admin users are directed to the Admin dashboard, Driver users are directed to the Driver dashboard, and Parent users are directed to the Parent dashboard. If no role has been assigned yet, the system shows a pending assignment screen until the administrator updates the account.

4.3.2.2 Admin Module Operation

The administrator monitors live fleet status, manages users and students, imports data, manages routes, handles emergency alerts, and reviews operational reports.

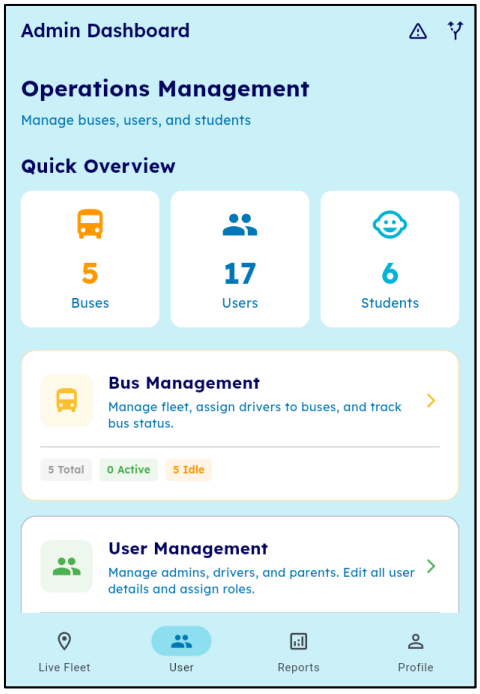


Figure 4.3.2.2.1: Admin Dashboard Main Navigation.

Figure 4.3.2.2.1 shows the Admin Dashboard main navigation screen. This screen serves as the central control panel for the administrator and provides access to the main operational functions of SchoolWay. The dashboard displays summary cards for buses, users, and students, allowing the administrator to quickly review the current system status. The bottom navigation bar provides direct access to the live fleet map, user-related functions, reports, and profile settings, making this screen the main entry point for administrative work.

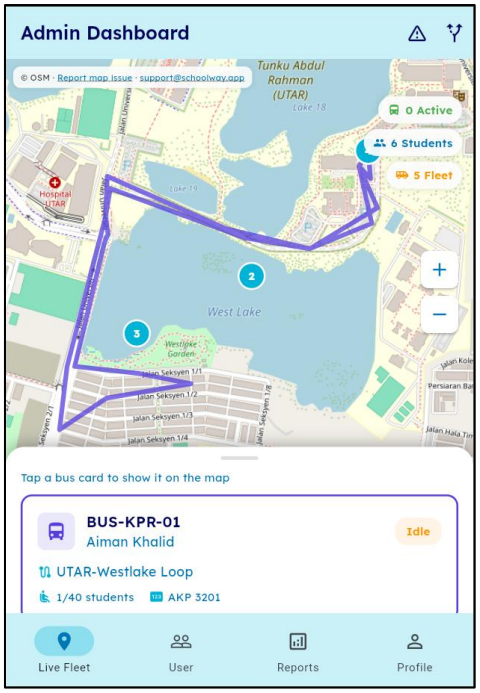


Figure 4.3.2.2.2: Live Fleet Monitoring Map in Admin Module.

Figure 4.3.2.2.2 shows the Live Fleet Monitoring Map in the Admin module. This screen allows the administrator to observe the real-time location of buses on the map and review route movement visually. The map view is used to monitor active fleet status, identify the current position of each bus, and support operational supervision during transport activities. The information card at the bottom provides summary details such as bus identity, route name, and current activity status.

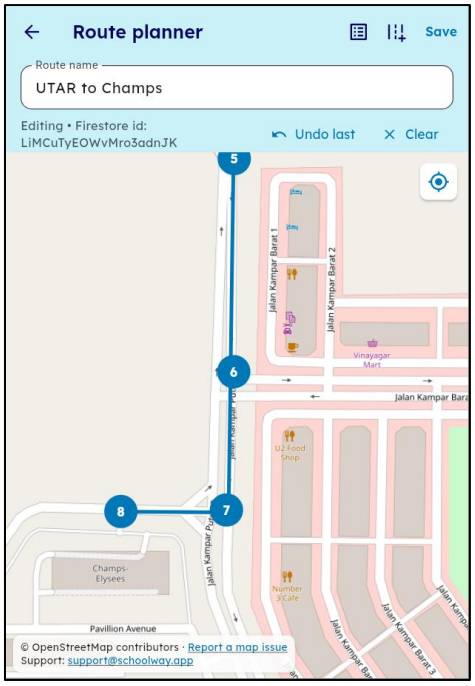


Figure 4.3.2.2.3: Route Planner Screen with Waypoint Planning Controls.

Figure 4.3.2.2.3 shows the Route Planner screen with waypoint planning controls. This interface is used by the administrator to design and edit bus routes through map-based waypoint selection. The route line and markers indicate the planned path, while the top controls provide actions such as saving, undoing, or clearing the current route. This screen supports route setup and modification, which are important for preparing transport paths before deployment.

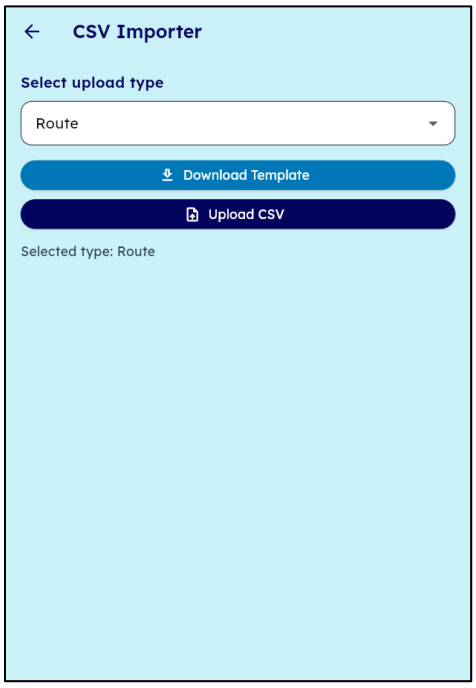


Figure 4.3.2.2.4: CSV Importer Screen for Bulk Data Setup.

Figure 4.3.2.2.4 shows the CSV Importer screen for bulk data setup. This screen is used to upload structured CSV files for importing operational data such as routes, buses, students, parents, drivers, or users. The Download Template button allows the administrator to obtain a prepared file format, while the Upload CSV button starts the import process. This function speeds up large-scale data entry and reduces the need for manual record creation.

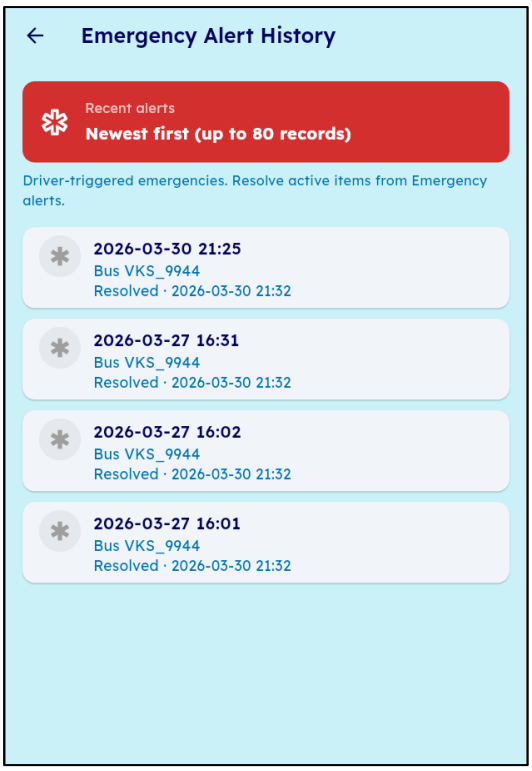


Figure 4.3.2.2.5: Emergency Alerts Management Screen.

Figure 4.3.2.2.5 shows the Emergency Alerts Management screen. This screen lists recent emergency records and allows the administrator to review alert history, status, and resolution information. The interface is used to monitor critical incidents that may occur during transport operations and provides a centralized view for safety response. By displaying resolved and active items, the screen helps the administrator track how alerts are handled over time.

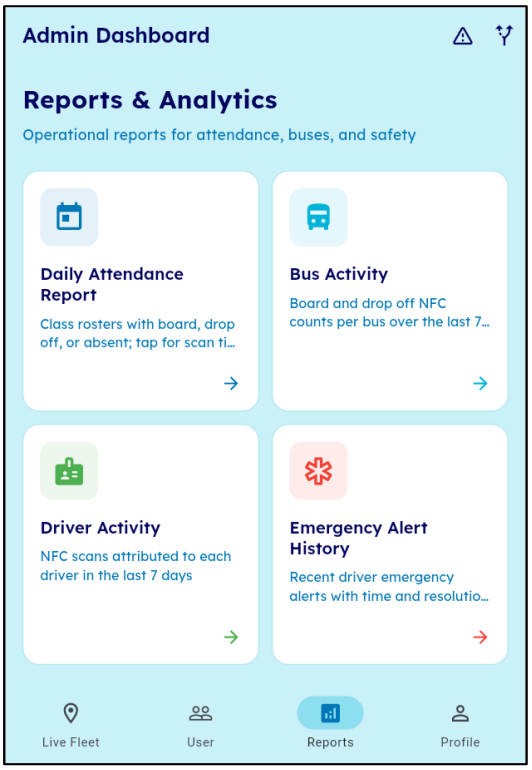


Figure 4.3.2.2.6: Administrative Reporting and Analytics Summary screen.

Figure 4.3.2.2.6 shows the Administrative Reporting and Analytics Summary screen. This interface provides a summary of operational reports, including daily attendance, bus activity, driver activity, and emergency alert history. Each report card gives the administrator a quick path to different monitoring categories so that operational performance can be reviewed efficiently. This screen supports decision-making by presenting key transport and safety information in a compact dashboard format.

4.3.2.3 Driver Module Operation

The driver starts a route session, updates live location, performs NFC scans, and tracks assigned students during operation.

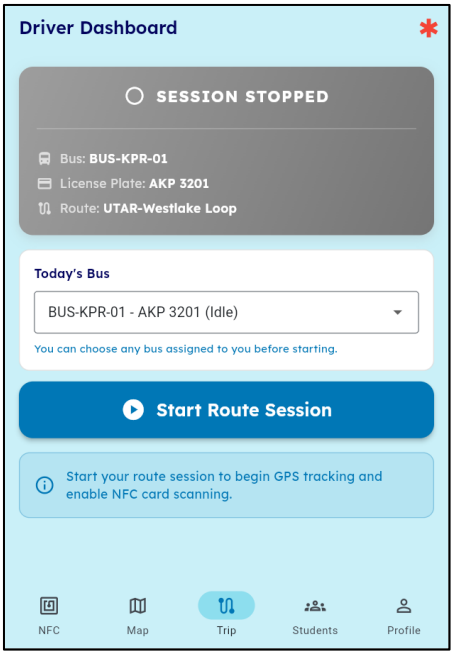


Figure 4.3.2.3.1: Driver trip management screen before departure.

Figure 4.3.2.3.1 shows the driver trip management screen before departure. At this stage, the route session is not yet active, and the driver reviews assigned route information and selected bus details before starting operation. The Start Route Session button is used to begin the trip officially, which also enables subsequent operational features such as GPS tracking and NFC attendance recording. This screen functions as the pre-departure checkpoint to ensure the driver is ready before the route begins.

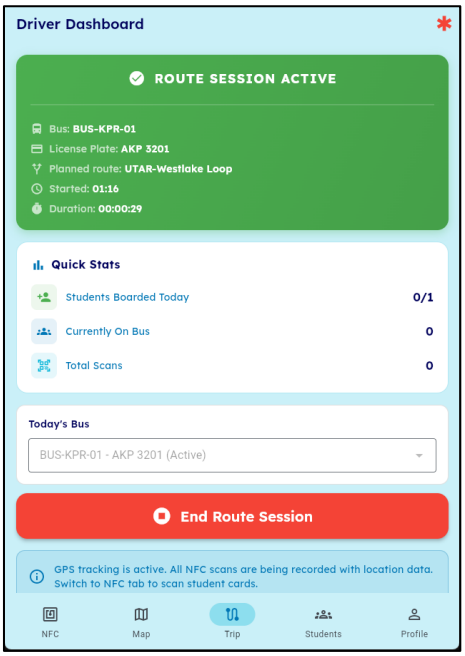


Figure 4.3.2.3.2: Active route session state in driver module.

Figure 4.3.2.3.2 shows the active route session state in the driver module. After the route session is started, the system status changes to active and displays trip details such as route identity and progress context. The End Route Session button allows the driver to close the trip when operation is completed. This screen confirms that the route is currently running and that live operational data can be recorded.

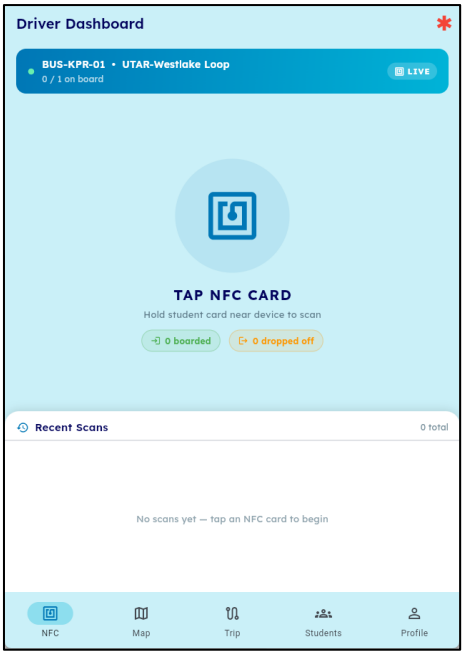


Figure 4.3.2.3.3: NFC scanning interface for boarding and alighting events.

Figure 4.3.2.3.3 shows the NFC scanning interface for boarding and alighting events. This screen is used during pickup and drop-off activities, where the driver scans student NFC cards to record attendance events. The interface provides scan status indicators and a recent scan area so the driver can verify whether each card was processed successfully. This function supports fast and structured boarding/alighting capture during real-time operation.

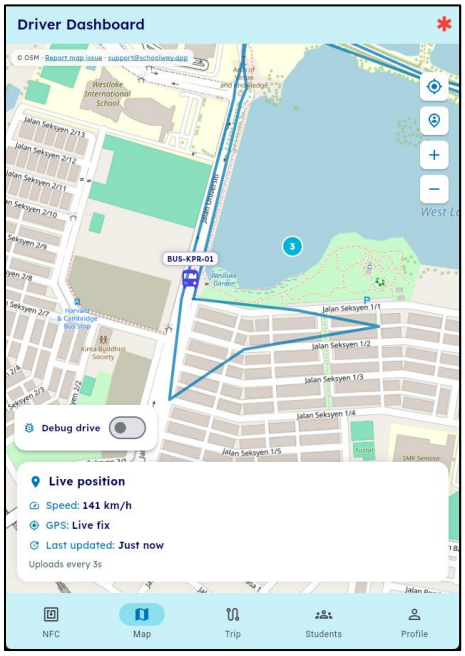


Figure 4.3.2.3.4: Driver live map with route and current position.

Figure 4.3.2.3.4 shows the driver live map with route and current position. During an active route session, this screen visualizes the bus location on the map and displays route movement context. The map helps the driver and system maintain location awareness while position updates are transmitted for monitoring. This screen provides operational visibility and supports real-time fleet tracking functionality.

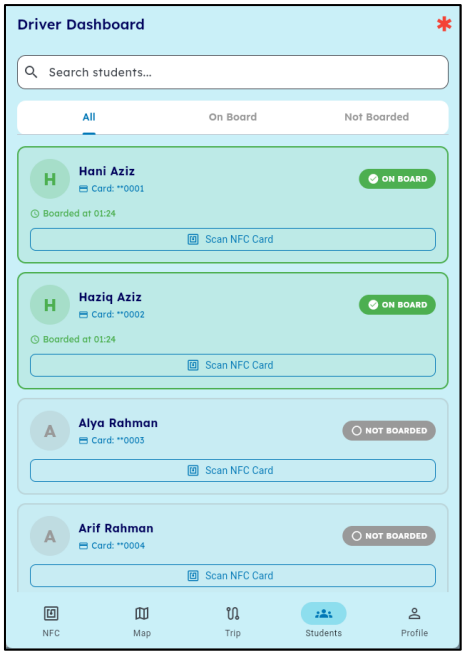


Figure 4.3.2.3.5: Assigned student roster with boarding status indicators.

Figure 4.3.2.3.5 shows the assigned student roster with boarding status indicators. The roster lists students linked to the route and displays their current attendance state, such as on-board or not boarded. This allows the driver to check student boarding progress and identify missing entries quickly. The screen acts as a live attendance checklist that complements NFC scan records throughout the trip.

4.3.2.4 Parent Module Operation

Parents view registered children, monitor live bus location, review trip history, and check notifications.

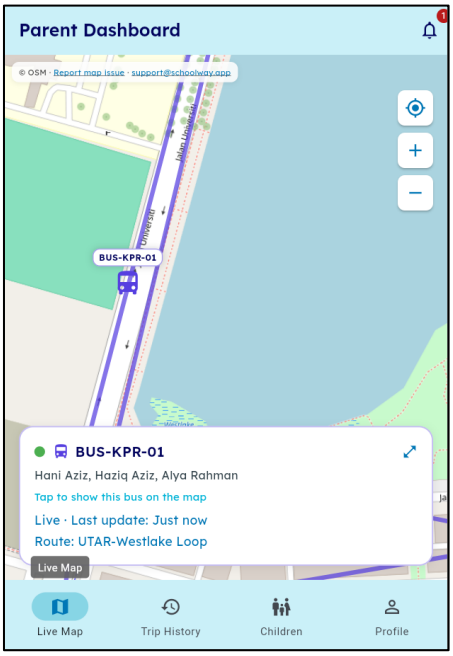


Figure 4.3.2.4.1: Parent dashboard main interface.

Figure 4.3.2.4.1 shows the Parent dashboard main interface. This screen functions as the parent's primary entry point after login and provides quick access to key monitoring features through bottom navigation tabs, including live map, trip history, children, and profile. It gives parents a centralized overview of transport status and allows fast switching between operational views. This interface is designed to support daily monitoring without requiring complex navigation.

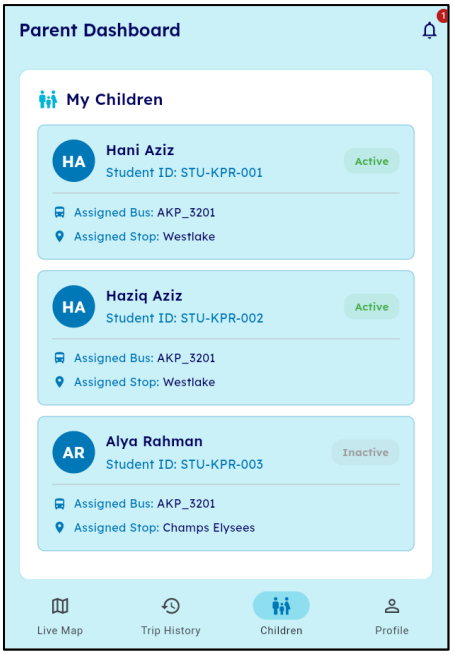


Figure 4.3.2.4.2: Children overview screen in parent module.

Figure 4.3.2.4.2 shows the children overview screen in the parent module. This screen lists the children linked to the parent account and displays their assignment information, such as student identification, assigned bus, and assigned route. It allows parents to verify enrolment and transport linkage details for each child. The status labels also help parents quickly identify whether each child is currently active in the transport workflow.

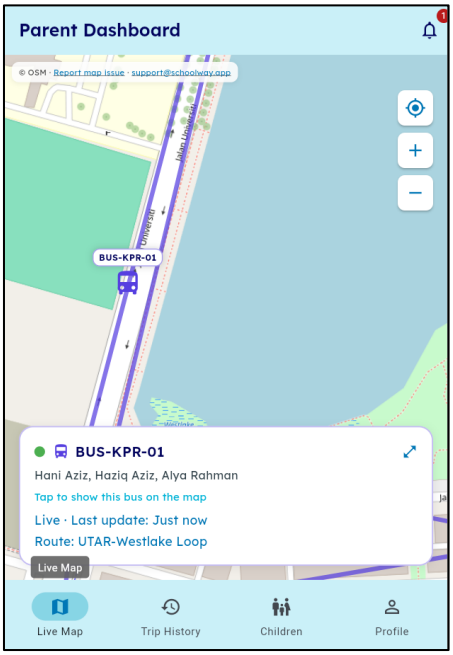


Figure 4.3.2.4.3: Parent live map showing real-time bus location.

Figure 4.3.2.4.3 shows the parent live map with real-time bus location. The map interface displays the current bus position and route context so parents can monitor vehicle movement during pickup and drop-off periods. The information card provides key operational details such as bus identity, route name, and latest location update time. This screen improves visibility and reduces uncertainty by providing near real-time tracking feedback.

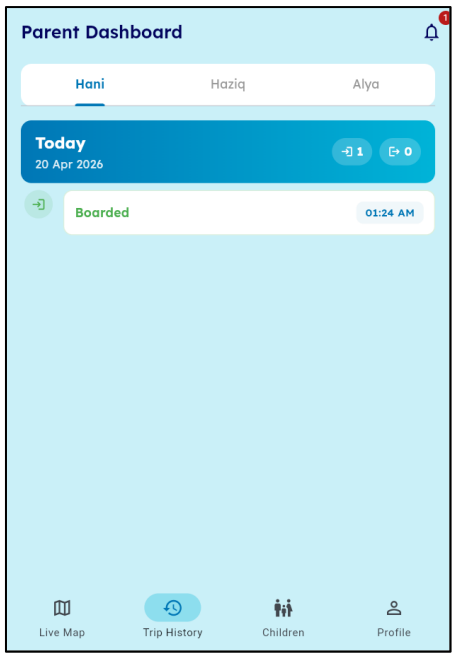


Figure 4.3.2.4.4: Trip history view for student boarding events.

Figure 4.3.2.4.4 shows the trip history view for student boarding events. This screen presents time-based records of transport events, allowing parents to review when a child boarded or exited the bus. The chronological presentation supports attendance traceability and helps parents validate daily transport activity. It also serves as a historical reference for transport-related follow-up when needed.

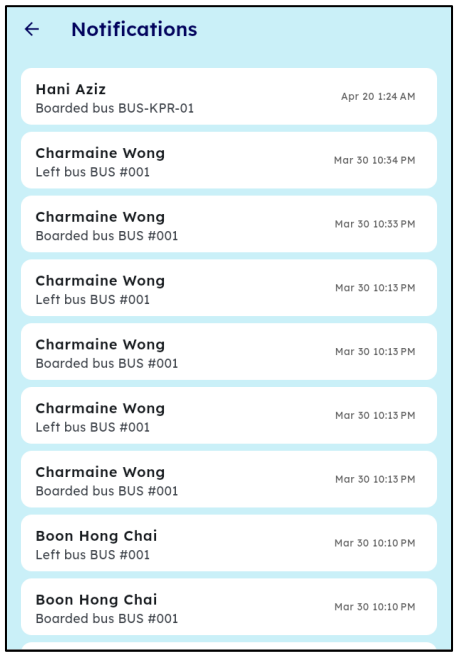


Figure 4.3.2.4.5: Parent notification inbox with event updates.

Figure 4.3.2.4.5 shows the parent notification inbox with event updates. This screen stores event-triggered notifications such as boarding confirmations, drop-off updates, and related transport alerts. Parents can review message content and timestamps even if they miss real-time push notifications. As a result, the inbox acts as a persistent communication log between system events and parent awareness.

4.4 Implementation Issues and Challenges

During implementation, several challenges were identified:

1. Permission orchestration complexity

   Location and notification permissions must be requested and handled safely at startup to avoid runtime failures in operational modules.

2. Web runtime stability considerations

   Firestore persistence behaviour on Web required defensive configuration to avoid assertion-related watch failures in specific update scenarios.

3. Security rule maturity gap

   Current rule design prioritizes development agility with authenticated access. Production hardening requires role-scoped authorization and stricter write constraints.

4. Platform capability differences

   NFC behaviour differs by platform, and unsupported environments require graceful fallback handling.

5. Real-time data consistency under intermittent network

   GPS and event updates require retry-safe logic and resilient UI states when network quality fluctuates.

6. Bulk data quality management

   CSV imports require validation and controlled error handling to prevent malformed records from contaminating operational collections.

Table 4.4: Summary of key implementation challenges and mitigation actions.

Challenge	Impact	Mitigation
Runtime permission handling for location and notifications	Some features cannot start until user grants access	Request essential permissions at startup and provide safe fallback states
Firestore persistence issues on Web	May cause runtime assertion or watch pipeline errors	Disable Firestore persistence on Web before any database use
Simplified Firestore security rules during development	Access control is less strict than production standards	Restrict deployment scope and note that production should use role-based rules or Cloud Functions
Platform differences in NFC support	NFC scanning is not available on every platform	Limit NFC features to supported mobile platforms and provide platform-aware behaviour
Offline synchronization and unstable network	Data such as GPS and boarding events may fail to upload immediately	Queue data locally or retry uploads periodically until connectivity returns

4.5 Concluding Remark

This chapter has presented the complete setup and configuration process, implementation details, and operational behaviour of the SchoolWay system. The application has reached a functional stage with integrated authentication, cloud database operations, live tracking, NFC event capture, and role-based dashboards for administrators, drivers, and parents.

The current implementation is suitable for project 1 demonstration and module-level validation. Future work should prioritize production security hardening, full analytics integration, and release deployment optimization. With these enhancements, the system is positioned to transition from academic prototype to robust real-world deployment.

CHAPTER 5

Conclusion

This project focuses on addressing key limitations in current school transportation management, particularly limited real-time visibility, dependence on manual attendance recording, and communication gaps among parents, drivers, and school administrators. Based on the design and implementation presented in this report, **SchoolWay** is positioned as a practical mobile-based prototype that addresses these issues through integrated digital workflows.

The first objective is to design and develop a role-based mobile school transportation monitoring system. At the current stage, this objective is achieved at prototype level through the implementation of dedicated modules for administrators, drivers, and parents, supported by a centralized cloud backend. The administrator module supports operational monitoring and data management, the driver module supports trip operation and attendance capture, and the parent module provides access to bus location and student transport updates. This role-based structure improves system organization and supports user-specific access to operational information.

The second objective is to implement NFC-based attendance logging for student boarding and alighting events. In the current project I implementation, the NFC workflow is integrated and functional in supported environments, demonstrating the feasibility of replacing manual attendance steps with tap-based event capture. The implemented flow provides timestamped records and improves event traceability. However, full-scale reliability evaluation under broader operational conditions remains part of future work in project II.

The third objective is to integrate real-time GPS tracking into the system. The prototype includes live map visualization and location update mechanisms that allow parents and administrators to monitor bus movement during active operation. This implementation demonstrates how real-time tracking can improve transparency and reduce uncertainty in daily transport monitoring, while further optimization for robustness and deployment readiness is planned for the next phase.

From a technical perspective, the selected stack is suitable for the project scope. Flutter supports efficient cross-platform mobile development, Firebase supports authentication and real-time synchronization, and NFC/GPS integration enables the system's core monitoring functions. As discussed in Chapter 4, the system has reached a functional project I stage and is suitable for demonstration and module-level validation. At the same time, several areas still require enhancement before production deployment, including stronger security hardening, better handling of edge cases, improved offline/retry behaviour, and stricter data validation controls.

In conclusion, this project demonstrates the feasibility and academic value of a unified mobile-based school transportation monitoring approach. At the present stage, SchoolWay serves as a validated prototype that establishes a strong foundation for project II, where emphasis will be placed on deeper testing, refinement, and production-oriented enhancement.

5.1 Project Limitations

Although the current implementation demonstrates the core concept of SchoolWay, several limitations remain at the project I stage.

First, the system is currently validated at prototype and module level, not at full production scale. While core functions are operational, broader real-world testing with larger user groups and longer operational periods is still limited. Therefore, the current findings mainly confirm technical feasibility rather than full deployment readiness.

Second, NFC functionality is platform dependent. Device capability differences and environment constraints may affect user experience and consistency of attendance capture. The implemented workflow demonstrates operational potential, but further cross-device validation is required to ensure stable behaviour under diverse deployment conditions.

Third, real-time tracking performance is dependent on network quality and device status. In unstable connectivity scenarios, latency and synchronization timing may affect how quickly updates appear across modules. Retry and fallback strategies are introduced, but additional robustness testing is still required for field-level reliability.

Fourth, the current security configuration prioritizes development and controlled demonstration. Production-grade hardening, including stricter role-scoped

access control and tighter write constraints, is not yet fully completed in project I. This is acceptable for academic prototyping but remains a key limitation for live deployment.

Fifth, bulk data onboarding (for example, CSV import) still requires stricter validation and exception handling to prevent malformed records from affecting operational collections. Current controls are functional but can be further strengthened for safer administrative usage.

Overall, these limitations do not invalidate the current prototype outcomes. Instead, they define clear technical boundaries of the project I phase and guide the enhancement priorities for project II.

5.2 Future Work for Project II

For project II, future work should focus on completing and refining system functions so that the project demonstrates stronger functional depth and clearer academic outcomes.

First, the attendance module can be expanded with manual fallback and correction features. In addition to NFC scanning, the system can provide controlled manual check-in/check-out by authorized roles when NFC is unavailable. A correction workflow with reason logging can also be added so that attendance amendments are traceable and auditable.

Second, the parent module can be enhanced with smarter notification features. Beyond basic event alerts, the system can support configurable notification categories such as boarding, drop-off, delay, route change, quiet-hour preferences, and per-child alert settings. This enhances usability and enables academic evaluation of notification personalization.

Third, the tracking module can include geo-fence and ETA functions. The system can define key pickup and drop-off zones and trigger events when the bus enters or exits these areas. Estimated time of arrival can also be generated based on current location and route progress to provide more informative parent-side monitoring.

Fourth, the driver module can be extended with trip workflow enforcement features. Functions such as route start/end confirmation, checkpoint progression, student roster completion checks, and trip-close validation can make daily operation more structured and reduce missed process steps.

Fifth, the admin module can be strengthened with operational dashboards and filtering tools. Useful additions include route-based attendance summaries, daily exception lists, for example, missed scans, student event timelines, and searchable logs for faster issue review.

Sixth, the data management module can be improved through stronger import and validation features. CSV preview before commit, field-level error highlighting, duplicate detection, and downloadable import error reports can improve onboarding quality and reduce data inconsistency.

Seventh, the system can include offline-first functional behaviour for key actions. Attendance events and selected trip updates can be queued locally during unstable connectivity and auto synchronized when the network recovers, supported by clear UI indicators for pending and synchronized records.

Eighth, the parent module can be expanded with multi-child and guardian relationship support. This includes linking one guardian account to multiple students and supporting shared guardianship views, reflecting realistic family and transport arrangements.

Ninth, a basic in-app incident reporting function can be introduced for drivers and administrators. Typical events such as late arrivals, route disruptions, or student no-show cases can be logged with timestamps and remarks, creating a more complete operational record within the system.

Overall, project II should prioritize functional enhancement over deployment-oriented concerns. With these additions, the project can provide broader workflow coverage, improved module completeness, and stronger academic evaluation of integrated school transportation monitoring features.

REFERENCES

[1] K. Zeeshan, T. Hämäläinen, and P. Neittaanmäki, “Internet of Things for Sustainable Smart Education: An Overview,” *Sustainability*, vol. 14, no. 7, p. 4293, doi: 10.3390/su14074293.

[2] S. Malathy, P. Ambarish, S. D. Kumar, and G. A. G. Prashanth, “Smart School Bus: To Ensure the Safety of Children,” in *2021 7th International Conference on Advanced Computing and Communication Systems (ICACCS)*, in 2021 7th International Conference on Advanced Computing and Communication Systems (ICACCS), vol. 1. pp. 923–927. doi: 10.1109/icaccs51430.2021.9442044.

[3] D. R. Bolla, B. V Varsha, G. R. C. Kumar, M. D. Prajwal, and B. Sowmya, “Secured Transportation System To Enhance Child Safety,” in *2018 3rd IEEE International Conference on Recent Trends in Electronics, Information & Communication Technology (RTEICT)*, in 2018 3rd IEEE International Conference on Recent Trends in Electronics, Information & Communication Technology (RTEICT), vol. 00. pp. 2314–2318. doi: 10.1109/rteict42901.2018.9012099.

[4] J. T. Raj and J. Sankar, “IoT based smart school bus monitoring and notification system,” in *2017 IEEE Region 10 Humanitarian Technology Conference (R10-HTC)*, in 2017 IEEE Region 10 Humanitarian Technology Conference (R10-HTC). pp. 89–92. doi: 10.1109/r10-htc.2017.8288913.

[5] K. Ammar, M. Jalmoud, A. Boushehri, and K. Fakhro, “A Real-time School Bus Tracking and Monitoring System,” in *2019 IEEE 10th Annual Information Technology, Electronics and Mobile Communication Conference (IEMCON)*, in 2019 IEEE 10th Annual Information Technology, Electronics and Mobile Communication Conference (IEMCON), vol. 00. pp. 0654–0660. doi: 10.1109/iemcon.2019.8936199.

[6] S. Patrakar, S. Wadhai, H. Raghatate, T. Naitam, and A. Seloker, “Student Safety School Bus Tracking System Using Geo-Fencing Technology with Location Alert,” *International Journal of Electrical, Electronics and Computer Systems*, vol. 14, no. 1, pp. 160–165, doi: 10.65521/ijeecs.v14i1.423.

[7] H. Mansor, T. M. A. M. Fadzir, T. S. Gunawan, and Z. Janin, “Safety and security solution for school bus through RFID and GSM technologies,”

Indonesian Journal of Electrical Engineering and Computer Science, vol. 17, no. 2, p. 804, doi: 10.11591/ijeecs.v17.i2.pp804-814.

[8] R. S. Kumar, R. P. Sakthivel, R. Muruganandam, and M. Makarish, "IoT based Bus Tracking System," *International Journal of Innovative Research in Science Engineering and Technology (IJIRSET)*, vol. 14, no. 5, doi: 10.15680/ijirset.2025.1405242.

[9] N. Nayana, J. Tanishka, N. Palkhi, P. Shlok, and Mr.A.D.Talole, "An Android Application for School Bus Tracking and Student Monitoring System," in *2018 IEEE International Conference on Computational Intelligence and Computing Research (ICCIC)*, in 2018 IEEE International Conference on Computational Intelligence and Computing Research (ICCIC), vol. 00. IEEE, pp. 1–4. doi: 10.1109/iccic.2018.8782320.

[10] M. Sarosa, M. T. P. Ningrum, and P. E. Mas'udia, "Design and implementation of school bus information and tracking system application," *Indonesian Journal of Electrical Engineering and Computer Science*, vol. 30, no. 2, pp. 1047–1058, doi: 10.11591/ijeecs.v30.i2.pp1047-1058.

[11] B. Gadade, A. O. Mulani, and A. D. Harale, "IoT Based Smart School Bus and Student Monitoring System," *NATURALISTA CAMPANO*, vol. 28, no. 1, [Online]. Available: https://newapp.readcube.com/library/ff554111-e3a4-4215-9595-1efc61b9db30/item/91d284c5-a952-486b-820d-73de4db114de

[12] M. BENNOUNA, "Smart School Bus Tracking System," *SCHOOL OF SCIENCE & ENGINEERING – AL AKHAWAYN UNIVERSITY*, [Online]. Available: https://cdn.aui.ma/sse-capstone-repository/pdf/fall-2023/Smart%20School%20Bus%20Tracking%20System.pdf

[13] M. V. L. de A. Nascimento and M. O. de Andrade, "School transportation program as means to improve public education in a minor rural town in Northeastern Brazil *," *Ensaio: Avaliação e Políticas Públicas em Educação*, vol. 30, no. 114, pp. 182–206, 2021, doi: 10.1590/s0104-40362021002903093.

[14] M. Ramachandran, R. Sahandi, S. Prakoonwit, and W. Khan, "Intelligent Safety Transport Framework for Schools: A Review of Route Planning and Tracking Systems," *MATEC Web of Conferences*, vol. 81, p. 01007, 2016, doi: 10.1051/matecconf/20168101007.

[15] M. N. Saad, T. Z. T. Muda, N. Oulaganathan, and R. Kaliappan, "Online Student Attendance System Using QR-Code for Kolej Komuniti Segamat," *Advances in Social Science, Education and Humanities Research*, pp. 364–375, 2023, doi: 10.2991/978-2-38476-098-5_32.

[16] C. Nallusamy, R. S. Gowri, and S. H. Priya, "GPS using attendance tracking system," *Challenges in Information, Communication and Computing Technology*, pp. 106–111, 2025, doi: 10.1201/9781003559085-19.

[17] K. Kishore, S. Khare, V. Uniyal, and S. Verma, "Performance and stability Comparison of React and Flutter: Cross-platform Application Development," *2022 International Conference on Cyber Resilience (ICCR)*, vol. 00, pp. 1–4, 2022, doi: 10.1109/iccr56254.2022.9996039.

[18] X. Sun, X. Chen, Y. Liu, J. Grundy, and L. Li, "Taming Android Fragmentation Through Lightweight Crowdsourced Testing," *IEEE Transactions on Software Engineering*, vol. 49, no. 6, pp. 3599–3615, 2023, doi: 10.1109/tse.2023.3266324.

[19] L. Wei, Y. Liu, S.-C. Cheung, H. Huang, X. Lu, and X. Liu, "Understanding and Detecting Fragmentation-Induced Compatibility Issues for Android Apps," *IEEE Transactions on Software Engineering*, vol. 46, no. 11, pp. 1176–1199, 2017, doi: 10.1109/tse.2018.2876439.

[20] V. Şen and M. A. Aydın, "Mobile Device Security Comparison of Different Operating Systems: iOS and Android," *2023 8th International Conference on Computer Science and Engineering (UBMK)*, vol. 00, pp. 141–146, 2023, doi: 10.1109/ubmk59864.2023.10286662.

[21] R. Lane and S. A. Torri, "Comparing iOS and Android Anti-Virus Protection: A Survey," *2024 Ninth International Conference On Mobile And Secure Services (MobiSecServ)*, vol. CFP24RAC-ART, pp. 1–5, 2024, doi: 10.1109/mobisecserv63327.2024.10759904.

[22] S. A. Kinari, N. Funabiki, S. T. Aung, and H. H. S. Kyaw, "A Guided Self-Study Platform of Integrating Documentation, Code, Visual Output, and Exercise for Flutter Cross-Platform Mobile Programming," *Computers*, vol. 14, no. 10, p. 417, 2025, doi: 10.3390/computers14100417.

[23] J. Nanavati, S. Patel, U. Patel, and A. Patel, "Critical Review and Fine-Tuning Performance of Flutter Applications," *2024 5th International Conference on*

Mobile Computing and Sustainable Informatics (ICMCSI), vol. 00, pp. 838–841, 2024, doi: 10.1109/icmcsi61536.2024.00131.

[24] T. Bailey, A. Biessek, and T. Wills, *Flutter for Beginners: An introductory guide to building cross-platform mobile applications with Flutter 2.5 and Dart*. Packt Publishing, 2021. [Online]. Available: http://ieeexplore.ieee.org.libezp2.utar.edu.my/document/10162491

[25] H. Gade, K. Ghogare, A. Ingale, S. Pujari, N. V Puri, and P. S. G. Malvi, "Automated Attendance and Bus Transportation system using NFC and Face Recognition," *Int. J. Innov. Sci. Res. Technol.*, vol. 2, no. 4, 2017, [Online]. Available: www.ijisrt.com

[26] J. Dixon and A. Abuzneid, "An NFC Based Student Attendance Tracking/Monitoring System Using an IoT Approach," *2020 International Conference on Computational Science and Computational Intelligence (CSCI)*, vol. 00, pp. 1082–1086, 2020, doi: 10.1109/csci51800.2020.00201.

[27] L. A. Tawalbeh, A. Basalamah, R. Mehmood, and H. Tawalbeh, "Greener and Smarter Phones for Future Cities: Characterizing the Impact of GPS Signal Strength on Power Consumption," *IEEE Access*, vol. 4, pp. 858–868, 2016, doi: 10.1109/access.2016.2532745.

[28] L. Bouse, S. A. King, and T. Chu, "Simplified Indoor Localization Using Bluetooth Beacons and Received Signal Strength Fingerprinting with Smartwatch," *Sensors*, vol. 24, no. 7, p. 2088, 2024, doi: 10.3390/s24072088.

[29] I. Jan and J. Shafi, "Sqlite- A Better Choice as an Embedded Database for Smart Phones," *International Journal of Allied Practice, Research and Review*, vol. II, pp. 9–15, 2015, [Online]. Available: www.ijaprr.com

[30] V. Raparla, "A Mobile App Development Framework For Database Connectivity And High-Level Services," *All ETDs from UAB. 2789*, 2017, [Online]. Available: https://digitalcommons.library.uab.edu/etd-collection/2789

[31] P. R. Saraf, S. M. Jadhao, S. J. Wanjari, S. G. Kolwate, P. Ankush, and D. Patil, "A Review on Firebase (Backend as A Service) for Mobile Application Development," *International Journal for Research in Applied Science & Engineering Technology (IJRASET)*, vol. 10, 2022, doi: 10.22214/ijraset.2022.39958.

[32] M. Haklay and P. Weber, “OpenStreetMap: User-Generated Street Maps,” *IEEE Pervasive Comput.*, vol. 7, no. 4, pp. 12–18, 2008, doi: 10.1109/mprv.2008.80.

[33] P. Mooney and P. Corcoran, “The Annotation Process in OpenStreetMap,” *Transactions in GIS*, vol. 16, no. 4, pp. 561–579, 2012, doi: 10.1111/j.1467-9671.2012.01306.x.

[34] A. S. Shirazi, N. Henze, T. Dingler, M. Pielot, D. Weber, and A. Schmidt, “Large-scale assessment of mobile notifications,” *Conference on Human Factors in Computing Systems - Proceedings*, pp. 3055–3064, 2014, doi: 10.1145/2556288.2557189.

[35] “SafeStopApp.” Accessed: Apr. 19, 2026. [Online]. Available: https://safestopapp.com/

[36] “Student Attendance & Hall Pass Management.” Accessed: Apr. 19, 2026. [Online]. Available: https://schoolpass.com/solutions/attendance-automation/

[37] “Student Transportation & School Bus Tracking | Zonar.” Accessed: Apr. 19, 2026. [Online]. Available: https://www.zonar.com/industries/pupil-transportation

[38] I. Sommerville, “GlobAl EdiTioN Software Engineering TENTH EdiTioN,” 2016, Accessed: Apr. 19, 2026. [Online]. Available: www.pearsonglobaleditions.com

[39] R. Baxter and R. S. Pressman, “Software Engeneering A Practitioner ’s Approach,” *McGraw-Hill Education.*, p. 83, 2020.

[40] K. Beck *et al.*, “Manifesto for Agile Software Development Twelve Principles of Agile Software,” 2001. [Online]. Available: http://www.agilemanifesto.org

[41] M. Ann. Bopp, D. A. . Bing, and Sheila. Forte-Trammell, *Agile career development : lessons and approaches from IBM*. IBM Press/Pearson, 2010.

[42] “Firebase Documentation.” Accessed: Apr. 22, 2026. [Online]. Available: https://firebase.google.com/docs/

[43] “flutter_map Docs.” Accessed: Apr. 22, 2026. [Online]. Available: https://docs.fleaflet.dev/

[44] “hive docs.” Accessed: Apr. 22, 2026. [Online]. Available: https://pub.dev/documentation/hive/latest/

APPENDIX

A.1 Poster

